

AWS Incident Response Runbooks

Source: <https://github.com/aws-samples/aws-incident-response-runbooks>

License: MIT-0 (Sample Code) and CC BY-SA 4.0 (Documentation)

Total Files: 27

Table of Contents

- [CONTRIBUTING.md](#)
- [README.md](#)
- [playbooks/IRP-CredCompromise.md](#)
- [playbooks/IRP-DataAccess.md](#)
- [playbooks/IRP-DoS.md](#)
- [playbooks/IRP-PersonalDataBreach.md](#)
- [playbooks/IRP-Ransomware.md](#)
- [playbooks/Amazon GuardDuty Playbooks/EKS Findings/IRP-GD-PrivilegeEscalation-Kubernetes_PrivilegedContainer.md](#)
- [playbooks/Amazon GuardDuty Playbooks/EKS Findings/IRP-GD-SuccessfulAnonymousAccess-Kubernetes.md](#)
- [playbooks/Amazon GuardDuty Playbooks/IAM Findings/IAMAnomolousBehavior-Impact.md](#)
- [playbooks/Amazon GuardDuty Playbooks/Malware Findings/EC2MalwareFindings.md](#)
- [ai-playbooks/README.md](#)

- [ai-playbooks/skills/CLAUDE.md](#)
- [ai-playbooks/skills/skill-irp-api-security-breach.md](#)
- [ai-playbooks/skills/skill-irp-builder.md](#)
- [ai-playbooks/skills/skill-irp-credential-compromise.md](#)
- [ai-playbooks/skills/skill-irp-data-access.md](#)
- [ai-playbooks/skills/skill-irp-playbook-factory.md](#)
- [ai-playbooks/skills/skill-irp-ransomware.md](#)
- [ai-playbooks/steering/README.md](#)
- [ai-playbooks/steering/steering-irp-core.md](#)
- [ai-playbooks/steering/steering-factory/steering-factory-creation-guide.md](#)
- [ai-playbooks/steering/steering-factory/steering-irp-builder.md](#)
- [ai-playbooks/steering/reference/steering-irp-api-security-breach.md](#)
- [ai-playbooks/steering/reference/steering-irp-credential-compromise.md](#)
- [ai-playbooks/steering/reference/steering-irp-data-access.md](#)

- [ai-playbooks/steering/reference/steering-irp-ransomware.md](#)

■ CONTRIBUTING.md

Contributing Guidelines

Thank you for your interest in contributing to our project. Whether it's a bug report, new feature, correction, or additional

documentation, we greatly value feedback and contributions from our community.

Please read through this document before submitting any issues or pull requests to ensure we have all the necessary

information to effectively respond to your bug report or contribution.

Reporting Bugs/Feature Requests

We welcome you to use the GitHub issue tracker to report bugs or suggest features.

When filing an issue, please check existing open, or recently closed, issues to make sure somebody else hasn't already

reported the issue. Please try to include as much information as you can. Details like these are incredibly useful:

A reproducible test case or series of steps

The version of our code being used

Any modifications you've made relevant to the bug

Anything unusual about your environment or deployment

Contributing via Pull Requests

Contributions via pull requests are much appreciated. Before sending us a pull request, please ensure that:

1. You are working against the latest source on the *master* branch.
2. You check existing open, and recently merged, pull requests to make sure someone else hasn't addressed the problem already.
3. You open an issue to discuss any significant work - we would hate for your time to be wasted.

To send us a pull request, please:

1. Fork the repository.

2. Modify the source; please focus on the specific change you are contributing. If you also reformat all the code, it will be hard for us to focus on your change.

3. Ensure local tests pass.

4. Commit to your fork using clear commit messages.

5. Send us a pull request, answering any default questions in the pull request interface.

6. Pay attention to any automated CI failures reported in the pull request, and stay involved in the conversation.

GitHub provides additional document on [forking a repository](#) and

[creating a pull request](#).

Finding contributions to work on

Looking at the existing issues is a great way to find something to contribute on. As our projects, by default, use the default GitHub issue labels (enhancement/bug/duplicate/help wanted/invalid/question/wontfix), looking at any 'help wanted' issues is a great place to start.

Code of Conduct

This project has adopted the [Amazon Open Source Code of Conduct](#).

For more information see the [Code of Conduct FAQ](#) or contact

`opensource-codeofconduct@amazon.com` with any additional questions or comments.

Security issue notifications

If you discover a potential security issue in this project we ask that you notify AWS/Amazon Security via our [vulnerability reporting page](#). Please do **not** create a public github issue.

Licensing

See the [LICENSE](#) file for our project's licensing. We will ask you to confirm the licensing of your contribution.

We may ask you to sign a [Contributor License Agreement \(CLA\)](#) for larger changes.

AWS Incident Response Playbook Samples

These playbooks and ai-playbooks are created to be used as templates only. They should be customized by administrators working with AWS to suit their particular needs, risks, available tools and work processes. These guides are not official AWS documentation and are provided as-is to customers using AWS products and who are looking to improve their incident response capability.

The playbooks and ai-playbooks included cover several common scenarios faced by AWS customers. They outline steps based on the [NIST Computer Security Incident Handling Guide](#) (Special Publication 800-61 Revision 3) that can be used to:

Gather evidence

Contain and then eradicate the incident

Recover from the incident

Conduct post-incident activities, including post-mortem and feedback processes

The playbooks are markdown files designed to be used by humans as guidance when responding to an incident in their AWS environments. The ai-playbooks are designed to be consumed by an integrated development environment "IDE" leveraging a large language model "LLM" as "steering files" or "skills" (depending on your chosen IDE and LLM). Additional information for these files (including usage) are in the [README](#) in that directory.

Interested readers may also find the AWS Security Incident Response Guide (first published in June 2019) a useful guide as an overview of how the below steps were created.

Each playbook corresponds to a unique incident and there are 5 parts to handling each incident type, following the NIST guidelines referenced above. Each part corresponds to an action in that NIST document.

It is not sufficient to customize these scenarios to the need of your customers, organization or applications. It is important that these playbook scenarios are tested (for example, in Game Days) prior to deployment to your knowledge management system and that all responders are familiar with the actions required to respond to an incident.

Note that some of the incident response steps noted in each scenario may incur costs in your AWS account(s) for services used in either preparing for, or responding to incidents. Customizing these scenarios and testing them will help you to determine if additional costs will be incurred. You can use AWS Cost Explorer and look at costs incurred over a particular time frame (such as when running Game Days) to establish what the possible impact might be. Note also that if you have a subscription with an IDE or LLM, using the AI playbooks will also use tokens, contributing to whatever your token usage limitations are based on the plan(s) you have.

Usage

The playbooks are written in markdown to facilitate editing and consumption into a variety of user systems.

Security

See [CONTRIBUTING](#) for more information.

License Summary

The documentation is made available under the Creative Commons Attribution-ShareAlike 4.0 International License. See the LICENSE file.

The sample code within this documentation is made available under the MIT-0 license. See the LICENSE-SAMPLECODE file.

■ `playbooks/IRP-CredCompromise.md`

Incident Response Playbook Template

Incident Type

Credential Leakage/Compromise

Introduction

This playbook is provided as a template to customers using AWS products and who are building their incident response capability. You should customize this template to suit your particular needs, risks, available tools and work processes.

Security and Compliance is a shared responsibility between you and AWS. AWS is responsible for “Security of the Cloud”, while you are responsible for “Security in the Cloud”. For more information on the shared responsibility model, [please review our documentation](#).

You are responsible for making your own independent assessment of the information in this document. This document: (a) is for informational purposes only, (b) references current AWS product offerings and practices, which are subject to change without notice, and (c)

does not create any commitments or assurances from AWS and its affiliates, suppliers or licensors. This document is provided “as is” without warranties, representations, or conditions of any kind, whether express or implied. The responsibilities and liabilities of AWS to its customers are controlled by AWS agreements, and this document is not part of, nor does it modify, any agreement between AWS and its customers.

Summary

This Playbook

This playbook outlines response steps for Credential Leakage/Compromise incidents. These steps are based on the [NIST Computer Security Incident Handling Guide](#) (Special Publication 800-61 Revision 2) that can be used to:

Gather evidence

Contain and then eradicate the incident

Recover from the incident

Conduct post-incident activities, including post-mortem and feedback processes

Interested readers may also refer to the [AWS Security Incident Response Guide](#) which contains additional resources.

Once you have customized this playbook to meet your needs, it is important that you test the playbook (e.g., Game Days) and any automation (functional tests), update as necessary to achieve the desired results, and then publish to your knowledge management system and train all responders.

Note that some of the incident response steps noted in each scenario may incur costs in your AWS account(s) for services used in either preparing for, or responding to incidents. Customizing these scenarios and testing them will help you to determine if additional costs will be incurred. You can use [AWS Cost Explorer](#) and look at costs incurred over a particular time frame (such as when running Game Days) to establish what the possible impact might be.

In reviewing this playbook, you will find steps that involve processes that you may not have in place today. Proactively preparing for incidents means you need the right resource configurations, tools and services in place that allow you to respond to an incident.

The next section will provide a summary of this incident type, and then cover the five steps (parts 1 - 5) for handling credential compromise.

This Incident Type

Credential compromise occurs when credentials related to one or more of your IAM principals (such as an IAM user or role) have been obtained by an actor not authorized to use them. This means that the actor is either *not* the IAM user whom the credentials identify, or is not authorized to assume the IAM role that a set of temporary credentials are associated with. Once a malicious actor has obtained a set of credentials, they can use those credentials to perform any action that is allowed by IAM policies associated with those credentials. This may include the ability to elevate privilege if those allowed actions include various IAM API actions, such as `PassRole`, `CreateUser` or `AttachPolicy`. *As a preventative measure, you can use AWS IAM Permission Boundaries and AWS Organizations Service Control Policies (SCPs) to scope down user permissions and prevent potential privilege escalations.*

Incident Response Process

Part 1: Acquire, Preserve, Document Evidence

1. You become aware of potential indicators of compromise (IoCs). These could come in various forms:

- An internal ticketing system (the sources of the ticket are varied and could include any of the means below)
- A message from a contractor or third-party service provider
- From an alert in one of your monitoring systems either inside or external to AWS (for example, in AWS, in this particular situation, it could be via an Amazon GuardDuty finding, either directly in GuardDuty, or via AWS Security Hub, or it could be via a [CloudWatch metric indicating a change in IAM](#) or from your own systems ingesting AWS data)
- Alarms or observations that resources have been created in your account that cannot be accounted for in your CMDB, exist in regions that you do not operate infrastructure in, or themselves have generated alerts ([Amazon Detective](#) is a useful tool for understanding these relationships)

- *Via an anonymous tip*
- *Via independent or external security researchers*
- *Where credentials have been compromised, you are less likely to be contacted via the threat actor, as in this situation typically they want to maximize their use of resources and minimize the chance of being discovered, unless the compromise is being used to exfiltrate data or cause some other kind of public incident such as through the modification of your publicly facing resources*

2. Confirm a ticket/case has been raised for the incident. If not, manually raise one

3. Determine and begin to document any end-user impact/experience of the issue. From a user's perspective, for this type of scenario, there may be no direct user impact. Findings should be documented in the ticket/case related to the incident

4. In the case of automatically created tickets/cases, determine what internal alarms/metrics are currently indicating an issue (what caused the ticket to be created?) This may be an Amazon CloudWatch metric indicating an API was called that included a reference to an AWS policy, or an AWS Config rule that indicates some aspect of your AWS Identity and Access Management (IAM) configuration has fallen out of compliance, or an Amazon GuardDuty alert that indicates possible credential compromise. It may also be a billing alert where your billing costs have passed a predetermined threshold, triggering an alarm and notification. Determine exactly which of these has caused the ticket/case to be raised

5. Determine the set of credentials that have been compromised. Note that there will be additional steps in Part 3 to deal with scenarios where threat actors have planted resources to gain an account foothold.

1. If an internal ticket/case has been created, review the case to determine if the user/role name or ARN, user or role ID, or Access Key ID are provided in that ticket. If it is not, reference the alert specified in the ticket (for example, a GuardDuty finding) and go to point 2., below

2. If the alert has come from GuardDuty or AWS Security Hub, locate the specific event in the service's AWS console and then locate the Access Key ID for the impacted credentials. In GuardDuty, this will be located under the "Resource" section of the finding. "resourceType" should be Access Key and there will be a field named accessKeyDetails that will contain the Access Key ID, Principal ID, User Type and additional information

6. Determine the likely time at which the credentials were compromised (any API actions taken post that time should be considered as malicious and any resources created post that time should be considered compromised). As an example, for a GuardDuty finding, note the "eventFirstSeen" field in the "Service" section of the finding.

7. If there is service disruption for your application occurring (see point 2, above), determine if there are any known events that could be causing that disruption that may not be related to the credential leakage (deployed CRs or other application modifications, for example). Check the deployment pipeline to determine if any changes have been made leading up to the event (the time the event was first recorded, or the time the automated system cut the ticket/case)

8. Incident Communications:

1. Identify stakeholder roles from the application entry in the Configuration Management Database (CMDB) entry for that application, or via the application's risk register

2. Open a conference bridge war room for the incident

3. Notify identified stakeholders including (if required) legal personnel, technical teams and developers and add them to the ticket and the war room, so they are updated as the ticket is updated

9. External Communications:

1. Ensure your organizations legal counsel is informed and is included in status updates to internal stakeholders and especially in regards to external communications.

2. For colleagues in the organization that are responsible for providing public/external communication statements, ensure these internal stakeholders are added to the ticket so they receive regular status updates regarding the incident and can complete their own requirements for communications within and external to the business.

3. If there are regulations in your jurisdiction requiring reporting of such incidents, ensure the people in your organization responsible for notifying local or federal law enforcement agencies are also notified of the event/added to the ticket. Consult your legal advisor and/or law enforcement for guidance on collecting and preserving the evidence and chain of custody.

4. There may not be regulations, but either open databases, government agencies or NGOs may track this type of activity. Your reporting may assist others

Part 2: Contain the Incident

The immediate task will be to disable compromised credentials or revoke permissions associated with those credentials, thereby preventing any further API activity using the compromised credentials.

1. For the compromised credential identified from Part 1, disable those credentials

1. If they are long term IAM user credentials, [disable them using the IAM console or API](#)

2. If they are short term credentials obtained via the AWS Security Token Service (AWS STS) they will be associated with an IAM role. There are a couple of options available to disable these:

1. [Revoke all current role sessions](#) (note that if the threat actor has the ability to obtain new credentials, this won't solve the problem)

2. If the threat actor is able to obtain another set of credentials and the activity continues, it will then be necessary to remove all IAM policies attached to the role, modify the attached policies to block all access, or modify the role's trust policy to prevent the actor from assuming the role. As the credentials will remain valid for the specified time duration once issued, it is important to note that modifying the trust policy will allow any current valid credentials to continue to be used whilst still valid

Note that the above actions (step 2, substeps 1 and 2) will stop all users from using credentials obtained by assuming the role, including any legitimate users or

applications

2. The compromised credentials should now be disabled. Verify this by checking the AWS CloudTrail console for the next 30 minutes or so for ongoing credential use, whether by access key, IAM user, or Role.

Part 3: Eradicate the Incident

Now it is time to further investigate what API actions the compromised credentials have performed since the compromise occurred, and then determine if additional resources need to be deleted, terminated, or investigated further. From Part 1, you have identified the compromised credentials, and the likely time of compromise. It will now be necessary to determine what API actions have been performed and what resources have been created, deleted or modified after that time.

1. Using your preferred monitoring tool, access CloudTrail and search for all API actions performed by the compromised credentials, from the date and time of compromise through to the current time:

1. If this tool is a third-party tool such as Splunk, Sumo Logic or others, follow the normal procedure for obtaining log information from that tool

2. If you do not ingest CloudTrail logs into a third-party tool, but do send those logs to Amazon Simple Storage Service (Amazon S3), you will be able to use Amazon Athena to query the logs. The remaining steps will focus on AWS tools to retrieve the necessary information

2. *Create an Athena table referencing the bucket containing your CloudTrail logs that link also includes example queries that can be run in Athena*

3. *In the Athena console, run a query that shows all API actions taken by the compromised credentials post-compromise date/time*

4. *From the resulting list, determine which API calls:*

- *Accessed sensitive data (such as S3 GetObject)*
- *Created new AWS resources, such as databases, Amazon Elastic Compute Cloud (Amazon EC2) instances, AWS Lambda functions or S3 buckets, etc.*
- *Services that create resources should also be carefully checked [for], these include Amazon EC2 Auto Scaling Groups, Amazon EC2 Spot Fleets, Amazon Elastic Kubernetes Service (Amazon EKS)/Amazon Container Service (Amazon ECS) clusters, etc.*
- *Created or modified AWS identity resources that could be used to extend a foothold into the account (or other accounts, for example with AWS Security Token Service (AWS STS) API methods such as AssumeRole). Within an account, API methods including (but not limited to) the following should also be investigated:*
 - *CreateUser*

- *CreateRole*
- *AssumeRole*
- *GetToken*
- *AttachPolicy*
- *RunInstances* (especially with *PassRole*)
- *Image*
- *Provider*
- *Tag*
- *Untag*
- *Create*
- *Delete*
- *Update*
- etc.

4. Deleted existing AWS resources

5. Modified existing AWS resources

5. Based on the results of the previous step, determine if any applications are potentially impacted:

1. Obtain the ARN and/or tag information for each resource impacted (from step 4, above)

2. Go back to the CMDB and determine which application that resource belongs to

3. Notify the application owner based on the results of the above steps

6. Based on the above results, if additional credential-obtaining resources have been created (IAM users, roles, EC2 instances with roles, etc.), take the following steps:

1. Disable and/or delete any credentials for those resources, as per the steps outlined in Part 2, Step 1

2. Use your preferred logging tool to analyze CloudTrail again, this time specifying the additionally discovered credentials as a query parameter, as outlined in Part 3, steps 1 through to 6 (this step). Continue to iterate through this process until you discover that there were no further resources created capable of obtaining valid credentials for the AWS account.

3. For each set of newly discovered compromised credentials, go through the following steps from 7 in Part 3 (below) and all relevant steps in Part 4, also below

7. For new resources created during the compromise: Delete all resources created by the compromised credentials that were created post the date/time of the compromise occurring

8. For handling resources that were modified or deleted during the compromise, see Part 4

Part 4: Recover from the Incident

1. For resources that were modified during the compromise:

1. If the resource can be destroyed and replaced, do so. For example, EC2 instances in an EC2 Auto Scaling group, with a Launch Configuration referencing a non-compromised AMI, or Launch Templates used to create EC2 instances, terminate the instance, allow EC2 Auto Scaling to add a new one

2. If the resource cannot be replaced, either:

1. Restore the resource from a known good backup, or;

2. Prepare a new resource and configure it into the application's infrastructure, while isolating the compromised resource and removing it from the application's infrastructure. Update the CMDB accordingly

3. Either destroy the compromised resource, or continue to leave it isolated for post-incident forensics

2. For resources that were deleted during the compromise:

1. Determine what (if any) application the resource belonged to, by checking in the CMDB, or confirming the resource's tag(s) (Check AWS Config if the tags aren't listed in the CloudTrail entry and the [resource is supported by AWS Config](#))

2. If the deleted resource can be restored from backup, commence the restore procedure

3. If the deleted resource cannot be restored from backup, consult the CMDB to obtain the resource's configuration, and recreate the resource and configure it into the application's infrastructure

Part 5: Post-Incident Activity

This activity contains two parts. Firstly, some compromised resources may require forensic analysis, either to fulfil regulatory obligations or improved incident handling, both taking input from the root cause analysis that will result from forensic investigation. The second part is a “sharpen the saw” activity which helps teams to assess their response to the actual incident, determine what worked and what didn't, update the process based on that information and record these findings.

Firstly, perform any required forensic investigation to determine (for compromised resources) what methods the actors may have used and to determine if additional risks and risk mitigations are required for the resources and/or applications in question.

1. For any compromised resources that have been isolated for further analysis, perform the forensic activity on those resources and incorporate the findings into the post-incident report.
2. Ensure that the CMDB is correctly updated to reflect the current status of all resources and applications impacted

Secondly, review the incident itself and the response to it, to determine if anything needs to be changed for handling any similar incidents in the future.

1. Review the incident handling and the incident handling process with key stakeholders identified in Part 1, Step 8.
2. Document lessons learned, including attack vector(s) mitigation(s), misconfiguration, etc.

3. Store the artifacts from this process with the application information in the CMDB entry for the application and also in the CMDB entry for the credential compromise response process.

■ playbooks/IRP-DataAccess.md

Incident Response Playbook Template

Incident Type

Unintended access to an Amazon Simple Storage Service (Amazon S3) bucket

Introduction

This playbook is provided as a template to customers using AWS products and who are building their incident response capability. You should customize this template to suit your particular needs, risks, available tools and work processes.

Security and Compliance is a shared responsibility between you and AWS. AWS is responsible for “Security of the Cloud”, while you are responsible for “Security in the Cloud”. For more information on the shared responsibility model, [please review our documentation](#).

You are responsible for making your own independent assessment of the information in this document. This document: (a) is for informational purposes only, (b) references current AWS product offerings and practices, which are subject to change without notice, and (c) does not create any commitments or assurances from AWS and its affiliates, suppliers or licensors. This document is provided “as is” without warranties, representations, or conditions of any kind, whether express or implied. The responsibilities and liabilities of AWS to its customers are controlled by AWS agreements, and this document is not part of, nor does it modify, any agreement between AWS and its customers.

Summary

This Playbook

This playbook outlines response steps for unintended access to an Amazon Simple Storage Service (Amazon S3, or S3) bucket. These steps are based on the [NIST Computer Security Incident Handling Guide](#) (Special Publication 800-61 Revision 2) that can be used to:

Gather evidence

Contain and then eradicate the incident

recover from the incident

Conduct post-incident activities, including post-mortem and feedback processes

Interested readers may also refer to the [AWS Security Incident Response Guide](#) which contains additional resources.

Once you have customized this playbook to meet your needs, it is important that you test the playbook (e.g., Game Days) and any automation (functional tests), update as necessary to achieve the desired results, and then publish to your knowledge management system and train all responders.

Note that some of the incident response steps noted below may incur costs in your AWS account(s) for services used in either preparing for, or responding to incidents. Customizing this runbook and testing it will help you to determine if additional costs will be incurred. You can use [AWS Cost Explorer](#) and look at costs incurred over a particular time frame (such as when running Game Days) to establish what the possible impact might be.

In reviewing this playbook, you will find steps that involve processes that you may not have in place today. Proactively preparing for incidents means you need the right resource configurations, tools and services in place that allow you to respond to an incident.

The next section will provide a summary of this incident type, and then cover the five steps (parts 1 - 5) for handling unintended access to an S3 bucket.

This Incident Type

By default, all S3 buckets are private and can be accessed only by users that are explicitly granted access. Sometimes, customers will make changes to their bucket configuration to meet their requirements, or the requirements of their application. These changes (such as changes to a bucket policy, for example) may result in unintended access being granted to IAM principals or unauthenticated users. This playbook covers steps that can be used to deal with unintended access.

Incident Response Process

Part 1: Acquire, Preserve, Document Evidence

1. You become aware that there has been a possible unintended data access to an Amazon S3 bucket. This information could come via different means, for example:

- An internal ticketing system (the sources of the ticket are varied and could include any of the means below)
- A message from a contractor or third-party service provider
- From an alert in one of your own monitoring systems either internal or external to AWS. For example, in AWS, this might include an AWS Config managed rule, AWS CloudTrail via Amazon EventBridge or Amazon CloudWatch Events and Amazon Simple Notification Service (Amazon SNS), via Amazon GuardDuty, AWS Security Hub or a similar service
- From a threat actor (for example, requesting a ransom or they will disclose data)
- From a security researcher

- Via an anonymous tip
- From a public news article in the press, on a blog or in the news

2. At this point, you may not know if the issue is due to a mis-configured bucket, or a set of compromised credentials:

1. Use [AWS Identity and Access Management \(IAM\) Access Analyzer](#) to determine if any S3 buckets may have overly permissive configuration, giving unintended access to unauthorized principals

2. Use IAM Access Analyzer to determine if any IAM role trust policy provides unintended access to principals in the same or other AWS accounts

3. [Review CloudTrail logs](#) to determine if recent (prior to the date on which the unintended access occurred, if known) changes had been made to bucket configuration, such as the bucket's Block All Public Access settings, object ACLs, or bucket policies. You may choose to do this using a tool such as [Amazon Athena](#)

4. Review CloudTrail logs to determine if recent (prior to unintended access date, if known) changes have been made to IAM role trust policies

3. If you are already aware of the S3 bucket(s) involved, Firstly move to **Part 2** to contain the incident. Once that is done, return here and then move on to step 5. If you have not established which bucket(s) are involved, continue to step 4.

4. If you do not know which bucket is involved:

1. If you do not know which bucket is involved, but do know which data is involved:

1. Use internal tools that links that data to a bucket, such as checking a Configuration Management Database (CMDB)

2. Use the [S3 ls command](#) from the [AWS Command Line Interface \(AWS CLI\)](#) to list out the contents of your S3 bucket(s). For buckets with many objects, you can filter by specifying specific key prefixes in the ls command. This will be especially useful if you already know the data objects involved.

3. There are several common third-party tools that can also be used to search though files in Amazon S3

4. Query CloudTrail Data Event logs for one of the files in question, and determine the name of the bucket from the CloudTrail event. [You can do this using Amazon Athena](#) This will also provide you the identity of the principal making the API call

2. If you do not know specifically which bucket or which data was involved, there are several options:

1. If you know the set of credentials involved, search CloudTrail **Data Events** using a tool such as Amazon Athena, filtering on that set of credentials and [s3.amazonaws.com](#) as the event source. If you do not have an Amazon Athena table already configured, you can [quickly set one up from the AWS CloudTrail console](#).

2. Review [CloudTrail Insights](#) (this must be enabled prior to the event) to determine if any insights have been generated that may relate to the time and date of the unintended access, and determine which resources were involved by drilling down into the events highlighted by the specific insight. This will also provide the user identity of the principal that accessed the

object(s).

3. Simply go to the Amazon S3 console and review the “**Access**” column in the table listing all S3 buckets (under the **Buckets** menu). If the column lists a value of **Public** for a bucket, this means that unauthenticated users have access to objects in the bucket, by virtue of object ACLs or the bucket’s bucket policy. This is a useful approach for a single account.

4. If you want to check across multiple accounts in an AWS Organization, you can use Amazon Macie. Go to the Macie console and select **Summary** from the navigation menu. Review the percentage of buckets that are publicly accessible under the **Public** heading of the first table. Next, review the number of buckets that are shared with other AWS accounts by reviewing the information under the **Shared** heading of the first table.

5. If the buckets are under your administrative control (either in an AWS account you control, or in an AWS account for which you have authorization and appropriate access, for example via an IAM Role), move ahead to **Part 2** to contain the incident, then return to this point and move on to step 7. If they are not under your administrative control, move on to step 6

6. If you do not have administrative control of the S3 buckets involved, you will need to establish a communication channel with the provider/team/individual that does. This may involve opening a new case on the provider’s service platform, contacting their customer service, directly contacting your technical point of contact (PoC) in that organization, or something similar. Do this as a matter of urgency, as in Part 2, below, you will need to request that they take action to revoke the changes giving access.

7. Confirm a ticket/case has been raised for the incident. If not, manually raise one.

8. Determine if any user cases are already open for the bucket(s) that can be correlated to any notifications or data you already have in relation to the incident. Document any noted

end-user impact/experience of the issue in the relevant ticket. These may include (but not be limited to):

1. Files missing from the bucket
2. Unfamiliar new files in the bucket
3. Files that have had their access settings (such as ACLs) changed, in particular where those files have been made public
4. Modified bucket permissions, ACLs, Block Public Access settings (all less likely to be noted by regular users, but possible)
9. In the case of automatically created tickets/cases, determine what internal alarms/metrics are currently indicating an issue (what caused the ticket to be created? - see 1a, above)
10. Determine the application impacted (if any, this may be done quickly via Resource Tags, or by using your CMDB)
11. Determine if there are any known events that could be causing service disruption (for example, an application change roll-out that is now resulting in the mishandling API interactions with the S3 bucket.) This could include but may not be limited to:
 1. Writing incorrect files to the bucket
 2. Incorrectly deleting files from the bucket

3. Modifying file or bucket permissions, ACLs, or Block Public Access settings

12. Determine for the bucket and/or data in question, what is the [data classification](#) of the data that has been (or allegedly has been) accessed? The classification level may determine the incident response path taken, however the remainder of this playbook will assume that the classification of the data indicates some level of business impact (whether reputational, financial or other). This classification may be recorded in your CMDB or a specific data classification document.

13. Incident Communications:

1. Identify stakeholder roles from the application entry in the Configuration Management Database (CMDB) entry for that application, or via the application's risk register

2. Open a conference bridge war room for the incident

3. Notify other applicable internal stakeholders including legal personnel, technical teams and developers as required, and add them to the ticket and the war room, so they are updated as the ticket is updated

14. External Communications (for the relevant team/people, not necessarily the first responder - most likely legal, PR, Technical/Account Managers, C-levels, etc.):

1. Identify customers that could have/are impacted by the incident

2. Identify customer data that could have been accessed as a result of the issue

3. Compile this information and *securely* distribute it to the relevant people (for example, using a corporate file share where permissions and access can be controlled)

4. Ensure your organizations legal counsel is informed and is included in status updates to internal stakeholders and especially in regards to external communications.

5. If there are regulations in your jurisdiction requiring reporting of such incidents, ensure the people in your organization responsible for notifying local or federal law enforcement agencies are also notified of the event. Consult your legal advisor and/or law enforcement for guidance on collecting and preserving the evidence and chain of custody.

Those teams/individuals will also likely identify public domain material (news articles, blog posts, tweets, etc.) that mention the incident and determine if/how to respond to these (some factors to consider include: Are the materials factually correct? How much information can be shared and at what time so as not to compromise any ongoing or future investigations? Are there any potential legal ramifications that need to be considered when responding? etc.). This is not a task for the first responder and should not distract that person from the immediate task of handling the technical aspects of the incident.

6. There may not be regulations, but either open databases, government agencies or NGOs may track this type of activity. Your reporting may assist others.

Part 2: Contain the Incident

By now, ideally, you should have confirmed whether there has been an unintended access incident and whether the incident relates to a set of leaked credentials, misconfigured bucket policies, modified object ACLs, or a combination of these. Firstly, if compromised credentials were involved, disable those credentials or revoke permissions associated with those credentials, thereby preventing any further API activity using the compromised credentials. Next, move on to bucket policies (if relevant). Finally, move on to Block Public Access settings.

1. For the **compromised credential** details obtained from reviewing CloudTrail logs in Part 1, disable those credentials if they are from AWS accounts under your administrative control

1. If they are long term IAM user credentials, [disable them using the IAM console or API](#)

2. If they are short term credentials obtained via AWS Security Token Service (AWS STS) they will be associated with an IAM role. There are a couple of options available to disable these:

1. [Revoke all current roles sessions](#) (note that if the actor has the ability to obtain new credentials, this won't solve the problem)

2. If the actor is able to obtain another set of credentials and the activity continues, it will then be necessary to remove all IAM policies attached to the role, modify the attached policies to block all access, or modify the role's trust policy to prevent the actor from assuming the role. As the credentials will remain valid for the specified time duration once issued, it is important to note that **only** modifying the trust policy will allow any current valid credentials to continue to be used whilst still valid

Note that the above actions will stop all users from using credentials obtained by assuming the role, including any legitimate users or applications

3. If the role is attached to an EC2 instance, you will also need to review credentials related to the instance (for example, private/public key pairs used for SSH) to ensure that these have not also been compromised, allowing an actor to access an S3 bucket using the role credentials attached to the instance. Two key vectors:

1. A user obtains access to an EC2 instance using a set of credentials (private key) that allows them to SSH to the instance

2. The application on the EC2 instance is not correctly configured, and the instance/application form a reverse proxy, or behind certain types of web application firewalls, or has a role that is directly or indirectly internet-facing and a malicious user performs a Server-side Request Forgery (SSRF) with the goal of having the instance's application simply repeat the request to S3 and return the output. Correct application configuration and EC2 features such as [IMDSv2](#) can help mitigate these risks

2. The compromised credentials should now be disabled. Verify this by checking the CloudTrail console for the next 30 minutes or so for ongoing credential use, whether by access key, IAM user, or Role.

3. If the credentials used to obtain data are not under your administrative control, and the access was obtained using credentials (i.e., **not** unauthenticated access) the **bucket policy** is too permissive. IAM roles or users from other AWS accounts also need permissions to access a bucket in your account via the bucket policy. Therefore, it is necessary to modify the bucket's bucket policy to restrict access to those principals:

1. From the bucket(s) identified in Part 1, review the bucket policies to determine which principals have access to the buckets for data plane and/or management plane operations:

1. **Do this first:** You will need to modify control plane access (for example, S3 PutBucketPolicy) to prevent unauthorized users from making changes to the bucket's configuration (bucket policies, bucket ACL, bucket or account level Block Public Access settings)

2. You will need to modify data plane access Put/Get to prevent unauthorized users from accessing or modifying data that they should not have access to, for all applicable read and write operations

3. For specific objects of concern, identify any AWS accounts or predefined groups listed as having permission in the [object's ACL](#) (IAM users cannot be listed individually in an ACL). Another approach (if your permission model does not involve ACLs) is to modify the ACLs for all objects in the bucket to remove all other accounts or predefined groups and simply apply FULL_CONTROL to the bucket owner only (not the object owner - potentially from a different AWS account)

4. If the issue is that the bucket allows public (unauthenticated) access to objects, either via ACLs, the bucket policy ("principal": "") or a combination thereof, such settings will need to be modified:

1. the quickest way to remove this access is to modify the bucket's **Block All Public Access** configuration. However note this will impact **all** objects in the bucket, not just the one(s) you are concerned with

2. For each bucket, in the Amazon S3 console, go to the object, select it and then click on **Permissions >> Access Control List**. There will be a note on the Access Control List button indicating public access. Check the **Everyone** radio button under the **Public access** row and then remove the checks in the boxes in the ACL's settings. Finally, click **Save**. Note that this doesn't modify an individual existing object's ACL

3. For each object, in the Amazon S3 console, go to the object, select it and then click on **Permissions**. Under the **Public access** row, select the **Everyone** radio button and uncheck all the boxes in the resulting object ACL properties box and click **Save**. If many objects are involved, it will be quicker to use either the [AWS CLI](#) or one of the [AWS SDKs](#).

Part 3: Eradicate the Incident

Key components of the eradication process include the following:

1. Mitigate any vulnerabilities that were exploited

2. Rotate compromised passwords/credentials

Most of the steps in this section are taken from the [security best practices for Amazon S3](#). Check back with that document frequently for updates.

*1. Eradicate any attack vectors related to systems writing to or reading from the S3 bucket(s). As per **Part 2**, if an actor has gained access to systems that are writing to a bucket, the actor could now effectively use the instance's credentials (via the AWS CLI or AWS SDK) on an application instance, for example, to read data from the bucket:*

1. [Configure IMDSv2](#) on all application instances/role holding instances

2. [Rotate SSH credentials](#) for EC2 Linux. Consider the following steps:

1. Determine which instances are using that key pair: `aws ec2 describe-instances --query "Reservations[].Instances[].[KeyName, InstanceId]"` (Windows CMD example)

1. Decide on the following:

- If the instances should be terminated*

1. Establish if the instances can be terminated without impacting your application (for example, using your CMDB, your EC2 instance tags, etc.)

- Is it sufficient to rotate an instance's user credentials by:
- Modifying the contents of the `authorized_keys` file; or,
- Removing the user; and/or,
- Remove/modify the user's `.ssh` directory
- On Windows, modify the allowed RDP users via Remote Desktop Users.

At scale, use [AWS Systems Manager](#) to manage instance configuration.

2. If the instances have been launched using some kind of scaling mechanism:

1. Determine if you have any [Launch Configurations](#) referencing the key pair. If so, create either a new Launch Configuration referencing the new key pair and, associate it to the required resources, such as EC2 Auto Scaling Groups, or instead move on to step 4 and create a launch Template, giving you all the current features for EC2 Auto Scaling

2. Determine if you have any existing [Launch Templates](#) for EC2 referencing the existing key pair. If so, delete that template, follow the steps below and then create a new template once a new key pair has been created.

5. Delete the key pair from your EC2 console

6. Terminate the instances

7. Create a new key pair in AWS and issue it to any required personnel

8. Launch instances using the new key pair

9. Update user passwords for domain or local users using Microsoft Windows Remote Desktop Protocol (RDP) and verify users in the Remote Desktop Users group within Windows and/or Group Policy (for example, if you are adding Active Directory Global Security Groups to the Remote Desktop Users group on local operating systems, a fairly common Group Policy task)

3. [Revoke credentials for all existing role sessions](#). This will block access to AWS APIs from the EC2 instance until that instance obtains a new set of role credentials. The instance can obtain new role credentials by waiting until the existing set of credentials are rotated automatically, or by detaching and re-attaching the same role to the EC2 instance. If there are concerns that a threat actor has remote access to the instance (for example, by SSH or RDP), per step 2 in this section, terminate the existing EC2 instance and create a new one after revoking existing sessions. Note that if the instance is accessed using [Key Pairs defined in EC2](#) you should determine if other instances are using the same key pairs and take appropriate action to prevent the threat actor also accessing those instances. Such steps might include:

2. Mitigate vulnerabilities related to your service configurations. For example, with S3, determine which configuration items need to be updated:

1. Return modified object ACLs to their secure settings, removing unauthenticated access permissions

2. Implement least privilege in bucket policies

3. Ensure only appropriate principals have permission to perform data or control plane operations on the bucket

4. Ensure IAM policies are appropriately scoped and if necessary, [Permission Boundaries](#) and/or [Service Control Policies](#) have been deployed appropriately

5. Encrypt data at rest using [Amazon S3 Server Side Encryption](#) - this protects the data at rest, and means that principals must have permissions in **both** S3 and AWS Key Management Service (AWS KMS) (the AWS KMS Customer Master Key (CMK) key policy) to access the data

6. [Enable versioning](#) in S3 to enable data recovery, and consider:

7. [Object Lock](#) to prevent the deletion of critical data

3. If credential compromise was involved:

1. Flag for review the process for handling user credentials during Move/Add/Change (MAC) procedures for the post-incident activity

2. The compromised credentials should have been disabled in Part 2, you may now consider issuing new credentials to impacted users
3. Close down/remove user accounts or roles if they are not valid accounts/roles (or IAM IdPs, AWS Single Sign-On (AWS SSO) instances, etc.)
4. Enable [MFA Delete for S3](#), which means users must provide a valid MFA token along with the request to delete any versioned object in the bucket. This requires also enabling S3 Versioning
5. If you have objects that need to be public, rather than setting the ACL so that the object is world-readable, consider using an S3 pre-signed URL with a suitable expiry time and have your application pass that to the unauthenticated user to allow them to access the object.

Part 4: Recover from the Incident

The next step is to determine if the incident has been mitigated, or if additional tuning is needed to eradicate it. This includes reviewing pre- and post-mitigation logs to:

Determine the impact of the mitigations already performed

Identify any other attack signatures to attempt to eradicate or further mitigate the incident

Once this has been completed, it will likely be necessary to restore any lost or corrupted data.

Restore lost or modified data:

1. If you have [S3 Versioning](#), [restore an earlier version of the object](#) to replace the modified object. If the object has been deleted using a simple delete command (without specifying a version ID for the object, it will now have a [delete marker](#) in place of the object. you can [delete the delete marker](#) to recover the object
2. If the object was deleted and the delete command specified an object Version ID, that version of the object will be permanently deleted. You will need to establish if previous versions of the object still exist, or if the object can be retrieved from another location (see below steps)

3. If you move data to other S3 storage classes (S3 IA, Amazon Simple Storage Service Glacier, etc.) as part of a back-up or archival process, determine if the files still exist in that storage class and retrieve them (remember that files *can be deleted* from other storage classes). [Lifecycle policies](#) are used to move objects between storage classes

4. If you are using S3 Cross Region Replication (CRR) restore the data from the target bucket to the source bucket as necessary

5. If none of the previous actions allow you to restore the deleted or modified data, you will need to determine if the data is retrievable from another source (such as an on-premise system or an S3 bucket in a different account)

From the logs obtained in PART 1, review those obtained at the time of the initial investigation. Now that mitigations are in place, you need to go back and obtain those metrics and logs again, and compare them to the incident metrics and logs obtained earlier.

If data plane activity has returned to pre-attack levels and the logs show no further evidence of malicious use, the attack has been mitigated (at least for now). Continue to monitor the service post-attack; if suspicious data plane activity reoccurs, take the following steps:

1. Return to Part 1 and follow those steps again

2. Review logs and related data to determine if you have correctly identified the attack vector, or if the actor has changed the vector in response to the mitigation

Part 5: Post-Incident Activity

This “sharpen the saw” activity helps teams to assess their response to the actual incident, determine what worked and what didn’t, update the process based on that information and record these findings.

1. Review how the incident was handled, as well as the incident handling process generally, with key stakeholders identified in Part 1.
2. Document lessons learned, including attack vector(s) mitigation(s), misconfiguration, etc.
3. Store the artifacts from this process with the application information in the CMDB entry for the application and also in the CMDB entry for the S3 bucket and the data leak incident response process.

4. Update risk documents based on any newly discovered threat/vulnerability combinations that were discovered as a result of lessons learned

5. If new application or infrastructure configuration is required to mitigate any newly identified risks, conduct these change activities and update application and component configuration in the CMDB

6. Ask the following questions and assign people to resolve any issues that come up as a result:

1. What information would have helped me respond to this incident more swiftly?

2. What detection would have alerted me to the issue sooner?

3. What configuration (technical or process) would have prevented this data being exposed in this way?

4. What automation, tooling or access would have made it easier to investigate the root cause?

7. For any actions that are raised as the result of parts 3, 4 and 5, assign those actions and follow up to ensure they are completed

Incident Response Playbook Template

Incident Type

Web Application Dos/DDoS Attack

Introduction

This playbook is provided as a template to customers using AWS products and who are building their incident response capability. You should customize this template playbook to suit your particular needs, risks, available tools and work processes.

Security and Compliance is a shared responsibility between you and AWS. AWS is responsible for “Security of the Cloud”, while you are responsible for “Security in the Cloud”. For more information on the shared responsibility model, [please review our documentation](#).

You are responsible for making your own independent assessment of the information in this document. This document: (a) is for informational purposes only, (b) references current AWS product offerings and practices, which are subject to change without notice, and (c) does not create any commitments or assurances from AWS and its affiliates, suppliers or licensors. This document is provided “as is” without warranties, representations, or conditions of any kind, whether express or implied. The responsibilities and liabilities of AWS to its customers are controlled by AWS agreements, and this document is not part of, nor does it modify, any agreement between AWS and its customers.

Summary

This Playbook

This playbook outlines response steps for Web Application Dos/DDoS Attack incidents. These steps are based on the [NIST Computer Security Incident Handling Guide] (<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf>) (Special Publication 800-61 Revision 2) that can be used to:

Gather evidence

Contain and then eradicate the incident

recover from the incident

Conduct post-incident activities, including post-mortem and feedback processes

Interested readers may also refer to the [AWS Security Incident Response Guide](#) which contains additional resources.

Once you have customized this playbook to meet your needs, it is important that you test the playbook (e.g., Game Days) and any automation (functional tests), update as necessary to achieve the desired results, and then publish to your knowledge management system and train all responders.

Note that some of the incident response steps noted in each scenario may incur costs in your AWS account(s) for services used in either preparing for, or responding to incidents. Customizing these scenarios and testing them will help you to determine if additional costs will be incurred. You can use [AWS Cost Explorer](#) and look at costs incurred over a particular time frame (such as when running Game Days) to establish what the possible impact might be.

In reviewing this playbook, you will find steps that involve processes that you may not have in place today. Proactively preparing for incidents means you need the right resource configurations, tools and services in place that allow you to respond to an incident.

The next section will provide a summary of this incident type, and then cover the five steps (parts 1 - 5) for handling DoS and DDoS attacks.

This Incident Type

DoS and DDoS attacks are “denial of service” attacks, designed to prevent legitimate users from accessing your web applications. They target different layers of the OSI model and often exploit protocol weaknesses. Other attacks may simply use legitimate requests to your web application, but increase the volume of those requests to a point where your web application is overwhelmed and no longer able to service legitimate requests. When you use AWS services, you benefit from protections built in to those services. Additionally, if you follow the [AWS Best Practices for DDoS Resiliency](#), you will increase the availability of your application and its ability to withstand DDoS attacks.

Incident Response Process

Part 1: Acquire, Preserve, Document Evidence

1. You become aware that your application is not meeting availability requirements. This may mean that the application is unavailable, is providing a degraded service, or even appears to be functioning, but application performance metrics or logging are suggesting that there may be an issue. This information could come via different means, for example:

- Customer contact/feedback/support calls
- An internal ticketing system (the sources of the ticket are varied and could include any of the means below)
- A message from a contractor or third-party service provider
- From an alert in one of your own monitoring systems either internal or external to AWS (for example, in AWS, in this particular situation, it could be via CloudWatch metrics triggering an alarm from Amazon EC2, AWS Application Load Balancer, Amazon CloudFront, AWS WAF or other services that you have configured CloudWatch alarms for)
- Via an anonymous tip

- Via independent or external security researchers

2. Confirm an internal ticket/case has been raised for the incident within your organization. If not, manually raise one.

3. Determine and begin to document end-user impact/experience of the issue. This should be documented in the ticket/case related to the incident

4. In the case of automatically created tickets/cases, determine what internal alarms/metrics are currently indicating an issue (what caused the ticket to be created?)

5. Determine the application impacted (this may be done quickly via Resource Tags)

6. Determine if there are any known events that could be causing disruption to your application (increased traffic due to a sales event, or similar)

7. Check your Configuration Management Database (CMDB) to determine if there were any deployments to your production environment (code, software or infrastructure updates) that may result in an increase in traffic to the affected URL(s)

8. Incident Communications:

1. Identify stakeholder roles from the application entry in the CMDB entry for that application, or via the application's risk register

2. Open a conference bridge war room for the incident

3. Notify identified stakeholders including (if required) legal personnel, public relations, technical teams and developers and add them to the ticket and the war room, so they are updated as the ticket is updated

9. External Communications:

1. Ensure your organizations legal counsel is informed and is included in status updates to internal stakeholders and especially in regards to external communications.

2. For colleagues in the organization that are responsible for providing public/external communication statements, ensure these internal stakeholders are added to the ticket so they receive regular status updates regarding the incident and can complete their own requirements for communications within and external to the business.

3. If there are regulations in your jurisdiction requiring reporting of such incidents, ensure the people in your organization responsible for notifying local or federal law enforcement agencies are also notified of the event/added to the ticket. Consult your legal advisor and/or law enforcement for guidance on collecting and preserving the evidence and chain of custody.

4. There may not be regulations, but either open databases, government agencies or NGOs may track this type of activity. Your reporting may assist others

10. Determine the resources involved in serving the application (start with front-end resources, including CDNs, load balancers and front-end application servers, then move to supporting services, such as databases, caches, etc.)

11. Obtain the application's documented baseline and locate the metrics for standard application performance from the CMDB.

12. Determine if the application's resources are currently displaying metrics outside the baseline:

1. For Amazon CloudFront, check the following metrics:

1. Confirm if there is a significant variation in:

1. [Data transferred](#) over HTTP and/or HTTPS

2. [Total number](#) of HTTP requests

2. If monitoring CloudFront access logs, determine if there has been a spike in 4xx or 5xx HTTP return codes (in Amazon CloudWatch, All >> CloudFront >> Per-Distribution Metrics) with a Metric Name of 5xxErrorRate or 4xxErrorRate

2. For Application Load Balancers, [check the following metrics](#), which may give further information on the type of attack, and how it is impacting your application:

1. Confirm if there is a significant variation in:

1. HTTPCode_ELB_4xx_Count (indicators of attacks that may involve credential stuffing, malformed or incomplete requests, or overly aggressive website scraping, etc.)

2. HTTPCode_ELB_2xx_Count (a dramatic increase may indicate a HTTP flood attack)

3. ActiveConnectionCount (indicates total concurrent TCP connections to ALB, when used with SUM)

4. ClientTLSNegotiationErrorCount (this may indicate a protocol exploitation attack by abusing the SSL handshake process, such as sending only weak ciphers in a request)

5. RequestCount (a significant increase in requests may indicate a HTTP flood attack)

6. HTTPCode_Target_5XX_Count (as this is a response dedicated by a host or container behind the load balancer, this may indicate that the server is unable to respond to the number of received requests, this may also be the result of impacted services downstream, such as a supporting database)

7. TargetResponseTime (increasing response times may indicate server resources are not available to serve requests from the load balancer)

8. ELBAuthFailure (indicates that user authentication could not be completed, as the IdP denied access to the user, or a user attempted to replay an already used auth token)

2. Note the metrics of concern

3. For Amazon Elastic Compute Cloud (Amazon EC2) instances, check the [following metrics](#):

1. Confirm if there is significant variation in:

1. CPU utilization

2. Network utilization

3. Disk performance

4. Memory utilization (if using the [CloudWatch Logs Agent](#))

2. Note the metrics of concern

13. Obtain logs relevant to the incident (these will be the logs from the resources identified above, in point 4), these should include the following (note that these logs should be already configured to be sent to CloudWatch Logs):

1. Web server access logs

1. RHEL-based - `/var/log/apache/access.log`, or

2. Debian-based - `/var/log/apache2/access.log`

2. Load balancer logs:

1. Application Load Balancer Logs will be configured for the ALB and [sent to an Amazon Simple Storage Service \(Amazon S3\) bucket](#). If you are not sure which bucket the logs are being sent to, [use this document to help you determine the bucket name](#).

2. CloudFront Distribution logs. CloudFront will be configured to [send its access logs to an S3 bucket](#).

14. Once you have verified you have access to the logs, use your preferred tool (for example, Amazon Athena, internal, or third-party tools) to review the logs and determine if there are any identifiable attack signatures. Such signatures may be the following:

1. An unusual number of requests within a given time period

2. An unusual set of source IPs

3. A set of unrecognized URLs in the request section

4. POST or Query parameters that are not expected or supported by the application

5. Unexpected HTTP verbs (GET, where we expect POST, etc)

6. An unusually high number of 2xx, 4xx or 5xx responses

15. From the log dive outlined in previous steps, determine the likely attack vector (HTTP flood, SYN flood, credential stuffing, UDP reflection attack, etc.)

16. What resources are being targeted? Note the Amazon resource names (ARN)

17. Note specific information about the service hosted on the AWS resources For example, what type of application is it? What is the provenance of the application code? Are there other dependencies on this application?

Part 2: Contain the Incident

From the Acquire, Preserve, Document Evidence phase, you should now have a good idea of the attack vector. The following steps describe how to help to mitigate the attack.

1. If it is a single EC2 instance serving the public content:

1. determine from the application's developer whether the application is "cloud native" (can handle things such as Amazon EC2 Auto Scaling , for example as application instances spin up and down).

2. Determine the AMI ID for the EC2 instance(s) identified during the evidence phase and determine if this is an approved image for this application

3. If there is no pre-defined image for spinning up additional EC2 instances, consult with the application developer whether an image can be created from the existing instance, and additional instances spun up from that image. If this is possible, proceed with those steps

4. From EC2 metrics in CloudWatch outlined in the evidence phase (Part 1), determine if the most suitable [EC2 instance type](#) is in use and/or if additional instances should be added behind an [AWS elastic load balancer](#) to spread the load

5. If so, this will allow you to scale additional EC2 instances using [Amazon EC2 Auto Scaling](#), and put those instances behind an appropriate load balancer

6. Update DNS records in your Hosted Zone to point to the load balancer. For security groups on the EC2 instance, allow access only from the load balancer's security group, and do not allow direct public access from any port or protocol

2. If it is a group of EC2 instances behind a load balancer:

1. Determine if Amazon EC2 Auto Scaling is in use.

1. If not, create an Auto Scaling Group, define a [Launch Configuration](#) or [Launch Template](#) and add the existing instances to the Auto Scaling Group

2. If it is in use, move to step (2b).

2. Confirm with the relevant stakeholders that CloudFront can be used and;

3. [Deploy a CloudFront distribution](#), using the load balancer as an origin for the distribution

3. If it is a load balancer (or EC2 instance) behind a CloudFront distribution

1. Determine if it is a single EC2 instance serving as an origin:

1. If so, go to step 1 and work through those steps

2. If it is not, verify the following:

1. There is a load balancer

2. The target groups are spread across multiple Availability Zones

3. The EC2 instances are managed by EC2 Auto Scaling

3. If any of the points in (ii) are not true, go back and work through step 2, above

4. Finally, ensure that the security groups for the load balancer are appropriately configured. This can be done automatically by using AWS Lambda and the AWS published IP ranges for CloudFront. There is a [Git Repository with code that will set this up for you](#) and maintain the ranges if there are changes to the public IP ranges published.

Part 3: Eradicate the Incident

By following steps 1 through to 4 in PART 2, the service is now behind a CloudFront distribution, in addition to the standard protections provided by [AWS Shield](#) . Now further refine protections by using AWS WAF.

1. Determine if the CloudFront distribution or Application Load Balancer are protected by an AWS WAF WebACL.

1. For a CloudFront distribution:

1. Go to the CloudFront console and click on the name of the relevant distribution

2. Under the **General** tab, make note of the value in the AWS WAF Web ACL field

3. Go to the AWS WAF and Shield console and select Web ACLs from the navigation pane

4. Locate the Web ACL with the same name as noted from step (ii) and click on the name of the Web ACL listed under **AWS WAF**

2. For an Application Load Balancer:

1. Go to the EC2 console and select “Load Balancers” from the “Load Balancing” menu in the navigation pane

2. Select the name of the relevant Application Load Balancer and in the tabs below, select “Integrated services”

3. Note the name of the Web ACL listed under **AWS WAF**

3. Review the Web ACL and add rules if required;

1. The quickest solution is to deploy a set of AWS WAF Managed Rules. These are available in the AWS Marketplace for [third-party providers](#). There are also AWS-Managed rules available directly from AWS when you [create a new Web ACL](#).

2. Alternatively, if the application developers know what type of rules should be deployed to protect the application, work with those developers to [create the rules](#), deploy the Web ACL and then go back to step (a), above.

1. If it is determined that the CloudFront distribution or Application Load Balancer are not protected by a Web ACL, take one of the two following steps:

1. If a Web ACL already exists and your CMDB shows that this Web ACL should be associated to either the CloudFront Distribution or the Application Load Balancer, go to the AWS console or AWS Command Line Interface (AWS CLI) tools and associate that resource to the Web ACL.

2. If no reference exists to a Web ACL in the CMDB that should be associated to either the CloudFront Distribution or Application Load Balancer, there are now two possible options:

1. If the CMDB contains references to known Web ACL rules that should be contained in a Web ACL, go ahead and create those rules and add them to a new Web ACL. Finally, associate that Web ACL to the resources to be protected.

2. If the CMDB does not contain references to Web ACL rules that should be deployed, the quickest solution is to deploy a set of AWS WAF Managed Rules. These are available in the AWS Marketplace for [third-party providers](#). There are also AWS-Managed rules available directly from AWS when you [create a new Web ACL](#).

3. Alternatively, if the application developers know what type of rules should be deployed to protect the application, work with those developers to [create the rules](#), deploy the Web ACL and then go back to step (a), above.

Part 4: Recover from the Incident

The next step is to determine if the attack has been mitigated, or if additional tuning is needed to eradicate the attack. This includes reviewing pre- and post-mitigation logs to:

Determine the impact of the mitigations already performed

Identify attack signatures to attempt to eradicate or further mitigate the attack

From points (9) and (10) in PART 1, review the logs obtained once the incident was detected. Now that mitigations are in place, you need to go back and obtain those metrics and logs again, and compare them to the incident metrics and logs obtained earlier. If request activity has returned to baseline levels, and the service is confirmed to be available, the attack has subsided: Continue to monitor the service post-attack. If the service request rate is still elevated and the service is available, the attack has been mitigated: Continue to monitor the service for potential change in attack method. If the service is unavailable, return to Part 1 and review logs and related data to determine if you have correctly identified the attack vector, or if the attacker has changed the vector in response to the mitigation, or if your mitigations need to be refined.

Part 5: Post-Incident Activity

This “sharpen the saw” activity helps teams to assess their response to the actual incident, determine what worked and what didn’t, update the process based on that information and record these findings.

1. Review the incident handling and the incident handling process with key stakeholders identified in Part 1, Step 6.
 2. Document lessons learned, including attack vector(s) mitigation(s), misconfiguration, etc.
 3. Store the artifacts from this process with the application information in the CMDB entry for the application and also in the CMDB entry for the DDoS Response process.
 4. Update risk documents based on any newly discovered threat/vulnerability combinations that were discovered as a result of lessons learned
 5. If new application or infrastructure configuration is required to mitigate any newly identified risks, conduct these change activities and update application and component configuration in the CMDB
-

Incident Response Playbook Template

Incident Type

Personal Data Breach

Introduction

This playbook is provided as a template to customers using AWS products and who are building their incident response capability. You should customize this template to suit your particular needs, risks, available tools and work processes.

Security and Compliance is a shared responsibility between you and AWS. AWS is responsible for “Security of the Cloud”, while you are responsible for “Security in the Cloud”. For more information on the shared responsibility model, [please review our documentation](#).

You are responsible for making your own independent assessment of the information in this document. This document: (a) is for informational purposes only, (b) references current AWS product offerings and practices, which are subject to change without notice, and (c) does not create any commitments or assurances from AWS and its affiliates, suppliers or licensors. This document is provided “as is” without warranties, representations, or conditions of any kind, whether express or implied. The responsibilities and liabilities of AWS to its customers are controlled by AWS agreements, and this document is not part of, nor does it modify, any agreement between AWS and its customers.

Summary

This Playbook

This playbook outlines response steps for Personal Data Breach incidents. These steps are based on the [NIST Computer Security Incident Handling Guide](#) (Special Publication 800-61 Revision 2) that can be used to:

Gather evidence

Contain and then eradicate the incident

Recover from the incident

Conduct post-incident activities, including post-mortem and feedback processes

Interested readers may also refer to the [AWS Security Incident Response Guide](#) which contains additional resources.

Once you have customized this playbook to meet your needs, it is important that you test the playbook (e.g., Game Days) and any automation (functional tests), update as necessary to achieve the desired results, and then publish to your knowledge management system and train all responders.

Note that some of the incident response steps noted in each scenario may incur costs in your AWS account(s) for services used in either preparing for, or responding to incidents. Customizing these scenarios and testing them will help you to determine if additional costs will be incurred. You can use [AWS Cost Explorer](#) and look at costs incurred over a particular time frame (such as when running Game Days) to establish what the possible impact might be.

> **Warning**

>

> Besides the procedures outlined in this playbook, your jurisdiction may have requirements and/or legislation about privacy laws (e.g. General Data Protection Regulation (GDPR), California Consumer Privacy Act (CCPA) etc). It may require you to report a security breach and the suspected or confirmed loss or theft of any material or records data. We recommend establishing if this applies to your organization before operationalising this playbook.

In reviewing this playbook, you will find steps that involve processes that you may not have in place today. Proactively preparing for incidents means you need the right resource configurations, tools and services in place that allow you to respond to an incident.

The next section will provide a summary of this incident type, and then cover the five steps (parts 1 - 5) for handling credential compromise.

This Incident Type

A data breach occurs where there is an unauthorised access to or disclosure of personal information, or information is lost in circumstances where unauthorised access or disclosure is likely that could result in harm or inconvenience, such as fraud or identity theft, to an individual.

What is personal data?

All information that can identify an individual is personal data, directly or indirectly. One example is a person's name, but also things such as login-ID, birthdate, address, email-address, phone number, social-security number, etc. The concept of personal data goes even further: behavioral or performance data of an individual person (like geo-location, performance-metrics, working hours, device IDs, etc.) are all personal data. The rule of thumb is that if you're thinking about whether some element could be personal data—it usually is! You should verify whether your systems handle or store personal data. If you have internal teams that can help you with this (legal, policy) check with them. If you do not, you can create and document your own definition based on existing definitions today, such as [Article 4 of the GDPR](#), or the [definition used in various NIST Special Publications](#). Commonly, the personal data referred to in this playbook is known as Personally Identifiable Information (PII).

[Art. 4 \(1\) of GDPR](#) defines personal data as:

(1) 'personal data' means any information relating to an identified or identifiable natural person ('data subject'); an identifiable natural person is one who can be identified, directly or indirectly, in particular by reference to an identifier such as a name, an identification number, location data, an online identifier or to one or more factors specific to the physical, physiological, genetic, mental, economic, cultural or social identity of that natural person.

Incident Response Process

Part 1: Acquire, Preserve, Document Evidence

>Note

>

> It is important to exercise caution when collecting information during a PII data breach, as personal information must not be recorded. Instead, focus on collecting contextual information surrounding the breach to aid in the investigation.

1. You become aware that there has been a possible unintended personal data breach. This information could come via different means, for example:

1. PII was accidentally logged or found in the reporting systems, such as application log files, of your security operations.

2. An internal ticketing system (the sources of the ticket are varied and could include any of the means below)

3. A message from a contractor or third-party service provider

4. From an alert in one of your own monitoring systems, either internal or external, to AWS. For example, in AWS, this might include an AWS Config managed rule, AWS CloudTrail via Amazon EventBridge or Amazon CloudWatch Events and Amazon Simple Notification Service (Amazon SNS), via Amazon Macie, Amazon GuardDuty, AWS Security Hub or a similar service.

5. From an alert in one of your data loss prevention (DLP) systems

6. From a threat actor (for example, requesting a ransom or they will disclose data)

7. From a security researcher

8. Via an anonymous tip

9. From a public news article in the press, on a blog, or in the news

2. Confirm a ticket/case has been raised for the incident. If not, manually raise one.

3. Determine the sensitivity of the impacted data and record specific AWS resources that were the origin of the breach. It is also essential to determine the number of individuals who were affected by the incident with certainty (*Contextual information is often a key factors in determining the severity of an incident within an Organization*):

1. *Nature of personal data:* Your organization's [data classification](#) standards should also define how to handle a Personal Identifiable Information (PII) data breach. The steps or actions you take may vary depending on the sensitivity/classification of the data involved in the breach, you will refer to your data classification and handling policies to determine this. These standards determine the necessary level of protection for different types of information based on their sensitivity. For example, the level of seriousness for a data breach involving sensitive personal health information of an individual would be greater than that of a breach involving publicly available data:

1. According to your jurisdiction legal requirements and data classification specifications, it may be necessary to categorize breached data as Personally Identifiable Information (PII) and Publicly Available Information (PAI). Here are few examples:

- You should consider the breached personal data as PII unless proven otherwise.
- A common example of PAI is information that is readily accessible to the public, such as a person's name, address, and phone number which is listed in a public directory or on social media platforms.

2. *Number of AWS resources involved:* Below are a few potential scenarios:

1. An Amazon S3 bucket or an Amazon RDS database was compromised, containing sensitive information such as social security numbers, credit card numbers, banking

information, or medical records.

2. An Amazon Elasticsearch index was lost or compromised, containing personally identifiable information (PII) that should not be publicly accessible.

3. An Amazon S3 bucket containing personally identifiable information (PII) was compromised and is publicly available. Here are two example scenarios:

- The bucket was made publicly accessible when it should not have been.
- The bucket may not have been publicly accessible, but data was still exfiltrated due to credential leakage or an incorrectly configured system that accessed the data and made it publicly available.

3. *Number of individual affected:* This information may help your organization to assess the impact on affected individuals and take appropriate measures to mitigate the consequences, such as offering identity theft protection services or providing timely notifications to prevent further harm.

4. The collection of evidence (AWS CloudTrail logs, AWS Config rules finding etc.) in relation to AWS data services must adhere to a strict chain of custody to ensure the integrity and authenticity of the data as per your jurisdiction legal requirements. Evidence and artifacts can consist of, but aren't limited to:

1. All EC2 instance metadata

2. Amazon EBS disk snapshots

3. EBS disks streamed to S3

4. Memory dumps

5. Memory captured through hibernation on the root EBS volume

6. CloudTrail logs

7. AWS Config rule findings

8. Amazon Route 53 DNS resolver query logs

9. VPC Flow Logs

10. AWS Security Hub findings

11. Elastic Load Balancing access logs

12. AWS WAF logs

13. Custom application logs

14. System logs

15. Security logs

16. Any third-party logs

5. At this point, you may not know the cause of data breach:

1. Extrusion by attackers —an attackers penetrated the security perimeter and,gain access to sensitive personal data.

2. Insider threats — a malicious insider, or an attacker who has compromised a privileged user accounts, abuses their permissions and attempts to move data outside the organization.

3. Unintentional or negligent data exposure — an employees who lose sensitive data in public, provide open Internet access to data, or fail to restrict access per organizational policies.

6. If you are already aware of AWS resources involved (like Amazon S3 or Amazon RDS etc) and , Firstly move to **Part 2** to contain the incident. Once that is done, return here and then move on to step 5. If you have not established which bucket(s) are involved, continue

to step 4.

7. If you are *not aware* of the AWS resources involved incident personal data breach within an AWS account, follow these steps:

1. Check AWS services used for data storage and processing, such as Amazon S3, Amazon RDS, Amazon DynamoDB, etc.
2. Employ the use of internal tools to associate the data with a specific storage location, such as by checking a Configuration Management Database (CMDB).
3. Review AWS CloudTrail logs for suspicious activity or unauthorized access.
4. Examine the findings of Amazon GuardDuty in either the source AWS account or the central security monitoring account, focusing specifically on any recent findings, regardless of whether AWS resources are implicated in the incident.
5. Additionally, review AWS CloudTrail logs to detect any unauthorized access or changes to parameters/secrets in AWS Systems Manager Parameter Store and AWS Secrets Manager. This is important as these services may act as the starting point for managing credentials to access data storage services. To get you started, few examples are:
 1. [Analyze Security, Compliance, and Operational Activity Using AWS CloudTrail and Amazon Athena](#)

6. Please review the access logs for your S3 buckets to identify any unintended or unidentified activity. Here are a few examples of documentation to help you get started:

1. [Logging requests using server access logging](#)

2. [Identifying access to S3 objects by using CloudTrail](#)

7. Utilize the *Advanced Queries* feature of AWS Config to analyze any unintentional or unidentified changes to AWS resources and configurations. To get you started, few example queries mentioned [here](#).

8. Examine Amazon VPC Flow Logs and AWS WAF logs for any anomalous network activity.

8. If you are *aware* of the AWS resources involved:

1. Consider the following steps to identify suspicious activity or unauthorized access in Amazon S3:

1. [Review Amazon Macie policy findings related to your S3 data](#)

2. [Analyze S3 protection in Amazon GuardDuty](#)

2. For unintended access to an Amazon S3 bucket refer to [this incident response playbook](#).

9. As previously stated, you maintaining a clear chain of custody is crucial in ensuring the immutability of all evidence. Some of the techniques are mentioned in blog post - [Forensic investigation environment strategies in the AWS Cloud](#). In summary, here are the techniques:

1. Snapshots of Amazon EBS disks: The original EBS disks can be snapshotted, shared to a forensics account, converted into a volume, and mounted in read-only mode for offline analysis.

2. Manually captured Amazon EBS volumes: Linux tools such as dc3dd can be used to stream the volume to an S3 bucket, along with a hash, and made immutable using an S3 method.

3. Artifacts stored in an S3 bucket, such as memory dumps: Object Lock in S3 can prevent deletion or overwriting of objects for a specified duration or permanently. MFA delete requires multi-factor authentication to delete an object permanently. Glacier provides a Vault Lock feature to retain evidence in an immutable state over the long term.

4. Disk volumes: Read-only mode can be used for Linux and write-blocker applications for Windows, some of which are specifically designed for forensic use.

5. CloudTrail logs: Log file integrity can be validated using CloudTrail's SHA-256 hash and SHA-256 with RSA signing. S3 Object Lock - Governance Mode can be used for protection.

6. AWS Systems Manager inventory: By default, metadata on managed instances is stored in an S3 bucket and can be secured using the above methods.

7. AWS Config data: Data stored by AWS Config in an S3 bucket can also be protected through the aforementioned methods.

10. Check if there are any open support tickets for the data storage service, and search for connections with any alerts or information related to the incident. Note any perceived effects on end-users, which may encompass but are not limited to:

1. Missing data from the storage service

2. Unexpected presence of new data in the storage service

3. Changes in access settings (such as permissions) of data, especially if they have been made public

4. Modified data storage service permissions, access controls, or settings that prevent public access (these may be less noticeable to regular users, but still a possibility)

11. The following are recommendations for internal and external communication regarding a data breach incident, however they may vary based on legal requirements. This example is

based on an organization governed by GDPR regulations.

1. For internal communication, it is important to quickly gather relevant information and assess the scope and impact of the breach. This information should then be shared with the incident response team and relevant stakeholders within the organization. Regular updates should be provided throughout the investigation process to keep everyone informed.

2. For external communication, it is important to consider the privacy rights of individuals whose PII may have been compromised, as well as the organization's obligations under the GDPR or other relevant legislation/protocols for your jurisdiction. In general, organizations are required to report data breaches to relevant authorities within 72 hours and to notify affected individuals without undue delay.

Part 2: Contain the Incident

Early identification of unusual actions taken by users or strange network activity is crucial in minimizing the harm caused by incidents involving a PII data breach. To prevent the situation from worsening, it's important to take steps to contain the incident, as well as collaborating with your organization's legal and compliance team on any necessary responses and following the incident response plan outlined. Here are a few examples of containment actions for specific AWS services:

1. In this scenario, PII data breach was in an Amazon S3 bucket. Here are the steps you can take to contain the incident:

1. Identify IAM identities with access to AWS resources

1. The first step in the process is to identify the IAM identities (which can be either human users or non-human roles) that have access to the AWS resources in question. This is important because it helps to determine the scope of the breach and the extent to which sensitive data has been compromised. Based on this information, you can then take appropriate actions to mitigate the impact of the breach.

2. Remove access granting policy for the specific IAM role or user

1. In order to revoke S3 access from an IAM role or user, you need to navigate to the specific S3 bucket where the data breach has occurred. This is where the sensitive data was stored and where unauthorized access occurred.

2. Once you have navigated to the specific S3 bucket, you need to click on the "Permissions" tab. From there, select the "Bucket Policy" option.

3. The next step is to remove the access granting policy for the specific IAM role or user. Here is an example policy on how to [Managing user access to specific folders](#). This will effectively revoke the access of the IAM role or user to the S3 bucket, preventing further unauthorized access to the sensitive data

2. In this scenario, external vendor accounts have been accessing PII stored in DynamoDB tables through SQS and AWS Lambda. The PII data is encrypted using a KMS key at rest and in transit by application, however, due to a defect in the application, some of the PII information may not have been encrypted properly in transit. To contain this PII data breach, you can take the following actions:

1. Please evaluate the potential business impact of limiting or revoking vendor access to the DynamoDB table, if applicable.

2. If it is acceptable to revoke the access of the vendor, you can do so by updating the access policy to remove their permissions to access the impacted DynamoDB tables and the corresponding SQS queue. This will help to prevent any unauthorized access or use of the resources, which will limit the potential impact of the security breach.

3. Remove the PII data from the logs of the Lambda function that consumes SQS messages. This helps prevent the sensitive information from being further disclosed or compromised. By using Amazon Comprehend and a Lambda function, you can automatically detect and redact PII data in your S3 objects. Following is a tutorial on explaining how this can be accomplished:

1. [Tutorial: Detecting and redacting PII data with S3 Object Lambda and Amazon Comprehend](#)

2. [Sample project to demonstrate how to use AWS SDK for Amazon Comprehend to detect and redact PII data from logs generated by Java applications](#)

4. Conduct a table scan to identify all the rows with unencrypted PII data and update/delete columns.

Part 3: Eradicate the Incident

First step is to determine the affected resources using log data, resources, and automated tooling and then assess each affected resource to determine the business impact of deletion or restoration. Below is an example of eradication steps that can be taken for a personal data exposure that happened due to wide open security group:

1. Identify the potential immediate cause or causes for the PII exposure at hand:

1. Analyze the collected log data, resources, and tooling to determine the source of the exposure.

2. Identify if the cause, and then work back through the “5 whys” to get to the root cause <https://aws.amazon.com/blogs/mt/why-you-should-develop-a-correction-of-error-coe/>.

2. It may be helpful to review the infrastructure deployment pipelines, such as those implemented with CloudFormation via AWS Code Pipeline or Terraform, and the data

pipelines to determine which mechanism led to the misconfiguration or misclassification of data injection mechanisms. Are there enough checks and balances in the pipeline? If not, it may be necessary to address this issue.

3. If there are any specific issues that have been identified, take steps to remove any resources or configurations that were identified as the immediate causes. For example:

1. If the cause is a misconfigured security group, adjust the security group rules to prevent unauthorized access.

2. If the cause is an unauthorized user, revoke their access to the affected resources.

3. If the cause is a vulnerability, apply patches or upgrades to fix it.

4. There may be other causes, take the appropriate action.

4. Clean up the environment:

1. Delete any PII data that may have been exposed.

2. If PII data must be kept, encrypt it to prevent unauthorized access.

5. Restore normal operations:

1. Restore permissions and access for authorized users.
2. Reconnect systems or resources to the network.

Part 4: Recover from the Incident

In addition to restoring the service to a stable and reliable state, it is crucial to inform and provide clear guidance to affected users. Here are the steps you can follow:

1. Coordinate with PR and Legal teams:

1. Work with your organization's public relations and legal teams to determine the most appropriate approach for communicating with affected users.

1. Update FAQ and Help Center pages:

1. Utilize your organization's FAQ and Help Center web pages to provide clear and concise information to affected users.

1. Ensure that the FAQ and Help Center web page are updated regularly with the latest information regarding the incident, including any measures that users can take to safeguard themselves. Here are some sample guidelines that you can provide to your affected external customers in case of identity theft via your application:

1. *Verify the breach*: Confirm that your personal information has been exposed in the data breach by checking our help centre website.

2. *Secure your accounts*: Change passwords for any accounts that use the breached information, such as email and financial accounts, and enable two-factor authentication.

3. *Monitor your accounts*: Regularly monitor your accounts associated with the breached information for any unauthorized activities.

4. *Place a fraud alert*: Consider placing a fraud alert on your credit report to alert lenders and banks to check with you before opening new accounts.

5. *Consider credit monitoring*: Consider enrolling in a credit monitoring service to receive alerts about any changes or suspicious activity on your credit report.

6. *Report the breach*: Report the data breach to relevant government agencies if applicable.

7. *Stay vigilant*: Be mindful of phishing scams and other attempts to steal your information, and take steps to protect your information going forward.

1. Provide timely information:

1. Make sure that information is updated in a timely manner to keep affected users informed of the latest developments.

2. Ensure that the information provided is accurate and easy to understand.

By taking these steps, you can help affected users to stay informed and feel confident that their information and interests are being protected during an incident.

Part 5: Post-Incident Activity

>Note

>

> It is important to contact legal counsel **early** to determine if public and individual communication needs to be initiated. Legal counsel can provide guidance on the specific legal requirements and obligations associated with the notification of personal data breaches, including any applicable laws, regulations, and industry standards.

Once you have removed the affected resources, it's recommended that you perform a security assessment of your AWS account. This assessment can be accomplished by utilizing AWS Config rules, open-source tools like Prowler and ScoutSuite, or other providers. Furthermore, we suggest conducting vulnerability scans on your publicly accessible resources to identify any lingering risks.

In the aftermath of a personal data breach, it is critical to conduct a comprehensive post-incident review to identify areas for improvement and prevent similar incidents from happening in the future. The following post-incident activities can be performed in accordance with the correction of error (COE) concept that is discussed in the following links:

[Why you should develop a correction of error \(COE\)](#)

Correction of Error (COE)

1. Review the incident: Perform a thorough review of the incident, including the causes, severity, and impact. This information will be used to determine the effectiveness of the incident response plan and identify areas for improvement.
2. Document lessons learned: Document lessons learned from the incident and use this information to update the incident response plan. This information should include details about the attack vector, the methods used by the attacker, and any other relevant information.
3. Update the incident response plan: Based on the lessons learned, update the incident response plan to reflect any changes or improvements that should be made to the incident response process.
4. Perform a root cause analysis: Conduct a root cause analysis to determine the underlying causes of the incident and to identify any additional areas for improvement.

5. Update policies and procedures: Based on the results of the root cause analysis, update policies and procedures to ensure that the incident response plan is aligned with best practices and regulatory requirements.

6. Implement corrective actions: Implement any necessary corrective actions to prevent similar incidents from happening in the future.

■ `playbooks/IRP-Ransomware.md`

Incident Response Playbook Template

Incident Type

Ransomware

Introduction

This playbook is provided as a template to customers using AWS products and who are building their incident response capability. You should customize this template to suit your particular needs, risks, available tools and work processes.

Security and Compliance is a shared responsibility between you and AWS. AWS is responsible for “Security of the Cloud”, while you are responsible for “Security in the Cloud”. For more information on the shared responsibility model, [please review our documentation](#).

You are responsible for making your own independent assessment of the information in this document. This document: (a) is for informational purposes only, (b) references current AWS product offerings and practices, which are subject to change without notice, and (c) does not create any commitments or assurances from AWS and its affiliates, suppliers or licensors. This document is provided “as is” without warranties, representations, or conditions of any kind, whether express or implied. The responsibilities and liabilities of AWS to its customers are controlled by AWS agreements, and this document is not part of, nor does it modify, any agreement between AWS and its customers.

Summary

This Playbook

This playbook outlines response steps for handling ransomware incidents. These steps are based on the [NIST Computer Security Incident Handling Guide](#) (Special Publication 800-61 Revision 2) that can be used to:

- Gather evidence
- Contain and then eradicate the incident
- recover from the incident
- Conduct post-incident activities, including post-mortem and feedback processes

Interested readers may also refer to the [AWS Security Incident Response Guide](#) which contains additional resources.

Once you have customized this playbook to meet your needs, it is important that you test the playbook (e.g., Game Days) and any automation (functional tests), update as necessary to achieve the desired results, and then publish to your knowledge management system and train all responders.

Note that some of the incident response steps noted below may incur costs in your AWS account(s) for services used in either preparing for, or responding to incidents. Customizing this playbook and testing it will help you to determine if additional costs will be incurred. You can use [AWS Cost Explorer](#) and look at costs incurred over a particular time frame (such as when running Game Days) to establish what the possible impact might be.

In reviewing this playbook, you will find steps that involve processes that you may not have in place today. Proactively preparing for incidents means you need the right resource configurations, tools and services in place that allow you to respond to an incident.

The next section will provide a summary of this incident type, and then cover the five steps (parts 1 - 5) for handling ransomware incidents.

This Incident Type

Ransomware is malicious code designed by threat actors to gain unauthorized access to systems and data and to encrypt the data to block access by legitimate users. Once ransomware has locked users out of their systems and/or encrypted their sensitive data, the actors demand a ransom. In theory, if the ransom is paid, access to the data is returned (such as by providing an encryption key), but equally, some studies have suggested the victim will subsequently be attacked again. Alternatively, if not paid, the organization risks permanent data loss and/or data leaks to the public, competitors or other malicious actors.

There are usually limited options to mitigate a successful ransomware attack once it has occurred. The best mitigation is to reduce the chance that it can happen in the first place. The AWS Well-Architected security pillar provides a framework to implement AWS best practice, including operating workloads security (security foundations section), protecting compute resources (infrastructure protection section) and others. The security pillar encompasses the ability to protect data, systems, and assets to take advantage of cloud technologies to improve your security. This playbook covers steps that can be used to deal with ransomware.

Incident Response Process

Part 1: Acquire, Preserve, Document Evidence

1. You become aware that a possible ransomware incident has occurred. This information could come via different means, depending on your configurations in your AWS environment:

1. A colleague reports that an EC2 instance cannot be accessed by SSH or similar, however the instance appears to be correctly configured with appropriate network access in place, and there are no related service issues reported by AWS in the Service Health Dashboard

2. Your ticketing system creates a ticket for unusual metrics or logs from the EC2 instance

3. The instance is reporting network reachability issues in the AWS console, or via Amazon CloudWatch alarms

4. Message from threat actor through alternate communication channel such as email about the ransom demand

5. Findings through services like AWS Security Hub or Amazon GuardDuty.

6. Other alarms or metrics you have configured in your monitoring systems, either internal or external to AWS

2. Once you confirm that an event is a security incident, it is important to determine the scope of impact (quantity of resources as well as sensitivity of data).

1. Determine if there are any known events that could be causing service disruption, or impacting instance metrics (e.g., network CloudWatch metrics increasing due to a sales event, or similar)

2. Use Amazon Detective to investigate any ongoing activity using the time based analysis or a specific period when the incident was identified to identify any deviations from a “normal” operating baseline.

3. Obtain the application’s documented baseline and locate the metrics for standard application performance from the CloudWatch or other application performances monitoring tool used in the organization to compare baseline behavior to anomalous behavior as a result of the incident.

4. Determine the classification level of any data that resides in the EC2 instance, S3 bucket, Amazon Workspaces instance, etc.

5. Confirm a ticket/case has been raised for the incident. If not, manually raise one.

6. If you received an abuse notification from AWS, determine if any cases are already open for the resource that can be correlated to the abuse notification. This may provide

indications relating to prior unauthorized activity.

7. If there is a ticket/case already opened, determine what internal alarms/metrics are currently indicating an issue (if automated, what caused the ticket to be created?) If the ticket/case creation was not initiated automatically by an alarm or metric, document the alert/notification that led to identification of the issue if there was one (for example, a ransom demand popping up on the screen, or metrics that indicate the device is no longer on the network). If the incident was identified by an indicator that does not conclusively identify a ransomware incident as the cause, then verify the service disruption is not due to any planned (or other) event and document the actual vector.

8. Identifying the ransomware strain is key to recovery. For example, if objects in S3 buckets are inaccessible with an error of not having encryption key access, the first step would be to review the S3 object properties section to understand the encryption key applied. A similar approach can be leveraged with the Amazon Elastic Block Store (EBS)volumes in situations involving crypto ransomware.

9. Using your preferred monitoring tool, access AWS CloudTrail and search for any API actions which indicates any credential compromise attack vector and refer to the playbook, "Credential Leakage/Compromise".

10. Determine when the infection occurred using log search. CloudWatch can help you to review logs such as application logs, operating system logs, database logs, etc.

11. Determine the business impact:

a. Identify application(s) impacted; this may be achieved via Resource Tags, or by an internal Configuration Management Database (CMDB)

b. Identify business owner and workload classification (example: mission critical, high risk, etc.)

12. Determine and begin to document end-user impact/experience of the issue. This should be documented in the ticket/case related to the incident. If there are impacted users, determine from them the steps that led to the incident. This will assist you to establish the attack vector used to execute the ransomware. Mitigations against this vector should be placed in later steps of this process.

13. Internal Communications:

a. Identify stakeholder roles from your organization's incident response plan, the application entry in the Configuration Management Database CMDB (if one exists), or via the application's risk register

b. Open an Incident Response bridge to have a regular communication channel about the incident.

c. Notify identified stakeholders including (if required) legal personnel, technical teams and developers and add them to the ticket and the war room, so they are updated as the ticket is updated

14. External Communications:

- i. Ensure your organization's legal counsel is informed and is included in status updates to internal stakeholders and especially in regards to external communications.

- ii. For colleagues in the organization that are responsible for providing public/external communication statements, ensure these internal stakeholders are added to the ticket so they receive regular status updates regarding the incident and can complete their own requirements for communications within and external to the business.

- iii. If there are regulations in your jurisdiction requiring reporting of such incidents, ensure the people in your organization responsible for notifying local or federal law enforcement agencies are also notified of the event. Consult your legal advisor and/or law enforcement for guidance on collecting and preserving the evidence and the chain of custody.

- iv. There may not be regulations, but either open databases, government agencies or NGOs may track this type of activity. Your reporting may assist others

Part 2: Contain the Incident

Early detection of anomalous user behavior or network activity is key to reducing the impact of ransomware incidents. The below steps can be taken to help to contain the incident. If applicable, work with the legal and compliance team of your organization on any required response and continue the incident response process outlined here.

1. If possible, determine the type of ransomware used in the incident:

a. Crypto Ransomware - Objects/files will be encrypted

b. Locker ransomware - Locks out access to the device

c. A different type, or previously not observed type

2. For identified AWS resources associated with your impacted workloads, isolate network or internet connectivity by modifying Security groups, S3 bucket Policies or relevant identity and access management policies as applicable to minimize opportunities for the infection to be spread, or for threat actors to have access to those resources. Keep in mind that sometimes modifying security groups may not have the intended impact, due to connection tracking.

3. Determine whether the EC2 instance actually needs to be recovered, or not. For example, if the impacted instance is part of an AWS Application Auto Scaling group, removing the instance from the group will trigger a scaling action. If the incident is linked to a vulnerable package on the host's operating system, updating the AMI used in the Launch configuration and confirming the vulnerability has been patched (check the Mitre CVE database) will also be required.

4. Check your CloudTrail log for unauthorized activity such as creation of unauthorized IAM users, policies, roles or temporary security credentials. Delete any unauthorized IAM users, roles, and policies, and revoke any temporary credentials.

5. If you're dealing with this incident as part of a broader security incident in the account, a drastic approach could be to use AWS Organizations Service Control Policies (SCPs) to restrict any API call to be made from that AWS account (assuming it is not the master

account of the Organization). Please note this may impact other running workloads as SCPs applied at the account level will be enforced on all the IAM entities associated in it. This may prevent malicious actors from inflicting further damage on the account's resources and data.

6. If the attack vector was made possible by un-patched software, operating system updates, out-of-date malware/anti-virus tools, ensure that all EC2 instances are either updated to the latest version of operating system, all software packages and patches are up-to-date and virus signatures and definition files on all EC2 instances are up-to-date. This may be done by several methods:

a. Patch-in-place for mutable architectures

b. Re-deploy for immutable architectures

7. Depending on what occurred in Step 6 above, remove any remaining resources that are identified as being at risk of infection (that may have accessed the same vector that downloaded the ransomware, whether that be via email, visiting an infected website, or something else). If it is part of a larger fleet managed by Auto Scaling, containment efforts may be better focused on establishing the attack vector and then placing mitigations that will prevent other resources in the fleet from becoming infected via the same vector.

Part 3: Eradicate the Incident

1. It is important to understand if the impact from the incident is contained to a subset of the environment. If there is an ability to restore the ransomed data from backups/snapshots, you

can refer to the recover from the incident section. Note that there may still be value in exploring the incident under an isolated environment to run the root cause analysis and use controls to avoid situation in the future.

2. Investigate potential use of up-to-date antivirus or anti-malware software to clean the ransomware. Refer to this step with caution as it may alert the actor (see earlier steps to remove network access from the impacted EC2 instance). You can review the locked/encrypted objects in an isolated forensic environment.

3. Review any GuardDuty findings if there are any high or medium severity alerts that can help to reduce the additional effort required to search application level logs. GuardDuty findings provide recommendations on how to remediate the finding.

4. Remove any malware that was identified during the forensic analysis and identify Indicators of Compromise.

5. If the ransomware strain has been identified, determine if any 3rd party decryption tools are available, or if any other online resources may help

Part 4: Recover from the Incident

1. Identify the restore point for any restore operation performed from backup.

2. Review the backup strategy and see if you can recover all the objects and files. This will depend on the lifecycle policies applied on the resources.

3. Restore the data from your backup, or revert to an earlier snapshot of the EC2 instance's volumes. Apply forensic methods to confirm that data is clean before restoring it. Ensure any spread vectors are identified & resolved.

4. Alternatively, if you have been successful in using any open source de-crypter tool to retrieve the data, remove that data from the instance and perform any required analysis to ensure the data is clean. Then, recover the instance, terminate it or quarantine it and create a new one, and restore the data to a new instance.

5. If restoring from a backup and de-crypting the data is not an option, consider whether to start from entirely new environment is a possibility .

Part 5: Post-Incident Activity

1. Documenting and cycling lessons learned during simulations and live incidents back into “new normal” processes and procedures allows organizations to better understand how an incident occurred with their configurations and processes – such as where they were vulnerable, where automation may have failed, or where visibility was lacking – and the opportunity to strengthen their overall security posture.

2. If you identified the initial attack vector or point of entry, determine how best to mitigate the risk of a re-occurrence. For example, if the malware gained initial entry due to an un-patched public-facing EC2 instance, and assuming you applied the missing patch to all current instances, how can you improve your patching process to ensure that patches are tested and applied sooner and with more consistency and reliability?

3. If you've developed set technical steps to take for a given threat, assess opportunities to automate those actions upon detection to mitigate the threat as quickly as possible to minimize scope and severity of impact.

4. Ensure you collect lessons learned from all stakeholders and update your incident response plan, disaster recovery plans, and this playbook as necessary. And where appropriate, consider priorities and funding for new technical capabilities and personnel skills to fill any gaps.

Appendix A

Preparation

As with other incident response scenarios, mitigating ransomware threats requires a strong preventative strategy and preparation is very important. More so than others though, mitigation techniques rely on adequate preparation prior to a ransomware incident. Implementing best practices based on the AWS Well-Architected Framework will help you in your preparation. If key preventative measures are not already in place, these must be considered as part of any post-incident activity including any tools or techniques that failed during the incident response process itself, such as automation, integration with partner offerings, application and systems observability, etc. The below points cover some key preparation steps.

1. Adopt foundational best practices: Leverage AWS Cloud Adoption Framework (CAF) and AWS Well-Architected best practices to your organization's cybersecurity framework such as the NIST CyberSecurity Framework for AWS, a framework focused on security outcomes organized around five functions (Identify, Protect, Detect, Respond, Recover) and foundational activities that map to existing standards, accreditations and frameworks.
2. Training: Cybersecurity awareness training for your employees. Social engineering is one of the common methods used to induce end users to download an infected file.

3. Deep dive on assets and configurations: Identifying your IT assets is a vital component of governance and security. You must have visibility into all your data storage services and resources in order to evaluate their security posture and effectively configure them. Reduce potential attack surface, for instance, by analysing your public data footprint. If you use Amazon Simple Storage Service (S3), ensure that your S3 buckets use the correct policies and are not publicly accessible. Implement the idea of least privilege enterprise-wide. With S3 Versioning, existing versions of your data are immutable: actors cannot change existing objects, and any modification is going to result in a new version. Use MFA delete to require a second element of authentication for S3 data deletion. In addition, S3 includes a number of security controls to consider when developing and implementing your own security policies; for more information, please see [security best practices for Amazon S3](#).

4. Stay up-to-date with system patches: Ransomware often relies on exploit kits to gain access to a system or network using known threats. Amazon Elastic Compute Cloud (Amazon EC2) resources can use AWS Systems Manager to automatically collect software inventory, apply OS patches, create system images, and configure Windows and Linux operating systems. These capabilities enable automated configuration and ongoing management of systems at scale, and help maintain software compliance for instances running in EC2.

5. Continuous monitoring:

a. Use Amazon Inspector to assess your deployed EC2 instances for CVEs and deviation from best practice. Build a CI/CD pipeline to remediate security findings automatically & periodically. Amazon Inspector finds applications by querying the package manager or software installation system on the operating system where the agent is installed. This means that software that was installed through the package manager is assessed for vulnerabilities. You can also leverage AWS Security Competency Partner offerings to help inspect your application deployments for security risks and vulnerabilities, while providing priorities and advice to assist with remediation.

b. AWS developed a new open source Self-Service Security Assessment tool (with ransomware analysis modules) that provides customers with a point-in-time assessment to

quickly gain valuable insights into the security posture of their AWS account. For continuous monitoring of your security posture, AWS recommends enabling AWS Security Hub's Foundational Security Best Practices standard, which also provides automated security checks.

6. Data back-up: Create secure backups of your data on a regular basis. AWS Backup is a fully managed backup service that makes it easy to centralize and automate the backup of data across AWS services. If backups are not a requirement, ensure that there are regular snapshots or an up-to-date AMI that can be used either for recovery or replacement of the impacted EC2 instance. Amazon S3 and Amazon Glacier are durable, low-cost storage platforms. Both offer unlimited capacity and require no volume or media management as backup data sets grow. Backup and Recovery Approaches Using AWS.

7. Testing: Implement, review and test disaster recovery scenarios for your organizational needs. This will help you determine the path for implementing recovery mechanisms and mitigate risks. Disaster recovery is about preparing for and recovering from an event that has a negative impact on your business goals. You can use CloudEndure Disaster Recovery to quickly recover your environment, minimizing data loss and downtime in the case of a ransomware incident.

8. Automation: Where you identify fixed technical procedures that don't deviate or require a human to make a decision, explore the opportunity to automate protections and response actions to mitigate the threat as quickly as possible to minimize scope and severity of impact.

Communication channels: Identify communication channels such as an Incident Response bridge that you will need to use during the incident to communicate with the internal and external stakeholders.

■ playbooks/Amazon GuardDuty Playbooks/EKS
Findings/IRP-GD-PrivilegeEscalation-Kubernetes_PrivilegedContainer.md

Incident Response Playbook Template

Incident Type

GuardDuty Finding: PrivilegeEscalation-Kubernetes:PrivilegedContainer

Introduction

This playbook is provided as a template to customers using AWS products and who are building their incident response capability. You should customize this template to suit your particular needs, risks, available tools and work processes.

Security and Compliance is a shared responsibility between you and AWS. AWS is responsible for “Security of the Cloud”, while you are responsible for “Security in the Cloud”. For more information on the shared responsibility model, [please review our documentation](#).

You are responsible for making your own independent assessment of the information in this document. This document: (a) is for informational purposes only, (b) references current AWS product offerings and practices, which are subject to change without notice, and (c) does not create any commitments or assurances from AWS and its affiliates, suppliers or licensors. This document is provided “as is” without warranties, representations, or conditions of any kind, whether express or implied. The responsibilities and liabilities of AWS to its customers are controlled by AWS agreements, and this document is not part of, nor does it modify, any agreement between AWS and its customers.

Summary

This Playbook

This playbook outlines response steps for incidents involving deployment of a privileged container. These steps are based on the [NIST Computer Security Incident Handling Guide](#) (Special Publication 800-61 Revision 2) that can be used to:

Gather evidence

Contain and then eradicate the incident

Recover from the incident

Conduct post-incident activities, including post-mortem and feedback processes

Interested readers may also refer to the [AWS Security Incident Response Guide](#) which contains additional resources.

Once you have customized this playbook to meet your needs, it is important that you test the playbook (e.g., Game Days) and any automation (functional tests), update as necessary to achieve the desired results, and then publish to your knowledge management system and train all responders.

Note that some of the incident response steps noted in each scenario may incur costs in your AWS account(s) for services used in either preparing for, or responding to incidents. Customizing these scenarios and testing them will help you to determine if additional costs will be incurred. You can use [AWS Cost Explorer](#) and look at costs incurred over a particular time frame (such as when running Game Days) to establish what the possible impact might be.

In reviewing this playbook, you will find steps that involve processes that you may not have in place today. Proactively preparing for incidents means you need the right resource configurations, tools and services in place that allow you to respond to an incident.

The next section will provide a summary of this incident type, and then cover the five steps (parts 1 - 5) for handling privileged containers.

This Incident Type

An Amazon GuardDuty finding represents a potential security issue detected within your network. GuardDuty generates a finding whenever it detects unexpected and potentially malicious activity in your AWS environment. All GuardDuty finding references in this playbook will be related to the GuardDuty finding JSON that can be seen in the GuardDuty console, downloaded from the GuardDuty or Security Hub console, or exported to S3.

PrivilegeEscalation:Kubernetes/PrivilegedContainer

A privileged container with root level access was launched on your Kubernetes cluster.

This finding informs you that a privileged container was launched on your Kubernetes cluster using an image has never before been used to launch privileged containers in your

cluster. A privileged container has root level access to the host. Adversaries can launch privileged containers as a privilege escalation tactic to gain access to and then compromise the host.

Details on what resources are involved with this activity can be found in the [finding details](#).

Incident Response Process

Part 1: Acquire, Preserve, Document Evidence

For Any Incident

1. You become aware of potential indicators of compromise (IoCs). These could come in various forms, but the original source is a GuardDuty finding:

An internal ticketing system (the sources of the ticket are varied and could include any of the means below)

From an alert in one of your monitoring systems either inside or external to AWS (that are ingesting GuardDuty Findings, in AWS, this could include AWS Security Hub)

Alarms or observations that resources have been created or deleted in your account that cannot be accounted for in your CMDB, exist in regions that you do not operate infrastructure in, or themselves have generated alerts ([Amazon Detective](#) is a useful tool for understanding these relationships)

1. Confirm a ticket/case has been raised for the incident. If not, manually raise one

1. Determine and begin to document any end-user impact/experience of the issue. Findings should be documented in the ticket/case related to the incident

1. Open an incident war room using your standard communication tools, and page in the relevant stakeholders

1. In the case of automatically created tickets/cases, verify the finding in GuardDuty (what caused the ticket to be created?)

For This Incident Type

1. Identify the specific EKS cluster impacted. In GuardDuty, this will be in the Resource.EksClusterDetails.Name section of the finding.

*1. Identify Pod, Node, and User information to be used in **Part 2**.*

- Pod information can be found in the Resource.KubernetesWorkloadDetails section of the GuardDuty finding.*

- Node information can be found after determining the pod name. Once you have the pod name you can run the command provided below to determine the node name.
- `kubectrl get pods --namespace -o=jsonpath='{.spec.nodeName}'`
- User information can be found in the `Resource.KubernetesDetails.KubernetesUserDetails` section of the `GuardDuty Finding`.

1. Identify the origination information from which the Kubernetes APIs were called. In `GuardDuty`, Look at the `Resource.Service.Action` section of the finding

- Verify that the IP address is valid for your enterprise users
- Verify that the Location and/or ASN/Organization is known and valid for this request

1. If the principal that launched the privileged container is associated with a person in the organization, contact them to verify the launched privilege container is valid and was intended.

- If the person states that they did launch the privilege container and the configuration change was intended, verify that there is a valid Change Management (CM) request or other authorization:
- Once verified, move on to Part 5 to review the incident, and propose improvements that would stop or automatically archive findings for valid configuration changes within your organization.
- If authorization for the change cannot be verified, communicate to stakeholders your intent to remediate or rollback the change and proceed to Part 2.

- If the person states they did not launch the privileged container, proceed to **Part 2**.

Part 2: Contain the Incident

If you determine that the activity is unauthorized, or decide it is prudent to assume so, the first priority is to prevent further compromise without impact to production workloads.

1. Verify that disabling the Kubernetes user or isolating this pod will not result in a service outage.

1. If required, retain the user account for further forensic analysis by removing permissions from the Kubernetes User resource responsible for the activity using the appropriate steps below.

1. Built-in Kubernetes admin – The default user assigned by Amazon EKS to the IAM identity that created the cluster. This user type is identified by the user name kubernetes-admin.

- To revoke access of a built-in Kubernetes admin:*
- Identify the userType from the Access Key details section.*
- If the userType is Role and the role belongs to an EC2 instance role:*
- Identify that instance then follow the instructions in Remediating a compromised EC2 instance*
(https://docs.aws.amazon.com/guardduty/latest/ug/guardduty_remediate.html#compromised-ec2).
- If the userType is User, or is a Role that was assumed by a user:*

1. Rotate the access key of that user with the steps listed below:

To rotate access keys for an IAM user without interrupting your applications (console)

1. While the first access key is still active, create a second access key.

- Sign in to the AWS Management Console and open the IAM console at*
(<https://console.aws.amazon.com/iam/>).
- In the navigation pane, choose Users.*

- Choose the name of the intended user, and then choose the Security credentials tab.

- Choose Create access key and then choose Download .csv file to save the access key ID and secret access key to a .csv file on your computer. Store the file in a secure location. You will not have access to the secret access key again after this closes. After you have downloaded the .csv file, choose Close. The new access key is active by default. At this point, the user has two active access keys.

2. Update all applications and tools to use the new access key.

3. Determine whether the first access key is still in use by reviewing the Last used column for the oldest access key. One approach is to wait several days and then check the old access key for any use before proceeding.

4. Even if the Last used column value indicates that the old key has never been used, we recommend that you do not immediately delete the first access key. Instead, choose Make inactive to deactivate the first access key.

5. Use only the new access key to confirm that your applications are working. Any applications and tools that still use the original access key will stop working at this point because they no longer have access to AWS resources. If you find such an application or tool, you can choose Make active to reenable the first access key. Then return to Step 3 and update this application to use the new key.

6. After you wait some period of time to ensure that all applications and tools have been updated, you can delete the first access key:

- Sign in to the AWS Management Console and open the IAM console at (<https://console.aws.amazon.com/iam/>).

- *In the navigation pane, choose Users.*
- *Choose the name of the intended user, and then choose the Security credentials tab.*
- *Locate the access key to delete and choose its X button at the far right of the row. Enter the access key ID to confirm the deletion and then choose Delete.*

2. Rotate any secrets that user had access to. Depending on where you stored your secrets will dictate what is the best process for rotation. If you are storing your secrets in the native secrets management capabilities of EKS use the kubernetes documentation to rotate your secrets (<https://kubernetes.io/docs/concepts/configuration/secret/>). If you are using an external secrets store follow the product specific directions for secrets rotation, for example if you are using AWS Secrets Manager you can follow this documentation to rotate secrets (<https://docs.aws.amazon.com/secretsmanager/latest/userguide/rotating-secrets.html>).

3. Review the information in My AWS account may be compromised for further details (<https://aws.amazon.com/premiumsupport/knowledge-center/potential-account-compromise/>).

1. To revoke access of an OIDC authenticated user, which is typically a user has an email address as a user name follow the steps below:

- *To check if your cluster uses OIDC use the following AWS CLI command: "aws eks list-identity-provider-configs --cluster-name **your cluster name**"*
- *Rotate the credentials of that user in the OIDC provider.*

- Rotate any secrets that user had access to. Depending on where you stored your secrets will dictate what is the best process for rotation. If you are storing your secrets in the native secrets management capabilities of EKS use the [kubernetes documentation](https://kubernetes.io/docs/concepts/configuration/secret/) to rotate your secrets (<https://kubernetes.io/docs/concepts/configuration/secret/>). If you are using an external secrets store follow the product specific directions for secrets rotation, for example if you are using AWS Secrets Manager you can follow this [documentation](https://docs.aws.amazon.com/secretsmanager/latest/userguide/rotating-secrets.html) to rotate secrets (<https://docs.aws.amazon.com/secretsmanager/latest/userguide/rotating-secrets.html>).

1. To revoke access of an AWS ConfigMap user:

- Use the following command to open the ConfigMap.

```
kubectl edit configmaps aws-auth --namespace kube-system
```

1. Identify the role or user entry under the `mapRoles` or `mapUsers` section with the same user name as the one reported in the Kubernetes user details section of your GuardDuty finding. See the following example, where the admin user has been identified in a finding.

```
apiVersion: v1
```

```
data:
```

```
mapRoles: |
```

- `rolearn: arn:aws:iam::444455556666:role/eksctl-my-cluster-nodegroup-standard-wo-NodeI`

`user name: system:node:EC2_PrivateDNSName`

`groups:`

- `system:bootstrappers`

- `system:nodes`

`mapUsers: |`

- `userarn: arn:aws:iam::123456789012:user/admin`

`username: admin`

`groups:`

- `system:masters`

- `userarn: arn:aws:iam::111122223333:user/ops-user`

`username: ops-user`

`groups:`

- `system:masters`

1. Remove that user from the ConfigMap. See the following example where the admin user has been removed.

```
apiVersion: v1
```

```
data:
```

```
mapRoles: |
```

- `rolearn: arn:aws:iam::111122223333:role/eksctl-my-cluster-nodegroup-standard-wo-NodeI`

```
username: system:node:{{EC2PrivateDNSName}}
```

```
groups:
```

- `system:bootstrappers`

- `system:nodes`

```
mapUsers: |
```

- `userarn: arn:aws:iam::111122223333:user/ops-user`


```
username: ops-user
```

```
groups:
```

- `system:masters`

1. If the finding does not have a "accessKeyDetails" section, the user is a Kubernetes service account. The service account provides an identity for pods and can be identified by a user name with the following format:

`system:serviceaccount:namespace:service_account_name`. To revoke access to a service account:

- Rotate the service account credentials to do this you will need to rotate the IAM role credentials that were assigned to the service account with the steps below.

1. While the first access key is still active, create a second access key.

- Sign in to the AWS Management Console and open the IAM console at (<https://console.aws.amazon.com/iam/>).
- In the navigation pane, choose Users.
- Choose the name of the intended user, and then choose the Security credentials tab.
- Choose Create access key and then choose Download .csv file to save the access key ID and secret access key to a .csv file on your computer. Store the file in a secure

location. You will not have access to the secret access key again after this closes. After you have downloaded the .csv file, choose Close. The new access key is active by default. At this point, the user has two active access keys.

2. Update all applications and tools to use the new access key.

3. Determine whether the first access key is still in use by reviewing the Last used column for the oldest access key. One approach is to wait several days and then check the old access key for any use before proceeding.

4. Even if the Last used column value indicates that the old key has never been used, we recommend that you do not immediately delete the first access key. Instead, choose Make inactive to deactivate the first access key.

5. Use only the new access key to confirm that your applications are working. Any applications and tools that still use the original access key will stop working at this point because they no longer have access to AWS resources. If you find such an application or tool, you can choose Make active to reenable the first access key. Then return to Step 3 and update this application to use the new key.

6. After you wait some period of time to ensure that all applications and tools have been updated, you can delete the first access key:

- Sign in to the AWS Management Console and open the IAM console at (<https://console.aws.amazon.com/iam/>).*
- In the navigation pane, choose Users.*
- Choose the name of the intended user, and then choose the Security credentials tab.*

- *Locate the access key to delete and choose its X button at the far right of the row. Enter the access key ID to confirm the deletion and then choose Delete.*

1. If the user has write privileges, it is recommended to audit all changes made by the user in question. This can be accomplished by querying the EKS audit logs in [CloudWatch Logs](#).

- *Locate the Log group for the relevant cluster (you noted this in Part 1) under ``Logs >> Log groups` in the CloudWatch console. They will start with `/aws/eks/cluster_name/audit`*

- *Look for unusual entries in Audit logs*

- *You can also use CloudWatch Log Insights to query CloudWatch logs. An example can be found below that looks for create, update, and delete operation to ClusterRoleBindings. You can find more examples at (<https://aws.github.io/aws-eks-best-practices/security/docs/detective/#analyze-logs-v>)*

```
1. fields @timestamp, @message
```

```
| sort @timestamp desc
```

```
| limit 100
```

```
| filter objectRef.resource="clusterrolebindings" and verb in ["create",  
"update", "patch", "delete"]
```

1. Continue to provide updates to incident stakeholders on the current state of the incident response.

1. Now that we have contained the user specified in the GuardDuty finding next we will want to contain the pod associated with the finding. Use the sections below to isolate the Pods.

- A deny all traffic rule may help stop an attack that is already underway by severing all connections to the pod. The following Network Policy will apply to a pod with the label app=web.

apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:

name: default-deny

spec:

podSelector:

matchLabels:

`app: web`

`policyTypes:`

- `Ingress`

- `Egress`

Attention

1. A Network Policy may prove ineffective if an attacker has gained access to underlying EC2 worker node. If you suspect that has happened, you can use AWS Security Groups to isolate a compromised worker node from other worker nodes. When changing a worker nodes security group, be aware that it will impact all containers running on that worker node.

1. Next we need to cordon the worker node. By deploying a privileged container we have given a container root level permissions to the worker node potentially allowing that container to manipulate the underlying system. By cordoning the impacted worker node, you're informing the scheduler to avoid scheduling pods onto the affected node. This will allow you to remove the node for forensic study without disrupting other workloads.

1. Enable termination protection on impacted worker node - An attacker may attempt to erase their misdeeds by terminating an affected node. Enabling

termination protection can prevent this from happening. Instance scale-in protection will protect the node from a scale-in event. To do this follow the steps below.

- Open the Amazon EC2 console at (<https://console.aws.amazon.com/ec2/>).
- Select the instance, and choose Actions, Instance Settings, Change Termination Protection.
- Choose Yes, Enable.

1. Label the offending Node with a label indicating that it is part of an active investigation - This will serve as a warning to cluster administrators not to tamper with the affected Pods/Nodes until the investigation is complete. To apply a label to a node use the command below.

- `kubectl label nodes \ \=`

1. Capture volatile artifacts on the worker node

- Capture the operating system memory. This will capture the Docker daemon and its subprocess per container. MargaritaShotgun (<https://github.com/ThreatResponse/margaritashotgun>), a remote memory acquisition tool, can aid in this effort.

- *Perform a netstat tree dump of the processes running and the open ports. This will capture the docker daemon and its subprocess per container.*
- *Run docker commands before evidence is altered on the worker node.*
- *``docker container top CONTAINER`` for processes running.*
- *``docker container logs CONTAINER`` for daemon level held logs.*
- *``docker container port CONTAINER`` for list of open ports.*
- *``docker container diff CONTAINER`` to capture changes to files and directories to container's filesystem since its initial launch.*
- *Pause the container for forensic capture.*
- *Snapshot the instance's EBS volumes. Steps to do this below.*
- *Open the Amazon EC2 console at (<https://console.aws.amazon.com/ec2/>)*
- *In the navigation pane, choose Snapshots, Create snapshot.*
- *For Resource type, choose Volume.*
- *For Volume ID, select the volume from which to create the snapshot. The Encryption field indicates the selected volume's encryption status. If the selected volume is encrypted, the snapshot is automatically encrypted using the same KMS key. If the selected volume is unencrypted, the snapshot is not encrypted.*
- *(Optional) For Description, enter a brief description for the snapshot.*

- *(Optional) To assign custom tags to the snapshot, in the Tags section, choose Add tag, and then enter the key-value pair. You can add up to 50 tags.*
- *Choose Create snapshot.*

Part 3: Eradicate the Incident

This is the stage for taking remedial action to minimize the impact of the unintended activities.

1. Using your preferred monitoring tool, access CloudTrail and search for all API actions performed by the AWS EKS resource in the last 90 days:

1. If this tool is a third-party tool such as Splunk, Sumo Logic or others, follow the normal procedure for obtaining log information from that tool

1. If you do not ingest CloudTrail logs into a third-party tool, but do send those logs to Amazon Simple Storage Service (Amazon S3), you will be able to use Amazon Athena to query the logs. The remaining steps will focus on AWS tools to retrieve the necessary information

1. [Create an Athena table referencing the bucket containing your CloudTrail logs](#) that link also includes example queries that can be run in Athena

1. In the Athena console, run a query that shows all API actions taken by the compromised resources post-compromise date/time. From the resulting list, determine which API calls:

- Accessed sensitive data (such as S3 GetObject)

NOTE to get S3 data events CloudTrail must be configured to collect S3 data events prior to the incident. [More information about CloudTrail data events can be found here](#)

- Created new AWS resources, such as databases, Amazon Elastic Compute Cloud (Amazon EC2) instances, AWS Lambda functions or S3 buckets, etc.
- Services that create resources should also be carefully checked; for example, CloudFormation Stacks/StackSets, AWS Firewall Manager Security Policies, AWS Elastic Beanstalk resources, etc.
- Created or modified AWS identity resources that could be used to extend a foothold into the account (or other accounts, for example with AWS Security Token Service (AWS STS) API methods such as AssumeRole). Within an account, API methods including (but not limited to) the following should also be investigated:

- *CreateUser*
- *CreateRole*
- *AssumeRole*
- *GetToken*
- *AttachPolicy*
- *RunInstances* (especially with *PassRole*)
- *Image*
- *Provider*
- *Tag*
- *Untag*
- *Create*
- *Delete*
- *Update*
- *etc.*

- Deleted existing AWS resources
- Modified existing AWS resources

1. Based on the results of the previous step, determine if any applications are potentially impacted:

- Obtain the ARN and/or tag information for each resource impacted (from step 4, above)
- Go back to your CMDB and determine which application that resource belongs to
- Notify the application owner based on the results of the above steps

1. At this point you will need to follow the appropriate playbook based on the type of application impact or further compromised resources to determine full impact and contain any further compromise.

Part 4: Recover from the Incident

1. After identifying, containing, and eradicating any involved pods, nodes, users, and any other AWS resources follow the steps below to complete recovery (note that some steps below could also be considered as eradication).

1. For resources that were modified during the compromise:

1. If the resource can be destroyed and replaced, do so. For example, Identify the vulnerability that compromised the pods, implement the fix for that vulnerability and start new replacement pods, and then delete the vulnerable pods.

1. If the resource cannot be replaced, either:

1. Restore the resource from a known good backup, or;

1. Prepare a new resource and configure it into the application's infrastructure, while isolating the compromised resource and removing it from the application's infrastructure. Update your CMDB accordingly

1. Either destroy the compromised resource, or continue to leave it isolated for post-incident forensics

1. For resources that were deleted during the compromise:

1. Determine what (if any) application the resource belonged to, by checking your CMDB, or confirming the resource's tag(s) (Check AWS Config if the tags aren't listed in the CloudTrail entry and the [resource is supported by AWS Config](#))

1. If the deleted resource can be restored from backup, commence the restore procedure

1. If the deleted resource cannot be restored from backup, consult your CMDB to obtain the resource's configuration, and recreate the resource and configure it into the application's infrastructure

1. For resources that were created during the compromise:

1. Confirm these have been deleted or isolated for further analysis, per the steps in **Part 2**.

Part 5: Post-Incident Activity

This activity contains two parts. Firstly, some compromised resources may require forensic analysis, either to fulfil regulatory obligations or improve incident handling, both taking input from the root cause analysis that will result from forensic investigation. The second part is a

“sharpen the saw” activity which helps teams to assess their response to the actual incident, determine what worked and what didn’t, update the process based on that information and record these findings.

Firstly, perform any required forensic investigation to determine (for compromised resources) what methods the actors may have used and to determine if additional risks and risk mitigations are required for the resources and/or applications in question.

1. For any compromised resources that have been isolated for further analysis, perform the forensic activity on those resources and incorporate the findings into the post-incident report.

1. Ensure that the CMDB is correctly updated to reflect the current status of all resources and applications impacted

1. Once the analysis is completed, ensure relevant resources are terminated and the resulting data and any artifacts from the analysis maintained and/or entered into the relevant system for record-keeping

Secondly, review the incident itself and the response to it, to determine if anything needs to be changed for handling any similar incidents in the future.

1. Review the incident handling and the incident handling process with key stakeholders identified in Part 1, Step 8

1. Document lessons learned, including attack vector(s) mitigation(s), misconfiguration, etc.

1. Store the artifacts from this process with the application information in the CMDB entry for the application and also in the CMDB entry for the credential compromise response process.

Additional Resources

Incident Response

<https://aws.amazon.com/blogs/security/how-get-started-security-response-automation-aws/>

<https://docs.aws.amazon.com/whitepapers/latest/><https://docs.aws.amazon.com/whitepapers/latest/aws-security-incident-response/>

<https://aws.amazon.com/blogs/security/how-to-automate-incident-response-in-aws-cloud-for-ec2-instances/>

<https://aws.amazon.com/blogs/security/forensic-investigation-environment-strategies-in-the-aws-cloud/>

<https://aws.amazon.com/blogs/security/how-to-automate-forensic-disk-collection-in-aws/>

<https://aws.github.io/aws-eks-best-practices/security/docs/incidents/>

<https://docs.aws.amazon.com/guardduty/latest/ug/guardduty-remediate-kubernetes.html>

GuardDuty

https://docs.aws.amazon.com/guardduty/latest/ug/guardduty_findings-summary.html

Other

https://docs.aws.amazon.com/quicksight/latest/APIReference/API_Reference.html

<https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html>

■ [playbooks/Amazon GuardDuty Playbooks/EKS Findings/IRP-GD-SuccessfulAnonymousAccess-Kubernetes.md](#)

Incident Response Playbook Template

Incident Type

GuardDuty Finding: Discovery:Kubernetes/SuccessfulAnonymousAccess

Introduction

This playbook is provided as a template to customers using AWS products and who are building their incident response capability. You should customize this template to suit your particular needs, risks, available tools and work processes.

Security and Compliance is a shared responsibility between you and AWS. AWS is responsible for “Security of the Cloud”, while you are responsible for “Security in the Cloud”. For more information on the shared responsibility model, [please review our documentation](#).

You are responsible for making your own independent assessment of the information in this document. This document: (a) is for informational purposes only, (b) references current AWS product offerings and practices, which are subject to change without notice, and (c) does not create any commitments or assurances from AWS and its affiliates, suppliers or licensors. This document is provided “as is” without warranties, representations, or conditions of any kind, whether express or implied. The responsibilities and liabilities of AWS to its customers are controlled by AWS agreements, and this document is not part of, nor does it modify, any agreement between AWS and its customers.

Summary

This Playbook

This playbook outlines response steps for incidents involving an API operation that was successfully invoked by the `system:anonymous` user in a Kubernetes cluster. These steps are based on the [NIST Computer Security Incident Handling Guide](#) (Special Publication 800-61 Revision 2) that can be used to:

Gather evidence

Contain and then eradicate the incident

Recover from the incident

Conduct post-incident activities, including post-mortem and feedback processes

Interested readers may also refer to the [AWS Security Incident Response Guide](#) which contains additional resources.

Once you have customized this playbook to meet your needs, it is important that you test the playbook (e.g., Game Days) and any automation (functional tests), update as necessary to achieve the desired results, and then publish to your knowledge management system and train all responders.

Note that some of the incident response steps noted in each scenario may incur costs in your AWS account(s) for services used in either preparing for, or responding to incidents. Customizing these scenarios and testing them will help you to determine if additional costs will be incurred. You can use [AWS Cost Explorer](#) and look at costs incurred over a particular time frame (such as when running Game Days) to establish what the possible impact might be.

In reviewing this playbook, you will find steps that involve processes that you may not have in place today. Proactively preparing for incidents means you need the right resource configurations, tools and services in place that allow you to respond to an incident.

The next section will provide a summary of this incident type, and then cover the five steps (parts 1 - 5) for handling privileged containers.

This Incident Type

An Amazon GuardDuty finding represents a potential security issue detected within your network. GuardDuty generates a finding whenever it detects unexpected and potentially malicious activity in your AWS environment. All GuardDuty finding references in this playbook will be related to the GuardDuty finding JSON that can be seen in the GuardDuty console, downloaded from the GuardDuty or Security Hub console, or exported to S3.

Discovery:Kubernetes/SuccessfulAnonymousAccess

An API commonly used to discover resources in a Kubernetes cluster was invoked by an unauthenticated user.

This finding informs you that an API operation was successfully invoked by the system:anonymous user. API calls made by system:anonymous are unauthenticated. The

observed API is commonly associated with the discovery stage of an attack when an adversary is gathering information on your Kubernetes cluster. This activity indicates that anonymous or unauthenticated access is permitted on the API action reported in the finding and may be permitted on other actions. If this behavior is not expected, it may indicate a configuration mistake or that your credentials are compromised.

Details on what resources are involved with this activity can be found in the [finding details](#).

Incident Response Process

Part 1: Acquire, Preserve, Document Evidence

For Any Incident

1. You become aware of potential indicators of compromise (IoCs). These could come in various forms, but the original source is a GuardDuty finding:

An internal ticketing system (the sources of the ticket are varied and could include any of the means below)

From an alert in one of your monitoring systems either inside or external to AWS (that are ingesting GuardDuty Findings, in AWS, this could include AWS Security Hub)

Alarms or observations that resources have been created or deleted in your account that cannot be accounted for in your CMDB, exist in regions that you do not operate infrastructure in, or themselves have generated alerts ([Amazon Detective](#) is a useful tool for understanding these relationships)

1. Confirm a ticket/case has been raised for the incident. If not, manually raise one

1. Determine and begin to document any end-user impact/experience of the issue. Findings should be documented in the ticket/case related to the incident

1. Open an incident war room using your standard communication tools, and page in the relevant stakeholders

1. In the case of automatically created tickets/cases, verify the finding in GuardDuty (what caused the ticket to be created?)

For This Incident Type

1. Identify the specific EKS cluster impacted. In GuardDuty, this will be in the Resource.EksClusterDetails.Name section of the finding.

*1. Identify Pod, Node, and User information to be used in **Part 2**.*

- Pod information can be found in the Resource.KubernetesWorkloadDetails section of the GuardDuty finding.*

- Node information can be found after determining the pod name. Once you have the pod name you can run the command provided below to determine the node name.
- `kubectrl get pods --namespace -o=jsonpath='{.spec.nodeName}'`
- User information can be found in the `Resource.KubernetesDetails.KubernetesUserDetails` section of the `GuardDuty Finding`.

1. Identify the origination information from which the Kubernetes APIs were called. In `GuardDuty`, Look at the `Resource.Service.Action` section of the finding

- Verify that the IP address is valid for your enterprise users
- Verify that the Location and/or ASN/Organization is known and valid for this request

1. Contact the owner of the cluster where the API operation was invoked by the `system:anonymous` user to verify this was valid and intended.

- If the person states that they did invoke the API using anonymous access, verify that there is a valid requirement for enabling anonymous access:
- Once verified, move on to Part 5 to review the incident, and propose improvements that would either automatically archive findings for valid use of anonymous access within your organization, or eliminate the need to enable anonymous access entirely.
- If the requirement for anonymous access cannot be verified, communicate to stakeholders your intent to remediate and proceed to Part 2.

- *If the person states they did not invoke the API operation or that the `system:anonymous` user access is not valid, or you or the owner of the cluster are unable to verify who invoked the API operation, proceed to **Part 2**.*

Part 2: Contain the Incident

If you determine that the activity is unauthorized, or decide it is prudent to assume so, the first priority is to prevent further compromise without impact to production workloads.

1. *Verify that there is not a valid business requirement for use of the `system:anonymous` user that will cause an impact to production workloads if removed.*

1. *The first step is to examine the permissions that have been granted to the `system:anonymous` user, and determine what permissions are needed. To accomplish this, you need to first understand what permissions the `system:anonymous` user has. You can use an [rbac-lookup tool](#) to list the Kubernetes roles and cluster roles bound to users, service accounts, and groups. An alternative method can be found at this [GitHub page](#).*

```
./rbac-lookup | grep -P 'system:(anonymous)|(unauthenticated)'
```

<code>system:anonymous</code>	<code>cluster-wide</code>	<code>ClusterRole/system:discovery</code>
-------------------------------	---------------------------	---

<code>system:unauthenticated</code>	<code>cluster-wide</code>	<code>ClusterRole/system:discovery</code>
-------------------------------------	---------------------------	---

<code>system:unauthenticated</code>	<code>cluster-wide</code>	<code>ClusterRole/system:public-info-viewer</code>
-------------------------------------	---------------------------	--

1. Next, disassociate the `system:unauthenticated` group from `system:discovery` and `system:basic-user` ClusterRoles, by editing the ClusterRoleBinding.

- **Note:** Make sure to not remove `system:unauthenticated` from the `system:public-info-viewer` cluster role binding, because that will prevent the Network Load Balancer from performing health checks against the API server
- Run the command `kubectl edit clusterrolebindings system:discovery`. This command will open the current definition of `system:discovery` ClusterRoleBinding in your editor as shown in the sample `.yaml` configuration file below:

Please edit the object below. Lines

and an empty file will abort the edit

file will be reopened with the relevant

#

`apiVersion: rbac.authorization.k8s.io/v1`

`kind: ClusterRoleBinding`

`metadata:`

`annotations:`

`rbac.authorization.kubernetes.io/autoupdate: "true"`

`creationTimestamp: "2021-06-17T20:50:49Z"`

`labels:`

`kubernetes.io/bootstrapping: rbac-defaults`

name: system:discovery

resourceVersion: "24502985"

selfLink: /apis/rbac.authorization.k8s.io/v1/clusterrolebindings/system%3Adiscovery

uid: b7936268-5043-431a-a0e1-171a423abeb6

roleRef:

apiGroup: rbac.authorization.k8s.io

kind: ClusterRole

name: system:discovery

subjects:

- *apiGroup: rbac.authorization.k8s.io*

kind: Group

name: system:authenticated

- *apiGroup: rbac.authorization.k8s.io*

```
kind: Group
```

```
name: system:unauthenticated
```

- *Delete the entry for system:unauthenticated group*
- *Repeat the same steps for system:basic-user ClusterRoleBinding.*

1. Continue to provide updates to incident stakeholders on the current state of the incident response.

Part 3: Eradicate the Incident

This is the stage for taking remedial action to minimize the impact of the unintended activities.

1. Using your preferred monitoring tool, access CloudTrail and search for all API actions performed by the AWS EKS resource in the last 90 days:

1. If this tool is a third-party tool such as Splunk, Sumo Logic or others, follow the normal procedure for obtaining log information from that tool

1. If you do not ingest CloudTrail logs into a third-party tool, but do send those logs to Amazon Simple Storage Service (Amazon S3), you will be able to use Amazon Athena to query the logs. The remaining steps will focus on AWS tools to retrieve the necessary information

1. [Create an Athena table referencing the bucket containing your CloudTrail logs](#) that link also includes example queries that can be run in Athena

1. In the Athena console, run a query that shows all API actions taken by the compromised resources post-compromise date/time. From the resulting list, determine which API calls:

- Accessed sensitive data (such as S3 GetObject)

NOTE to get S3 data events CloudTrail must be configured to collect S3 data events prior to the incident. [More information about CloudTrail data events can be found here](#)

- *Created new AWS resources, such as databases, Amazon Elastic Compute Cloud (Amazon EC2) instances, AWS Lambda functions or S3 buckets, etc.*
- *Services that create resources should also be carefully checked; for example, CloudFormation Stacks/StackSets, AWS Firewall Manager Security Policies, AWS Elastic Beanstalk resources, etc.*
- *Created or modified AWS identity resources that could be used to extend a foothold into the account (or other accounts, for example with AWS Security Token Service (AWS STS) API methods such as AssumeRole). Within an account, API methods including (but not limited to) the following should also be investigated:*
 - *CreateUser*
 - *CreateRole*
 - *AssumeRole*
 - *GetToken*
 - *AttachPolicy*
 - *RunInstances* (especially with PassRole)
 - *\Image*
 - *Provider*
 - *Tag*

- Untag
- *Create*
- Delete
- *Update*
- etc.
- Deleted existing AWS resources
- Modified existing AWS resources

Example queries

Get a list of total calls a user has made and whether they were successful

```
select eventname, count(*) as total, eventsource, awsregion, errormessage
```

```
from cloudtrail_logs
```

```
where useridentity.principalid = 'AROIA...'
```

```
group by eventname, eventsource, awsregion, errormessage
```

```
order by total desc;
```

Find all events by a particular user after a given date range (not including Describe/List events):

```
select eventname, useridentity.username, sourceIPAddress, eventtime, requestparameters t
```

```
where (useridentity.username like '%ExampleUser%')
```

```
and eventtime > '2017-06-01T00:00:00Z'
```

```
and (eventname not like '%Describe%' AND eventname not like '%List%')
```

```
order by eventtime asc;
```

For an IAM user:

```
select eventname, useridentity.username, sourceIPAddress, eventtime, requestparameters t
```

```
where (useridentity.username like '%ExampleUser%')
```

```
and eventtime > '2017-06-01T00:00:00Z'
```

```
and (eventname not like '%Describe%' OR eventname not like '%List%')
```

```
order by eventtime asc;
```

1. Based on the results of the previous step, determine if any applications are potentially impacted:

- Obtain the ARN and/or tag information for each resource impacted (from step 4, above)
- Go back to your CMDB and determine which application that resource belongs to
- Notify the application owner based on the results of the above steps

1. At this point you will need to follow the appropriate playbook based on the type of application impact or further compromised resources to determine full impact and contain any further compromise.

Part 4: Recover from the Incident

1. After identifying, containing, and eradicating any involved pods, nodes, users, and any other AWS resources follow the steps below to complete recovery (note that some steps below could also be considered as eradication).

1. For resources that were modified during the compromise:

1. If the resource can be destroyed and replaced, do so. For example, Identify the vulnerability that compromised the pods, implement the fix for that vulnerability and start new replacement pods, and then delete the vulnerable pods.

1. If the resource cannot be replaced, either:

1. Restore the resource from a known good backup, or;

1. Prepare a new resource and configure it into the application's infrastructure, while isolating the compromised resource and removing it from the application's infrastructure. Update your CMDB accordingly

1. Either destroy the compromised resource, or continue to leave it isolated for post-incident forensics

1. For resources that were deleted during the compromise:

1. Determine what (if any) application the resource belonged to, by checking your CMDB, or confirming the resource's tag(s) (Check AWS Config if the tags aren't listed in the CloudTrail entry and the [resource is supported by AWS Config](#))

1. If the deleted resource can be restored from backup, commence the restore procedure

1. If the deleted resource cannot be restored from backup, consult your CMDB to obtain the resource's configuration, and recreate the resource and configure it into the application's infrastructure

1. For resources that were created during the compromise:

1. Confirm these have been deleted or isolated for further analysis, per the steps in **Part 2**.

Part 5: Post-Incident Activity

This activity contains two parts. Firstly, some compromised resources may require forensic analysis, either to fulfill regulatory obligations or improve incident handling, both taking input from the root cause analysis that will result from forensic investigation. The second part is a “sharpen the saw” activity which helps teams to assess their response to the actual incident, determine what worked and what didn’t, update the process based on that information and record these findings.

Firstly, perform any required forensic investigation to determine (for compromised resources) what methods the actors may have used and to determine if additional risks and risk mitigations are required for the resources and/or applications in question.

1. For any compromised resources that have been isolated for further analysis, perform the forensic activity on those resources and incorporate the findings into the post-incident report.

1. Ensure that the CMDB is correctly updated to reflect the current status of all resources and applications impacted

1. Once the analysis is completed, ensure relevant resources are terminated and the resulting data and any artifacts from the analysis maintained and/or entered into the relevant system for record-keeping

Secondly, review the incident itself and the response to it, to determine if anything needs to be changed for handling any similar incidents in the future.

1. Review the incident handling and the incident handling process with key stakeholders identified in Part 1, Step 8

1. Document lessons learned, including attack vector(s) mitigation(s), misconfiguration, etc.

1. Store the artifacts from this process with the application information in the CMDB entry for the application and also in the CMDB entry for the credential compromise response process.

Additional Resources

Incident Response

<https://aws.amazon.com/blogs/security/how-get-started-security-response-automation-aws/>

<https://docs.aws.amazon.com/whitepapers/latest/><https://docs.aws.amazon.com/whitepapers/latest/aws-security-incident-response/>

<https://aws.amazon.com/blogs/security/how-to-automate-incident-response-in-aws-cloud-for-ec2-instances/>

<https://aws.amazon.com/blogs/security/forensic-investigation-environment-strategies-in-the-aws-cloud/>

<https://aws.github.io/aws-eks-best-practices/security/docs/incidents/>

<https://docs.aws.amazon.com/guardduty/latest/ug/guardduty-remediate-kubernetes.html>

GuardDuty

https://docs.aws.amazon.com/guardduty/latest/ug/guardduty_findings-summary.html

Other

https://docs.aws.amazon.com/quicksight/latest/APIReference/API_Reference.html

<https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html>

<https://docs.aws.amazon.com/pdfs/eks/latest/APIReference/eks-api.pdf>

■ `playbooks/Amazon GuardDuty Playbooks/IAM Findings/IAMAnomalousBehavior-Impact.md`

Incident Response Playbook Template

Incident Type

GuardDuty Finding: Impact:IAMUser/AnomalousBehavior

Introduction

This playbook is provided as a template to customers using AWS products and who are building their incident response capability. You should customize this template to suit your particular needs, risks, available tools and work processes.

Security and Compliance is a shared responsibility between you and AWS. AWS is responsible for “Security of the Cloud”, while you are responsible for “Security in the Cloud”. For more information on the shared responsibility model, [please review our documentation](#).

You are responsible for making your own independent assessment of the information in this document. This document: (a) is for informational purposes only, (b) references current AWS product offerings and practices, which are subject to change without notice, and (c) does not create any commitments or assurances from AWS and its affiliates, suppliers or licensors. This document is provided “as is” without warranties, representations, or conditions of any kind, whether express or implied. The responsibilities and liabilities of AWS to its customers are controlled by AWS agreements, and this document is not part of, nor does it modify, any agreement between AWS and its customers.

Summary

This Playbook

This playbook outlines response steps for incidents involving anomolous API usage from an AWS IAM resource. These steps are based on the [NIST Computer Security Incident Handling Guide](#) (Special Publication 800-61 Revision 2) that can be used to:

Gather evidence

Contain and then eradicate the incident

Recover from the incident

Conduct post-incident activities, including post-mortem and feedback processes

Interested readers may also refer to the [AWS Security Incident Response Guide](#) which contains additional resources.

Once you have customized this playbook to meet your needs, it is important that you test the playbook (e.g., Game Days) and any automation (functional tests), update as necessary to achieve the desired results, and then publish to your knowledge management system and train all responders.

Note that some of the incident response steps noted in each scenario may incur costs in your AWS account(s) for services used in either preparing for, or responding to incidents. Customizing these scenarios and testing them will help you to determine if additional costs will be incurred. You can use [AWS Cost Explorer](#) and look at costs incurred over a particular time frame (such as when running Game Days) to establish what the possible impact might be.

In reviewing this playbook, you will find steps that involve processes that you may not have in place today. Proactively preparing for incidents means you need the right resource configurations, tools and services in place that allow you to respond to an incident.

The next section will provide a summary of this incident type, and then cover the five steps (parts 1 - 5) for handling credential compromise.

This Incident Type

An Amazon GuardDuty finding represents a potential security issue detected within your network. GuardDuty generates a finding whenever it detects unexpected and potentially malicious activity in your AWS environment.

Impact: IAMUser/AnomalousBehavior

An API commonly used to tamper with data or processes in an AWS environment was invoked in an anomalous way.

This finding informs you that an anomalous API request was observed in your account. This finding may include a single API or a series of related API requests made in proximity by a single user identity. The API observed is commonly associated with impact tactics where an adversary is trying to disrupt operations and manipulate, interrupt, or destroy data in your

account. APIs for this finding type are typically delete, update, or put operations, such as, DeleteSecurityGroup, UpdateUser, or PutBucketPolicy.

Details on which factors of the API request are unusual for the user identity that invoked the request can be found in the [finding details](#).

Reasons for unexpected API usage could include the following:

The user is new and their behavior has no established baseline

The user has new responsibilities and requires activity outside the established baseline

The user has broad permissions and was acting on a request to perform the detected activity

The user has broad permissions and was engaged in unauthorized or malicious activity

Note This process focuses on AWS IAM Users as principals, but the same process is valid for AWS IAM Roles.

Incident Response Process

Part 1: Acquire, Preserve, Document Evidence

For Any Incident

1. You become aware of potential indicators of compromise (IoCs). These could come in various forms, but the original source is a GuardDuty finding:

- An internal ticketing system (the sources of the ticket are varied and could include any of the means below)
- From an alert in one of your monitoring systems either inside or external to AWS (that are ingesting GuardDuty Findings, in AWS, this could include AWS Security Hub)
- Alarms or observations that resources have been created or deleted in your account that cannot be accounted for in your CMDB, exist in regions that you do not operate infrastructure in, or themselves have generated alerts ([Amazon Detective](#) is a useful tool for understanding these relationships)

1. Confirm a ticket/case has been raised for the incident. If not, manually raise one

1. Determine and begin to document any end-user impact/experience of the issue. Findings should be documented in the ticket/case related to the incident

1. Open an incident war room using your standard communication tools, and page in the relevant stakeholders

1. In the case of automatically created tickets/cases, verify the finding in GuardDuty (what caused the ticket to be created?)

For This Incident Type

1. Identify the specific resources impacted. In GuardDuty, this will be "Resource ID" in the "Overview" section of the finding.

1. Identify the nature of the impact (modified, deleted, etc.). In GuardDuty, Look at the "Anomalous APIs" section of the finding. Look up each in the AWS [API Reference](#)

1. Determine if the Anomalous API call was successful. In GuardDuty, Look at the "Anomalous APIs" section of the finding. There are two types of subcategories: Successfully called and Error Response

1. Identify the origination information from which the APIs were called. In GuardDuty, Look at the "Actor" section of the finding

- Verify that the IP address is valid for your enterprise users.
- Verify that the Location or Organization is known or valid

1. Identify the owner or custodian of the *impacted* resource(s) and add them to the ticket

1. Determine what, if any, data was modified or deleted, and the date and time.

1. Identify the person or system associated the principal that resulted in the GuardDuty finding being generated

- If an IAM user, query the user details [in IAM](#)
- If an IAM role, verify the principal that assumed the role. For roles, this may involve linking the [corporate identity to the assumed role session in the relevant CloudTrail entry](#)

1. Contact the person associated with the principal (**Actor**) to validate the activity. At this point we assume this incident is based on alert generated by GuardDuty and not confirmed malicious activity. Do these steps now if the activities identified by the alert are presumed to be potentially authorized. If the changes made by the Actor are presumed as unauthorized for any reason, delay this until **Part 5**. These presumption should be based up your organization's accepted change practices and risk tolerance.

1. Collate the information obtained in previous steps.

1. Notify the the Actor's manager (or note in the ticket) of your intent to interview the person linked to the AWS principal

1. Interview the Actor to validate whether further investigation is necessary.

- The Actor should be able to say what they did and why they did it.
- If it exists, obtain and document evidence that the activity was authorized. Such evidence may include:
 - Change records in the official change management system.
 - Written instructions from the actor's manager, the resource owners or their managers, etc.
 - Documented request from Incident responders including an Incident record number.

1. Verify evidence provided with their respective sources.

1. Notify the actor's management of the outcome of the interview.

Part 2: Contain the Incident

If you determine that the activity is unauthorized, or decide it is prudent to assume so, the first priority is to prevent further compromise without impact to production workloads.

1. Verify that disabling the credentials of the IAM principal in question will not result in a service outage.

1. Record all permissions from the principal responsible for the activity.

1. Remove permissions from the principal responsible for the activity.

1. If the Principal is an AWS Role, only remove permissions if no other entities need to use it. Instead, do one of the following:

1. Remove the the Actor identified above from whatever authorization system approves the access (Active Directory security groups, etc.)

1. Attach an inline policy with a Condition that excludes an ARN with the above Actors Session Identifier [Use an IAM policy Conditions tp limit access](#)

1. Disable any current Access Keys for the principal responsible for the activity

1. Change the password (or remove console access) for the principal responsible for the activity

1. If required, retain user action logs (such as CloudTrail) further forensic analysis

1. Continue to update stakeholders on the current state of the incident response. These may include:

- Technology Risk organization
- Data owner and management
- General Counsel and/or Privacy Officer
- Manager of the entity whose credentials were used in the incident

Part 3: Eradicate the Incident

This is the stage for taking remedial action to minimize the impact of the unauthorized activities.

1. Using your preferred monitoring tool, access CloudTrail and search for all API actions performed by the principal in the last 90 days:

1. If this tool is a third-party tool such as Splunk, Sumo Logic or others, follow the normal procedure for obtaining log information from that tool

1. If you do not ingest CloudTrail logs into a third-party tool, but do send those logs to Amazon Simple Storage Service (Amazon S3), you will be able to use Amazon Athena to query the logs. The remaining steps will focus on AWS tools to retrieve the necessary information

1. [Create an Athena table referencing the bucket containing your CloudTrail logs](#) that link also includes example queries that can be run in Athena

1. In the Athena console, run a query that shows all API actions taken by the compromised credentials post-compromise date/time. From the resulting list, determine which API calls:

- Accessed sensitive data, such as S3 GetObject (NOTE! To do this, the trail will need to have data events configured for the relevant bucket(s))
- Created new AWS resources, such as databases, Amazon Elastic Compute Cloud (Amazon EC2) instances, AWS Lambda functions or S3 buckets, etc.

- Services that create resources should also be carefully checked; for example, CloudFormation Stacks/StackSets, AWS Firewall Manager Security Policies, AWS Elastic Beanstalk resources, etc.
- Created or modified AWS identity resources that could be used to extend a foothold into the account (or other accounts, for example with AWS Security Token Service (AWS STS) API methods such as AssumeRole). Within an account, API methods including (but not limited to) the following should also be investigated:
 - CreateUser
 - CreateRole
 - AssumeRole
 - GetToken
 - AttachPolicy
 - RunInstances (especially with PassRole)
 - Image
 - Provider
 - Tag
 - Untag

- Create
- *Delete*
- Update*
- etc.
- Deleted existing AWS resources
- Modified existing AWS resources

1. Based on the results of the previous step, determine if any applications are potentially impacted:

1. Obtain the ARN and/or tag information for each resource impacted (from step 4, above)

1. Go back to the CMDB and determine which application that resource belongs to

1. Notify the application owner based on the results of the above steps

1. Based on the above results, if additional credential-obtaining resources have been created (IAM users, roles, EC2 instances with roles, etc.), take the following steps:

1. Disable and/or delete any credentials for those resources, as per the steps outlined in Part 2, Step 1

1. Use your preferred logging tool to analyze CloudTrail again once any such credentials have been deleted or disabled, this time specifying the additionally discovered credentials as a query parameter, as outlined in Part 3, steps 1 through to 6 (this step). Continue to iterate through this process until you discover that there were no further resources created capable of obtaining valid credentials for the AWS account.

1. For each set of newly discovered compromised credentials, repeat the above process

Part 4: Recover from the Incident

1. For resources that were modified during the compromise:

1. If the resource can be destroyed and replaced, do so. For example, EC2 instances in an EC2 Auto Scaling group, with a Launch Configuration referencing a non-compromised AMI, or Launch Templates used to create EC2 instances, terminate the instance, allow EC2 Auto Scaling to adjust capacity as required

1. If the resource cannot be replaced, either:

1. Restore the resource from a known good backup, or;

1. Prepare a new resource and configure it into the application's infrastructure, while isolating the compromised resource and removing it from the application's infrastructure. Update the CMDB accordingly

1. Either destroy the compromised resource, or continue to leave it isolated for post-incident forensics

1. For resources that were deleted during the compromise:

1. Determine what (if any) application the resource belonged to, by checking in the CMDB, or confirming the resource's tag(s) (Check AWS Config if the tags aren't listed in the CloudTrail entry and the [resource is supported by AWS Config](#))

1. If the deleted resource can be restored from backup, commence the restore procedure

1. If you are using immutable architectures or similar deployment methods, redeploy the relevant resources

1. If neither of the above options are possible, consult the CMDB to obtain the resource's configuration, and recreate the resource and configure it into the application's infrastructure.

1. If the CMDB information is inadequate or non-existent, assign resource creation to the relevant personnel in your organization

1. For resources that were created during the compromise:

1. Confirm these have been deleted or isolated for further analysis, per the steps in **Part 3**.

1. Validate that the affected resources were remediated and the expected state of any impacted systems were restored

Part 5: Post-Incident Activity

This activity contains two parts. Firstly, some compromised resources may require forensic analysis, either to fulfil regulatory obligations or improve incident handling, both taking input from the root cause analysis that will result from forensic investigation. The second part is a “sharpen the saw” activity which helps teams to assess their response to the actual incident, determine what worked and what didn’t, update the process based on that information and record these findings.

Firstly, perform any required forensic investigation to determine (for compromised resources) what methods the actors may have used and to determine if additional risks and risk mitigations are required for the resources and/or applications in question.

1. For any compromised resources that have been isolated for further analysis, perform the forensic activity on those resources and incorporate the findings into the post-incident report.

1. Ensure that the CMDB is correctly updated to reflect the current status of all resources and applications impacted

Secondly, review the incident itself and the response to it, to determine if anything needs to be changed for handling any similar incidents in the future.

1. Review the incident handling and the incident handling process with key stakeholders identified in Part 1, Step 8.

1. Document lessons learned, including attack vector(s) mitigation(s), misconfiguration, etc.

1. Store the artifacts from this process with the application information in the CMDB entry for the application and also in the CMDB entry for the credential compromise response process.

Additional Resources

Incident Response

<https://aws.amazon.com/blogs/security/how-get-started-security-response-automation-aws/>

<https://docs.aws.amazon.com/whitepapers/latest/aws-security-incident-response-guide/welcome.html>

<https://aws.amazon.com/blogs/security/how-to-automate-incident-response-in-aws-cloud-for-ec2-instances/>

<https://aws.amazon.com/blogs/security/forensic-investigation-environment-strategies-in-the-aws-cloud/>

<https://aws.amazon.com/blogs/security/how-to-automate-forensic-disk-collection-in-aws/>

GuardDuty

https://docs.aws.amazon.com/guardduty/latest/ug/guardduty_findings-summary.html

Other

https://docs.aws.amazon.com/quicksight/latest/APIReference/API_Reference.html

<https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html>

■ [playbooks/Amazon GuardDuty Playbooks/Malware Findings/EC2MalwareFindings.md](#)

Incident Response Playbook Template

Incident Type

GuardDuty Finding: Execution:EC2/MaliciousFile

Introduction

This playbook is provided as a template to customers using AWS products and who are building their incident response capability. You should customize this template to suit your particular needs, risks, available tools and work processes.

Security and Compliance is a shared responsibility between you and AWS. AWS is responsible for “Security of the Cloud”, while you are responsible for “Security in the Cloud”. For more information on the shared responsibility model, [please review our documentation](#).

You are responsible for making your own independent assessment of the information in this document. This document: (a) is for informational purposes only, (b) references current AWS product offerings and practices, which are subject to change without notice, and (c) does not create any commitments or assurances from AWS and its affiliates, suppliers or licensors. This document is provided “as is” without warranties, representations, or conditions of any kind, whether express or implied. The responsibilities and liabilities of AWS to its customers are controlled by AWS agreements, and this document is not part of, nor does it modify, any agreement between AWS and its customers.

Summary

This Playbook

This playbook outlines response steps for incidents involving deployment of a privileged container. These steps are based on the [NIST Computer Security Incident Handling Guide](#) (Special Publication 800-61 Revision 2) that can be used to:

Gather evidence

Contain and then eradicate the incident

Recover from the incident

Conduct post-incident activities, including post-mortem and feedback processes

Interested readers may also refer to the [AWS Security Incident Response Guide](#) which contains additional resources.

Once you have customized this playbook to meet your needs, it is important that you test the playbook (e.g., Game Days) and any automation (functional tests), update as necessary to achieve the desired results, and then publish to your knowledge management system and train all responders.

Note that some of the incident response steps noted in each scenario may incur costs in your AWS account(s) for services used in either preparing for, or responding to incidents. Customizing these scenarios and testing them will help you to determine if additional costs will be incurred. You can use [AWS Cost Explorer](#) and look at costs incurred over a particular time frame (such as when running Game Days) to establish what the possible impact might be.

Throughout this playbook you will see references to AWS API calls that can assist for gathering information or making modifications to resources. These commands can be executed in multiple ways including from CloudShell, within the AWS Management Console, or from your favorite IDE or CLI utility.

In reviewing this playbook, you will find steps that involve processes that you may not have in place today. Proactively preparing for incidents means you need the right resource configurations, tools and services in place that allow you to respond to an incident.

The next section will provide a summary of this incident type, and then cover the five steps (parts 1 - 5) for handling privileged containers.

This Incident Type

An Amazon GuardDuty finding represents a potential security issue detected within your network. GuardDuty generates a finding whenever it detects unexpected and potentially malicious activity in your AWS environment. All GuardDuty finding references in this playbook will be related to the GuardDuty finding JSON that can be seen in the GuardDuty console, downloaded from the GuardDuty or Security Hub console, or exported to S3.

Execution:EC2/MaliciousFile

A malicious file has been detected on an EC2 instance

This finding indicates that the GuardDuty Malware Protection scan has detected one or more malicious files on the listed EC2 instance within your AWS environment. This listed instance might be compromised.

Details on what resources are involved with this activity can be found in the [finding details](#).

Incident Response Process

Part 1: Acquire, Preserve, Document Evidence

For any Incident

1. You become aware of potential indicators of compromise (IoCs). These could come in various forms, but the original source is a GuardDuty finding:

An internal ticketing system (the sources of the ticket are varied and could include any of the means below)

From an alert in one of your monitoring systems either inside or external to AWS (that are ingesting GuardDuty Findings, in AWS, this could include AWS Security Hub)

Alarms or observations that resources have been created or deleted in your account that cannot be accounted for in your CMDB, exist in regions that you do not operate infrastructure in, or themselves have generated alerts [Amazon Detective](#) is a useful tool for understanding these relationships)

1. Confirm a ticket/case has been raised for the incident using your designated ticketing system. If not, manually raise one.

1. Determine and begin to document any end-user impact/experience of the issue. Findings should be documented in the ticket/case related to the incident

1. Open an incident war room using your standard communication tools, and page in the relevant stakeholders

1. In the case of automatically created tickets/cases, verify the finding in GuardDuty (what caused the ticket to be created?)

For This Incident Type

1. Identify the specific EC2 instance impacted documenting the Instance ID, Account Id, and Region associated with the Instance. In GuardDuty this will be in the Overview section of the finding details.

2. Identify the volume Amazon Resource Name (ARN) where the malware is located. In GuardDuty this will be in the Threats Detected section and labeled Volume ARN.

3. Identify and document the location and name of the malware identified by GuardDuty. In GuardDuty this will be in the Threats Detected section and labeled File path and File name.

4. Identify what GuardDuty finding caused the malware scan to occur. In GuardDuty this will be in the Malware scan details labeled Trigger finding ID. This information will help determine what this malware was doing, for example command and control activity. To see a list of findings that initiate a malware protection scan please visit the [GuardDuty Documentation](#)

5. List security group information that will be used to quarantine the instance in the containment section using the AWS CLI command below.

```
aws ec2 describe-instances \
```

```
--instance-ids $Instance_ID \
```

```
--query "Reservations[0].Instances[0].NetworkInterfaces[].Groups"
```

5. Acquire images of the instance volumes using the AWS CLI command below.

```
aws ec2 create-snapshot \
```

```
--volume $Data_Volume_ID \
```

```
--description "Data Volume Snapshot of Bad Web Server from Account $Account_ID in  
Region $Region" \
```

--tag-specifications

```
'ResourceType=snapshot,Tags=[{Key=Name,Value="${Account_ID}_${Region}_BadWebServer_DataVolume_${date +%Y-%m-%d}_${date -u +%H:%M:%SZ}"}]'
```

6. Enable termination protection for the instance using the AWS CLI command below. This step is needed if you want to preserve the instance for forensics.

```
aws ec2 modify-instance-attribute \
```

```
--instance-id $Instance_ID \
```

```
--attribute disableApiTermination \
```

```
--value true
```

7. Disable the "DeleteOnTermination" setting for all volumes using the command below. This will ensure that if the instance is deleted the volumes are not. This step is needed if you want to preserve the instance for forensics.

```
aws ec2 modify-instance-attribute \
```

```
--instance-id $Instance_ID \
```

```
--block-device-mappings "[{ \"DeviceName\":  
\"/dev/xvda\", \"Ebs\": { \"DeleteOnTermination\": false } }]"
```

8. Ensure instance shutdown behavior is set to "Stop" versus "Terminate" with the command below to ensure stopping or shutting down does not terminate the instance.

```
aws ec2 modify-instance-attribute \
```

```
--instance-id $Instance_ID \
```

```
--attribute instanceInitiatedShutdownBehavior \
```

```
--value stop
```

9. Tag the instance for future identification during containment with the AWS CLI command below.

```
aws ec2 create-tags \
```

```
--resources $Instance_ID \
```

```
--tags Key=Status,Value=Quarantine
```

10. Follow your defined playbook for performing host based forensics on EC2. If you do not have a defined playbook AWS has published different resources to help you get started.

[Forensic investigation environment strategies in the AWS Cloud](#), [How to automate forensic disk collection in AWS](#), [Automated Forensics Orchestrator for Amazon EC2](#), and a hands on workshop [EC2 Incident Response and Forensic Analysis Workshop](#)

Part 2: Contain the Incident

If you determine that the activity is unauthorized, or decide it is prudent to assume so, the first priority is to prevent further compromise without impact to production workloads.

1. Determine if disabling the EC2 instance in question will not result in a service outage.

2. If the EC2 instance is a part of an Auto Scaling group it will need to be removed from the group. You can remove the instance from the auto-scaling group with the AWS CLI command below.

```
aws autoscaling detach-instances \
```

```
--instance-ids $Instance_ID \
```

```
--auto-scaling-group-name
```

3. Next the instance will need to be deregistered if attached to an Elastic Load Balancer to prevent requests from being sent.

```
aws elb deregister-instances-from-load-balancer \
```

```
--instances $Instance_ID \
```

```
--load-balancer-name
```

4. Remove the instance profile from the instance with the AWS CLI command below to remove IAM permissions.

```
aws ec2 disassociate-iam-instance-profile --association-id $Association_ID
```

5. Prevent further connections to Isolate and quarantine the instance via security groups with the AWS CLI commands below in step 5. (Note that this isolation security group needs to be created beforehand. This can be a security group that doesn't allow any traffic, or traffic allowed specific to your use case)

Note

Be aware that there is currently no way to completely isolate/contain/quarantine an Instance using Security Groups, due to Connection Tracking. While Security Groups can be used to prevent future connections, they CANNOT be used to terminate existing Tracked Connections, as noted in the documentation [Connection Tracking](#). The following are the only methods to completely isolate/contain/quarantine an Instance:

1. Shutting the instance down

- *You must ensure that you've collected all necessary volatile data needed for investigations like memory, disk, and/or system logs BEFORE shutting down!*

2. Using the endpoint's local/native utilities

- *Implementing OS/host firewall rules to terminate/block connections at the system level*

3. Using an Endpoint Agent

- *Leveraging an EDR or similar endpoint agent with containment/isolation capabilities*

4. NACL's

- *Note that using NACL's will affect an entire subnet and cannot be used to isolate a singular Instance*

5. Disconnecting the ENI

- *This is only possible for non-Primary ENI's as you cannot disconnect a primary ENI*

*aws ec2 modify-instance-attribute *

```
--instance-id $Instance_ID \
```

```
--groups $Isolation_SG_ID
```

You can also automate the incident response and forensics capabilities utilizing AWS services. Examples on how to automate this activity can be found at this [AWS documentation](#)

Part 3: Eradicate the Incident

This is the stage for taking remedial action to minimize the impact of the unauthorized activities.

1. To remediate the compromised EC2 instance, follow the steps below

1. Investigate the potentially compromised instance for malware and remove any discovered malware. You may check the [AWS Marketplace](#) to see if there are helpful partner products to identify and remove malware.

2. If you are unable to stop unauthorized activity on your EC2 instance, we recommend that you terminate the compromised EC2 instance and replace it with a new instance as needed or only leave it running for additional data acquisition. Use the AWS CLI Command below to shut the system down.

```
aws ec2 stop-instances --instance-ids $Instance_ID
```

3. If it is determined in your forensic analysis (part 5) that your AMI is potentially compromised you will need to follow your playbook for creating or recreating AMIs to ensure full eradication of any potential compromise.

Part 4: Recover from the Incident

1. For resources that were modified during the compromise:

1. If the resource can be removed and replaced, do so. For example, EC2 instances in an EC2 Auto Scaling group, with a Launch Configuration referencing a non-compromised AMI, or Launch Templates used to create EC2 instances, terminate the instance, allow EC2 Auto Scaling to adjust capacity as required

1. If the resource cannot be replaced, either:

1. Restore the resource from a known good backup, or;

1. Prepare a new resource and configure it into the application's infrastructure, while isolating the compromised resource and removing it from the application's infrastructure. Update the CMDB accordingly

1. Either remove the compromised resource, or continue to leave it isolated for post-incident forensics

1. For resources that were deleted during the compromise:

1. Determine what (if any) application the resource belonged to, by checking in the CMDB, or confirming the resource's tag(s) (Check AWS Config if the tags aren't listed in the CloudTrail entry and the [resource is supported by AWS Config](#))

1. If the deleted resource can be restored from backup, commence the restore procedure

1. If you are using immutable architectures or similar deployment methods, redeploy the relevant resources

1. If neither of the above options are possible, consult the CMDB to obtain the resource's configuration, and recreate the resource and configure it into the application's infrastructure.

1. If the CMDB information is inadequate or non-existent, assign resource creation to the relevant personnel in your organization

1. For resources that were created during the compromise. For more details on what to do if you notice unauthorized activity refer to [this knowledge center article](#)

*1. Confirm these have been deleted or isolated for further analysis, per the steps in **Part 3**.*

1. Validate that the affected resources were remediated and the expected state of any impacted systems were restored

Part 5: Post-Incident Activity

This activity contains two parts. Firstly, some compromised resources may require forensic analysis, either to fulfil regulatory obligations or improve incident handling, both taking input from the root cause analysis that will result from forensic investigation. The second part is a “sharpen the saw” activity which helps teams to assess their response to the actual incident, determine what worked and what didn’t, update the process based on that information and record these findings.

Firstly, perform any required forensic investigation to determine (for compromised resources) what methods the actors may have used and to determine if additional risks and risk mitigations are required for the resources and/or applications in question.

1. For any compromised resources that have been isolated for further analysis, perform the forensic activity on those resources and incorporate the findings into the post-incident report. For an example, please refer to the [Automated Forensics Orchestrator for Amazon EC2](#)

1. Ensure that the CMDB is correctly updated to reflect the current status of all resources and applications impacted

Secondly, review the incident itself and the response to it, to determine if anything needs to be changed for handling any similar incidents in the future.

1. Review the incident handling and the incident handling process with key stakeholders identified in Part 1, Step 8.

1. Document lessons learned, including attack vector(s) mitigation(s), misconfiguration, etc.

1. *Store the artifacts from this process with the application information in the CMDB entry for the application and also in the CMDB entry for the credential compromise response process.*

Additional Resources

Incident Response

<https://aws.amazon.com/blogs/security/how-get-started-security-response-automation-aws/>

<https://docs.aws.amazon.com/whitepapers/latest/aws-security-incident-response-guide/welcome.html>

<https://aws.amazon.com/blogs/security/how-to-automate-incident-response-in-aws-cloud-for-ec2-instances/>

<https://aws.amazon.com/blogs/security/forensic-investigation-environment-strategies-in-the-aws-cloud/>

<https://aws.amazon.com/blogs/security/how-to-automate-forensic-disk-collection-in-aws/>

GuardDuty

https://docs.aws.amazon.com/guardduty/latest/ug/guardduty_findings-summary.html

Preperation

Security and Networking sections in [Best practices for Amazon EC2](#).

[Amazon EC2 security groups for Linux instances](#) and [Amazon EC2 security groups for Windows instances](#).

[Tips for securing your EC2 instances \(Linux\)](#).

[AWS security best practices](#)

[Infrastructure Domain Incidents on AWS](#)

Browse for further assistance on the AWS developer forums:

<https://forums.aws.amazon.com/index.jspa>*

If you are a Premium Support package subscriber, you can submit a [technical support](#) request.

AI-Powered Incident Response Automation

This README relates specifically to this directory subsection of the aws-incident-response-playbook repository. The markdown files in this directory are to be used to create - and then use - a set of guidance files that can be used with your AI-powered IDE of choice and will assist you to respond to security incidents.

These markdown documents are created to be used as templates only. They should be customized by administrators working with AWS to suit their particular needs, risks, available tools and work processes. These files are not official AWS documentation and are provided as-is to customers using AWS products and who are looking to improve their incident response capability, by powering it with artificial intelligence ("AI").

These markdown documents are written to facilitate editing and consumption into a variety of integrated development environments ("IDE"s) as guidance files. In Kiro, these are known as "steering files" and in Claude Code, they are known as "skills". In both examples, the files are written in markdown for consumption into the relevant IDE.

Note that you need to copy the files in this repo into the correct directory in your IDE. In Kiro, we recommend setting up a new project, and defining a [workspace](#) and keeping your steering files in the steering directory *for the workspace*. Steering files specific to the workspace will take priority over global steering files. This also means that you don't risk having steering files in the global workspace that you don't want there, and having kiro reference those files when you are in a workspace for a project completely unrelated to those steering files. Refer to the sections below to ensure you have copied the files to the correct location for your IDE.

For Kiro, users should do similar preparation to move steering files to `.kiro/steering/` folder

The markdown documents included cover several common scenarios faced by AWS customers. They outline steps based on the [NIST Computer Security Incident Handling Guide](#) (Special Publication 800-61 Revision 3) and codify these steps as structured guidance that AI agents can use to assist a human operator investigate and resolve an incident. This includes the following steps outlined by NIST:

Gather evidence

Contain and then eradicate the incident

Recover from the incident

Conduct post-incident activities, including post-mortem and feedback processes

Usage

Kiro IDE

1. Copy the `ai-playbooks/steering/steering-irp-core.md` into your workspace's `.kiro/steering` directory
2. Copy the `ai-playbooks/steering/reference` directory and all files in it into the `.kiro/steering` directory

```
mkdir -p .kiro/steering && cp ai-playbooks/steering/steering-irp-core.md  
.kiro/steering/ && cp -r ai-playbooks/steering/reference .kiro/steering/
```

File types

There are three categories of markdown files we have created for this AI-powered incident response solution:

Steering Factory: Use the

`.kiro-steering/steering-factory/steering-factory-creation-guide.md` to create your own incident response steering files. This file is *_only_* used when you want to create new `steering-irp-.md` steering files

Steering Core: The `.kiro-steering/steering-irp-core.md` file is *_always_* used when you are using the Kiro IDE to investigate and respond to a security incident. The `steering-irp-core.md` file can be saved under `.kiro/steering/` within a given workspace. We recommend you create a [separate workspace](#) specifically for incident response.

Steering Incident Response: These files are invoked as needed based on the initial problem description given by the operator; the steering core file helps Kiro make a decision about which incident-specific playbook to load. These files are also stored under `.kiro/steering/` along with the core file.

How the files direct Kiro IDE

The core file (`steering-irp-core.md`) is set to inclusion: *always*, so it's loaded into every conversation automatically. It acts as the orchestrator — it defines the NIST 800-61 lifecycle, contains keyword pattern matching logic to classify the incident type, and then tells the AI which specific playbook to pull in.

The incident-specific files (such as "credential compromise", "unintended data access", and "ransomware") are all inclusion: *manual*. They don't load unless explicitly invoked. But the

core file's keyword matching logic determines which one(s) to invoke based on what the operator describes.

So the flow is:

User describes a security incident

Core file (always loaded) kicks in, analyzes the prompt against its keyword patterns

Core file directs the AI to invoke the appropriate manual steering file(s)

The specific playbook guides the actual IR steps

It's essentially a router pattern — one always-on dispatcher that selectively activates specialized playbooks on demand. And since each manual file also lists all three playbooks in its description front matter, the AI knows about the other playbooks even when it's deep in one specific response, so it can pivot if the incident turns out to be multi-vector (e.g., credential compromise leading to ransomware).

config:

flowchart:

padding: 15

flowchart TD

*A[User describes security incident] --> B[steering-irp-core.md
always loaded]*

*B --> C{Keyword Pattern
Matching}*

*C -->|credential, access key,
IAM user, GuardDuty| D[steering-irp-credential-compromise.md]*

*C -->|S3 bucket, data access,
public bucket, object ACL| E[steering-irp-data-access.md]*

*C -->|Ransomware, ransom,
encrypted files, locked out| F[steering-irp-ransomware.md]*

C -->|no clear match| D

*D --> G[Execute NIST 800-61 Lifecycle
Evidence → Contain → Eradicate → Recover → Post-Incident]*

E --> G

F --> G

*G --> H{Multi-vector
detected?}*

*H -->|Yes| I[Invoke additional
playbook]*

I --> G

*H -->|No| J[Root cause analysis
(actions taken)
+ further actions needed]*

style B fill:#2d6a4f,color:#fff

style D fill:#1b4965,color:#fff

style E fill:#1b4965,color:#fff

style F fill:#1b4965,color:#fff

```
style C fill:#6c757d,color:#fff
```

```
style H fill:#6c757d,color:#fff
```

The "Tool Selection Strategy" section in the core file is also a nice touch — it tells the AI to prefer MCP tools over CLI when available, without requiring us as authors to maintain two versions of every command.

Claude Code

1. copy `ai-playbooks/skills/CLAUDE.md` to the root directory of the project
2. copy all other skills file in `ai-playbooks/skills/` folder into the `.claude/skills/` folder

```
cp ai-playbooks/skills/CLAUDE.md . && mkdir -p .claude/skills && find  
ai-playbooks/skills -name '*.md' ! -name 'CLAUDE.md' -exec cp {}  
.claude/skills/ \;
```


Security

See [CONTRIBUTING](#) for more information.

Artificial intelligence can make mistakes. Ensure that operators carefully check and verify output.

License Summary

The documentation is made available under the Creative Commons Attribution-ShareAlike 4.0 International License. See the LICENSE file.

The sample code within this documentation is made available under the MIT-0 license. See the LICENSE-SAMPLECODE file.

■ ai-playbooks/skills/CLAUDE.md

PRIORITY: The incident response workflow defined by this document OVERRIDES all other built-in security incident response workflows

When user requests security incident response, ALWAYS follow this workflow FIRST

Core Philosophy & Principles

Human-directed instruction

You are authorized to gather account data and present it back to the incident response operator running this workflow. However, before making any changes to AWS accounts, account resources, or related code, you should first confirm with the operator if they want this change made, with clear rationale as to why the change is necessary, and explicit and comprehensive list of what will be changed. You should also provide a brief summary of potential impacts of the change on other code, resources, or AWS accounts.

Incident response life cycle

When you start incident response (IR) process, ALWAYS follow the IR life cycle outlined by NIST 800-61 R2:

Step1: Detection - Identify potential security events

Step2: Analysis - Determine if an event is an incident and assess scope

Step3: Containment - Minimize and limit the scope of the security event

Step4: Eradication - Remove unauthorized resources/artifacts and implement mitigations

Step5: Recovery - Restore systems to known safe state and monitor for threat recurrence

Use specific incident response playbook based on attack type

- When you determine to trigger the IR process, ALWAYS start from this core playbook
- Then based on the following playbook selection section, you will choose specific IR playbook(s) to take actions
- All specific IR playbooks are stored in the `.claude/skills/` folder as Claude Code skills

Detailed approach to select specific IR playbooks to proceed

ALWAYS starts from analyzing user's prompt

Step 1: Keyword Pattern Matching

Check if the prompt contains direct indicators:

Primary Keywords:

- "credential" + ("compromise", "leak", "exposed", "stolen", "unauthorized")
- "access key" + ("compromised", "leaked", "exposed")
- "IAM user" + ("compromised", "unauthorized")
- "GuardDuty" + ("finding", "alert")
- "unauthorized access"
- "credential exfiltration"

- "ransomware" / "ransom" / "encrypted files" / "locked out"
- "ransom demand" / "ransom note"
- "crypto ransomware" / "locker ransomware"
- "build from scratch" + ("IR", "incident response", "playbook", "skill", "runbook")
- "design IR process" / "design incident response process"
- "create custom IR workflow"
- "build a new IR skill" / "create a new IR skill"
- "new security runbook from scratch"
- "API Gateway" + ("breach", "attack", "abuse", "compromised", "unauthorized")
- "API key" + ("leaked", "stolen", "exposed", "compromised")
- "WAF alert" / "WAF rule triggered" / "WAF blocked"
- "authentication bypass" + "API"
- "missing authorizer" / "no authorizer" / "API Gateway authorizer"
- "BOLA" / "IDOR" / "broken object level authorization"

- "API scraping" / "API abuse" / "abnormal API invocation"
- "API Gateway" + ("4XX spike", "5XX spike", "invocation spike")

Secondary Keywords:

- "suspicious activity"
- "unknown API calls"
- "billing spike" / "unexpected costs"
- "security alert"
- "CloudTrail" + ("suspicious", "unauthorized")
- "instance unreachable" + ("encrypted", "locked", "ransom")
- "files encrypted" / "data encrypted" / "objects inaccessible"
- "strange bucket" / "unknown bucket" / "unexpected S3 bucket" / "suspicious bucket name"
- "unrecognized resource" / "unknown resource" / "unexpected resource"

Step 2: Context Analysis

Check for incident characteristics mentioned:

1. Alert Sources:

- GuardDuty findings mentioned
- Security Hub alerts
- CloudWatch alarms on IAM
- AWS Config non-compliance
- Billing anomalies
- External notification (researcher, tip)

2. Suspicious Activities:

- Unfamiliar IAM users/roles created
- New access keys on existing users
- API calls from unusual locations/IPs
- Unauthorized resource creation (EC2, Lambda, S3)
- IAM policy modifications
- CloudTrail logging disabled

3. Timeline Indicators:

- "First seen" timestamps
- "Started happening" timeframes
- Recent IAM changes

4. Ransomware Indicators:

- Ransom demand or ransom note received
- Files or S3 objects encrypted with unknown keys
- EC2 instances unreachable despite correct network configuration
- EBS volume encryption changes
- Unusual data transfer patterns (potential exfiltration before encryption)
- Anti-malware or endpoint protection alerts

5. Builder Indicators (route to skill-irp-builder):

- User describes an incident type but no existing skill covers it
- User explicitly requests building/designing a new workflow
- User says "from scratch", "custom", or "design" in authoring context

Dispatch: API Security Breach Skill

- If API Gateway keywords match -> invoke `skill-irp-api-security-breach`

Dispatch: Builder Skill

- If Builder Indicators match -> invoke `skill-irp-builder`
- `skill-irp-builder` is an authoring workflow, not a live IR workflow — do not combine it with live IR skills

Default option

- If no keywords from Step 1 or Step 2 match, fall back to `skill-irp-credential-compromise` as the starting point.
- The default does NOT override or suppress other skills — if keyword or context analysis matches multiple attack types, invoke ALL applicable skills in parallel.

Expected behaviour

- You start the incident response based on this core playbook
- Then decide which specific IR skill(s) to use from `.claude/skills/`
- **Re-evaluate skill selection as new evidence emerges** — if investigation findings reveal additional attack types (e.g., ransomware indicators discovered mid-investigation), invoke the corresponding additional skill(s) immediately rather than continuing with only the initially selected skill(s)
- Follow specific IR skill(s) to walk through the incident response life cycle
- Presents critical findings to user, and ask for approval **WHENEVER** you need to change any resources or their configurations
- By end of the process, **ALWAYS** present a root cause analysis to user, actions taken, and if any further actions still needed.

Investigation Resilience Principle

During Phase 2 (Analysis), some investigation paths may be blocked — for example, an IAM policy or SCP may deny access to certain APIs, a service may be disabled, or expected data may be in a different location than the skill prescribes. **Do not stop the investigation when blocked.** Apply reasoning to find an equivalent fallback method to retrieve the same data through a different path.

Examples of fallback thinking:

- `cloudtrail:LookupEvents` denied → find the CloudTrail S3 trail via `describe-trails`, then download and parse `.json.gz` log files directly from S3
- GuardDuty API denied → look for equivalent signals in CloudTrail management events, Security Hub, or AWS Config
- S3 server access logs unavailable → check CloudTrail S3 data events if enabled
- Direct API access denied → check if data is available via Resource Explorer, AWS Config recorded state, or a related service

Apply this principle broadly across all IR phases: if a specific tool or API is unavailable, reason about what other data sources in the AWS ecosystem could provide equivalent evidence before escalating or stopping.

Tool Selection Strategy

When executing IR actions, use the most efficient tool available in your environment. Follow this priority order:

1. **MCP tools (preferred):** If an MCP server is available (e.g., AWS API MCP Server with `call_aws`), use it for AWS API calls. MCP tools offer structured responses, better error handling, and batch execution (up to 20 parallel calls) which is critical for IR speed.
2. **AWS CLI (fallback):** If MCP tools are not available, execute the equivalent `aws` CLI commands as specified in the IR skills.

The CLI commands in each IR skill represent the logical operations to perform. Translate them to MCP calls when MCP is available rather than shelling out CLI commands.

When MCP batch execution is especially valuable during IR:

- Checking multiple resources simultaneously (e.g., public access settings across all S3 buckets)
- Querying CloudTrail for multiple access keys or event types in parallel
- Listing IAM users, roles, and access keys across the account
- Describing multiple EC2 instances or security groups at once

Important: Do NOT attempt to detect MCP availability — you already know what tools you have access to. Simply use the best available tool for each operation.

MANDATORY:

- DO NOT automatically delete or change any existing resources and their configurations without user approval
 - For read-only actions, try to action automatically where applicable
-

description: Incident response playbook for AWS API Gateway security breaches. Use this skill when responding to unauthorized API access, leaked or exposed API keys, API authentication bypass, missing or misconfigured API authorizers, abnormal API invocation volume or scraping, injection attacks via API endpoints, broken object-level authorization (BOLA/IDOR), data exfiltration through API Gateway, WAF bypass, AWS WAF alerts, CloudWatch alarms on API errors or invocation spikes, Security Hub findings on API Gateway misconfigurations, customer reports of API abuse, or billing spikes from unexpected API Gateway or Lambda invocations.

Playbook: API Gateway Security Breach

Incident Type

API Gateway Security Breach — When an AWS API Gateway endpoint (REST, HTTP, or WebSocket) has been attacked, abused, or misconfigured in a way that exposes backend systems or data to unauthorized actors. This includes leaked API keys, authentication bypass due to missing or misconfigured authorizers (IAM, Lambda, Cognito), abnormal request volume indicating scraping or abuse, injection attacks passed through the API to backend Lambda functions or data stores, broken object-level authorization allowing cross-tenant data access, and large-scale data exfiltration through the API layer. The application team owns the API Gateway infrastructure; the security team leads the response with application team support. Business impact includes sensitive data breach, service disruption for legitimate users, and downstream system compromise if the attacker pivots from the API to backend services.

Quick Reference

| Phase | Key Action | Verification |

|-----|-----|-----|

| Evidence | Identify affected API + stage, check authorizer config, query access/WAF logs, establish timeline | API ID, attacker IP(s), attack type, and timeline documented |

| Contain | Throttle stage + block attacker IP(s) via WAF rate-based rule and IP set | No new attack traffic for 15+ min, CloudWatch metrics return to baseline |

| Eradicate | Remove unauthorized API keys/IAM entities, restore API definition and authorizer config | API definition matches source control, all methods have correct authorizer |

| Recover | Re-enable stage, issue new API keys to legitimate clients, restore backend data | Legitimate clients confirmed working, metrics at baseline, no new CloudTrail events |

| Post-Incident | Document lessons learned, update defenses, notify stakeholders | Report filed, WAF rules hardened, playbook updated |

Part 1: Acquire, Preserve, Document Evidence

1.1 Identify the Alert Source

Common sources for API Gateway security breach alerts:

Primary Signals (often the first indicator):

- **AWS WAF** — rate limit rule exceeded, AWS managed rule group match (SQLi, XSS, known bad IPs), geo-block triggered, or custom rule fired
- **CloudWatch alarm** — spike in `4XXError` or `5XXError` metric on an API Gateway stage
- **CloudWatch alarm** — abnormal `Count` (total invocation volume) on an API Gateway stage
- **Customer/user report** — "API returning unexpected data", "seeing other users' records", or "our API is being hammered"

- **Billing alert** — unexpected spike in API Gateway invocation costs or Lambda invocation/duration costs driven by abnormal request volume

Secondary Signals (corroborating or configuration-based):

- **Security Hub** — `APIGateway.1` (access logging not enabled on stage)
- **Security Hub** — `APIGateway.2` (stage not using TLS 1.2 minimum)
- **Security Hub** — `APIGateway.9` (execution logging not enabled)
- **CloudTrail** — management-plane changes to API Gateway: new deployments, authorizer modifications, WAF web ACL disassociated, stage settings changed
- **GuardDuty** — credential-based finding (e.g., `UnauthorizedAccess:IAMUser/`) *where the affected principal has permissions to manage API Gateway resources*

1.2 Identify the Affected API and Stage

List all API Gateway REST APIs

```
aws apigateway get-rest-apis \
```

```
--query 'items[][id,name,createdDate]' \
```

```
--output table
```

List all stages for a specific API

```
aws apigateway get-stages \
```

```
--rest-api-id \
```

```
--query 'item[][stageName,createdDate,lastUpdatedDate,webAc1Arn]' \
```

```
--output table
```

Get full stage detail (logging, throttling, etc.)

```
aws apigateway get-stage \
```

```
--rest-api-id \
```

```
--stage-name
```

MCP batch opportunity: run get-stages across all API IDs in parallel

Document:

- *[] API ID and name*
- *[] Stage name and last modified timestamp*

- `[]` WAF web ACL ARN (if associated)
- `[]` Whether access logging is enabled (`accessLogSettings.destinationArn`)
- `[]` Whether execution logging is enabled (`methodSettings..loggingLevel`)

1.3 Check Authorizer Configuration

List authorizers on the API

```
aws apigateway get-authorizers \
```

```
--rest-api-id \
```

```
--query 'items[].[id,name,type,authorizerUri,identitySource]' \
```

```
--output table
```

Check resource policy (if using resource)

```
aws apigateway get-rest-api \
```

```
--rest-api-id \
```

```
--query 'policy'
```

List API keys in use

```
aws apigateway get-api-keys \
```

```
--include-values \
```

```
--query 'items[][id,name,enabled,createdDate,lastUpdatedDate]' \
```

--output table

Document:

- [] Authorizer type (IAM / Cognito / Lambda / None)
- [] Whether API key requirement is enforced on methods
- [] Whether any authorizers were recently modified

1.4 Analyze API Gateway Access Logs

Access logs are delivered to a CloudWatch Logs group configured on the stage.

Find the access log group for the stage

```
aws apigateway get-stage \
```

```
--rest-api-id \
```

```
--stage-name \
```

```
--query 'accessLogSettings.destinationArn'
```

Query access logs for high error rate

```
aws logs start-query \
```

```
--log-group-name \
```

```
--start-time $(date -u -v-1H +%s) \
```

```
--end-time $(date -u +%s) \
```

```
--query-string '
```

```
fields @timestamp, ip, httpMethod, resourcePath, status, responseLength
```

```
| filter status >= 400
```

```
| stats count() as errorCount by ip, resourcePath, status
```

```
| sort errorCount desc
```

```
| limit 50'
```

Wait for query to complete, then retr

```
aws logs get-query-results --query-id
```

Query for top source IPs by request v

```
aws logs start-query \
```

```
--log-group-name \
```

```
--start-time $(date -u -v-1H +%s) \
```

```
--end-time $(date -u +%s) \
```

```
--query-string '
```

```
fields @timestamp, ip, httpMethod, resourcePath
```

```
| stats count() as requestCount by ip
```

```
| sort requestCount desc
```

```
| limit 20'
```

Query for requests missing authorizat

```
aws logs start-query \
```

```
--log-group-name \
```

```
--start-time $(date -u -v-1H +%s) \
```

```
--end-time $(date -u +%s) \
```

```
--query-string '
```

```
fields @timestamp, ip, httpMethod, resourcePath, status, caller
```

```
| filter status == 403 or caller == "-"
```

```
| sort @timestamp desc
```

```
| limit 50'
```

MCP batch opportunity: run multiple Logs Insights queries in parallel

1.5 Analyze WAF Logs

WAF logs capture full request details including matched rules.

Find WAF log group or S3 bucket

```
aws wafv2 get-logging-configuration \
```

```
--resource-arn
```

If WAF logs go to CloudWatch Logs – c

```
aws logs start-query \
```

```
--log-group-name \
```

```
--start-time $(date -u -v-1H +%s) \
```

```
--end-time $(date -u +%s) \
```

```
--query-string '
```

```
fields @timestamp, httpRequest.clientIp, httpRequest.uri,
```

```
httpRequest.httpMethod, action, terminatingRuleId
```

```
| filter action = "BLOCK"
```

```
| stats count() as blockCount by httpRequest.clientIp, terminatingRuleId
```

```
| sort blockCount desc
```

```
| limit 20'
```

List WAF rules on the associated web

```
aws wafv2 get-web-acl \
```

```
--name \
```

```
--scope REGIONAL \
```

```
--id \
```

```
--query 'WebACL.Rules[].[Name,Priority,Action]' \
```

```
--output table
```

1.6 Investigate CloudTrail for Management-Plane Changes

Search for API Gateway configuration

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=ResourceType,AttributeValue=AWS::ApiGateway::RestApi
```

```
--start-time \
```

```
--end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
```

```
--max-results 50
```

Search for specific high-risk events

```
for event in CreateDeployment UpdateStage DeleteStage CreateAuthorizer \
```

```
UpdateAuthorizer DeleteAuthorizer CreateApiKey DeleteApiKey; do
```

```
echo "=== $event ==="
```

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventName,AttributeValue=$event \
```

```
--start-time \
```



```
--max-results 10
```

```
done
```

MCP batch opportunity: look up all event types in parallel

Document:

- [] Any recent deployments pushed to the affected stage
- [] Any authorizer changes
- [] Any WAF disassociation events
- [] IAM principal that made any changes

1.7 Establish Timeline and Scope

Document the following before proceeding to containment:

Item	Value
------	-------

-----	-----
-------	-------

First suspicious request timestamp	
------------------------------------	--

Alert trigger timestamp	
-------------------------	--

Last API Gateway config change	
--------------------------------	--

Attacker source IP(s)	
-----------------------	--

| Targeted endpoint(s) / resource path(s) | |

| Total suspicious request volume | |

| Backend Lambda functions invoked | |

| Sensitive data potentially returned | |

| Attack type(s) identified | |

1.8 Document and Communicate

- [] Create/update incident ticket with: API ID, stage, attacker IP(s), timeline, attack type
- [] Identify API/application owners from CMDB or API Gateway tags

- ☐ Open war room bridge
- ☐ Notify: Security team, Application team, Legal (if PII/sensitive data involved)
- ☐ Preserve log evidence: export access logs, WAF logs, CloudTrail events to S3

Part 2: Contain the Incident

Goal: Throttle all traffic to the API until the attack vector is identified and closed.

2.1 Preserve Log Evidence Before Any Changes

Before making any configuration changes, export current log evidence:

Export CloudWatch access logs to S3

```
aws logs create-export-task \
```

```
--log-group-name \
```

```
--from $(date -u -v-24H +%s000) \
```

```
--to $(date -u +%s000) \
```

```
--destination \
```

```
--destination-prefix "incident-/access-logs/"
```

Export WAF logs if in CloudWatch Logs

```
aws logs create-export-task \
```

```
--log-group-name \
```

```
--from $(date -u -v-24H +%s000) \
```

```
--to $(date -u +%s000) \
```

```
--destination \
```

```
--destination-prefix "incident-/waf-logs/"
```

2.2 Enable WAF Logging (if not already enabled)

Associate WAF logging to CloudWatch

```
aws wafv2 put-logging-configuration \
```

```
--logging-configuration '{
```

```
"ResourceArn": "",
```

```
"LogDestinationConfigs": ["arn:aws:logs::log-group:aws-waf-logs-"],
```

```
"RedactedFields": []
```

```
}'
```

2.3 Associate WAF with API Gateway Stage (if not already associated)

Create a new web ACL (if none exists)

```
aws wafv2 create-web-acl \
```

```
--name "emergency-api-protection" \
```

```
--scope REGIONAL \
```

```
--default-action Allow={} \
```

```
--visibility-config SampledRequestsEnabled=true,CloudWatchMetricsEnabled=true,MetricName=
```



```
--rules '[]'
```

Associate web ACL with API Gateway stage

```
aws wafv2 associate-web-acl \
```

```
--web-acl-arn \
```

```
--resource-arn arn:aws:apigateway:::/restapis//stages/
```

■■ **Warning:** Associating a new WAF web ACL with a default `Allow` action will not immediately block traffic — add rules in the following steps. Confirm the association before proceeding.

2.4 Block Attacker IP(s) via WAF IP Set

Create an IP set with attacker IPs

```
aws wafv2 create-ip-set \
```

```
--name "blocked-attacker-ips" \
```

```
--scope REGIONAL \
```

```
--ip-address-version IPV4 \
```

```
--addresses "/32" "/32"
```

Add a block rule to the web ACL using

```
aws wafv2 update-web-acl \
```

```
--name \
```

```
--scope REGIONAL \
```

```
--id \
```

```
--lock-token \
```

```
--default-action Allow={} \
```

```
--rules '[{
```

```
  "Name": "BlockAttackerIPs",
```

```
  "Priority": 0,
```

```
  "Statement": {
```

```
    "IPSetReferenceStatement": {
```

```
      "ARN": ""
```

```
    }
```

```
  },
```

```
  "Action": {"Block": {}},
```

```
"VisibilityConfig": {
```

```
  "SampledRequestsEnabled": true,
```

```
  "CloudWatchMetricsEnabled": true,
```

```
  "MetricName": "BlockAttackerIPs"
```

```
}
```

```
}]' \
```

```
--visibility-config SampledRequestsEnabled=true,CloudWatchMetricsEnabled=true,MetricName=BlockAttackerIPs
```

■■ **Warning:** Blocking by IP may affect legitimate users sharing a NAT gateway, corporate proxy, or CDN exit node. Verify the IP is not a shared egress point before blocking. If uncertain, use rate-based rules instead (Step 2.5).

2.5 Add WAF Rate-Based Rule

Add a rate-based rule to throttle any

```
aws wafv2 update-web-acl \
```

```
--name \
```

```
--scope REGIONAL \
```

```
--id \
```

```
--lock-token \
```

```
--default-action Allow={} \
```

```
--rules '[{
```

```
  "Name": "RateLimitRule",
```

```
  "Priority": 1,
```

```
  "Statement": {
```

```
"RateBasedStatement": {
```

```
  "Limit": 500,
```

```
  "AggregateKeyType": "IP"
```

```
}
```

```
},
```

```
"Action": {"Block": {}},
```

```
"VisibilityConfig": {
```

```
  "SampledRequestsEnabled": true,
```

```
  "CloudWatchMetricsEnabled": true,
```

```
  "MetricName": "RateLimitRule"
```

```
}
```

```
}]'
```

■■ **Warning:** The rate limit applies to ALL IPs, not just the attacker. Set the threshold above your expected legitimate peak request rate per IP to avoid blocking real users. Coordinate with the application team to confirm a safe threshold.

2.6 Reduce Stage-Level Throttling

Document current values before changing

```
aws apigateway get-stage \
```

```
--rest-api-id \
```

```
--stage-name \
```

```
--query '[defaultRouteSettings.throttlingBurstLimit,defaultRouteSettings.throttlingRateLimit]
```

Reduce burst and rate limits on the s

```
aws apigateway update-stage \
```

```
--rest-api-id \
```

```
--stage-name \
```

```
--patch-operations \
```

```
op=replace,path=/defaultRouteSettings/throttlingBurstLimit,value=100 \
```

```
op=replace,path=/defaultRouteSettings/throttlingRateLimit,value=50
```

■■ **Warning:** Stage-level throttling applies to all callers including legitimate users. Coordinate with the application team before reducing limits in production. Document the original values so they can be restored in Part 4.

2.7 Disable Compromised API Key (if API key auth is the attack vector)

List API keys to find the compromised

```
aws apigateway get-api-keys \
```

```
--include-values \
```

```
--query 'items[?name==].[id,name,enabled]'
```

Disable the API key

```
aws apigateway update-api-key \
```

```
--api-key \
```

```
--patch-operations op=replace,path=/enabled,value=false
```

■■ **Warning:** Disabling an API key immediately breaks all integrations using that key. Confirm with the application team which services or clients use this key before disabling.

2.8 Deploy Usage Plan with Hard Quotas (if rate-based rules are insufficient)

Create a restrictive usage plan

```
aws apigateway create-usage-plan \
```

```
--name "emergency-throttle" \
```

```
--api-stages apiId=,stage= \
```

```
--throttle burstLimit=50,rateLimit=25 \
```

```
--quota limit=1000,period=DAY
```

■■ **Warning:** Usage plan quotas apply to API keys associated with the plan. If legitimate high-volume clients are associated, they will be throttled. Confirm scope with the application team.

2.9 Take Stage Offline — Last Resort

Only use this if all other containment options have failed and the attack is causing active data breach or service degradation.

Set stage throttling to zero (effect:)

```
aws apigateway update-stage \
```

```
--rest-api-id \
```

```
--stage-name \
```

```
--patch-operations \
```

```
op=replace,path=/defaultRouteSettings/throttlingBurstLimit,value=0 \
```

```
op=replace,path=/defaultRouteSettings/throttlingRateLimit,value=0
```

■■ **Warning:** Setting throttling to 0 causes a complete service outage for ALL users of this API stage. This is a last resort. Obtain explicit approval from the application team and incident commander before executing. Document the exact time this action was taken.

2.10 Revoke IAM Credentials (if IAM auth is the attack vector)

If the API is using IAM authorization and an IAM principal's credentials were used to make unauthorized API calls, follow `skill-irp-credential-compromise` in parallel for credential revocation steps.

2.11 Verify Containment

Monitor for 15+ minutes after containment actions:

Check WAF blocked request metrics

```
aws cloudwatch get-metric-statistics \
```

```
--namespace AWS/WAFV2 \
```

```
--metric-name BlockedRequests \
```

```
--dimensions Name=WebACL,Value= Name=Region,Value= Name=Rule,Value=BlockAttackerIPs \
```

```
--start-time $(date -u -v -15M +%Y-%m-%dT%H:%M:%SZ) \
```

```
--end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
```

```
--period 60 \
```

```
--statistics Sum
```

Check API Gateway 4XX error rate return

```
aws cloudwatch get-metric-statistics \
```

```
--namespace AWS/ApiGateway \
```

```
--metric-name 4XXError \
```

```
--dimensions Name=ApiName,Value= Name=Stage,Value= \
```

```
--start-time $(date -u -v-15M +%Y-%m-%dT%H:%M:%SZ) \
```

```
--end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
```

```
--period 60 \
```

```
--statistics Sum
```

Check Lambda invocation count return

```
aws cloudwatch get-metric-statistics \
```

```
--namespace AWS/Lambda \
```

```
--metric-name Invocations \
```

```
--dimensions Name=FunctionName,Value= \
```

```
--start-time $(date -u -v-15M +%Y-%m-%dT%H:%M:%SZ) \
```

```
--end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
```

```
--period 60 \
```

```
--statistics Sum
```

MCP batch opportunity: run all CloudWatch metric queries in parallel

Containment is verified when ALL of the following are true:

- [] WAF blocked request count for attacker IP(s) has stabilized (no new requests getting through)

- ☐ CloudWatch 4XXError and Count metrics have returned to baseline
- ☐ Access logs show no new requests from attacker source IP(s) for 15+ minutes
- ☐ WAF logs confirm rate-based rule is actively blocking
- ☐ Lambda invocation count has returned to expected baseline
- ☐ No new suspicious CloudTrail management-plane events in the last 15 minutes

If new attack traffic appears from different IPs, the attacker may be rotating IPs — consider enabling AWS WAF Bot Control or Fraud Control managed rule groups.

Part 3: Eradicate the Incident

Goal: Remove all attacker-created resources, close the exploited vulnerability, and harden the API configuration to prevent recurrence.

3.1 Audit and Remove Unauthorized API Keys

List all API keys – look for keys cre

```
aws apigateway get-api-keys \
```

```
--include-values \
```

```
--query 'items[][id,name,enabled,createdDate,lastUpdatedDate]' \
```

```
--output table
```

For each suspicious API key, check w

```
aws apigateway get-usage-plan-keys \
```

```
--usage-plan-id \
```

```
--query 'items[].[id,name,type]' \
```

```
--output table
```

Delete unauthorized API key

```
aws apigateway delete-api-key \
```

```
--api-key
```

After removing unauthorized keys, rotate all remaining legitimate API keys:

Step 1: Create replacement key

```
aws apigateway create-api-key \
```

```
--name "-rotated" \
```

```
--enabled \
```

```
--stage-keys restApiId=,stageName=
```

Step 2: Associate new key with usage

```
aws apigateway create-usage-plan-key \
```

```
--usage-plan-id \
```

```
--key-id \
```

```
--key-type API_KEY
```

Step 3: Coordinate with application t

Step 4: Disable and delete old key at

```
aws apigateway update-api-key \
```

```
--api-key \
```

```
--patch-operations op=replace,path=/enabled,value=false
```

```
aws apigateway delete-api-key \
```

```
--api-key
```

■■ **Warning:** Deleting old API keys before clients are updated will break legitimate integrations. Coordinate the cutover with the application team.

3.2 Audit and Remove Unauthorized IAM Entities

List IAM users created after the comp

```
aws iam list-users \
```

```
--query 'Users[?CreateDate>=].[UserName,CreateDate,UserId]' \
```

```
--output table
```

List IAM roles created after the comp

```
aws iam list-roles \
```

```
--query 'Roles[?CreateDate>=].[RoleName,CreateDate,RoleId]' \
```

```
--output table
```

For each suspicious IAM entity, check

```
aws iam list-attached-user-policies --user-name
```

```
aws iam list-user-policies --user-name
```

```
aws iam list-attached-role-policies --role-name
```

```
aws iam list-role-policies --role-name
```

MCP batch opportunity: check policies for all suspicious entities in parallel

Remove unauthorized IAM user (after v

```
aws iam delete-access-key --user-name --access-key-id
```

```
aws iam detach-user-policy --user-name --policy-arn
```

```
aws iam delete-user-policy --user-name --policy-name
```

```
aws iam delete-user --user-name
```

Remove unauthorized IAM role (after v

```
aws iam detach-role-policy --role-name --policy-arn
```



```
aws iam delete-role-policy --role-name --policy-name
```

```
aws iam delete-role --role-name
```

3.3 Inspect and Restore API Definition

Check whether the API definition was tampered with via `ImportRestApi` or `PutRestApi`:

Export the current API definition

```
aws apigateway get-export \
```

```
--rest-api-id \
```

```
--stage-name \
```

```
--export-type oas30 \
```

```
--accepts application/json \
```

```
--parameters extensions=apigateway \
```

```
output.json
```

List all deployments – compare timest

```
aws apigateway get-deployments \
```

```
--rest-api-id \
```

```
--query 'items[].[id,description,createdDate]' \
```

```
--output table
```

Roll back to a known-good deployment

```
aws apigateway update-stage \
```

```
--rest-api-id \
```

```
--stage-name \
```

```
--patch-operations op=replace,path=/deploymentId,value=
```

Compare the exported API definition against your source control (e.g., OpenAPI spec in Git) to identify any unauthorized endpoints, integration changes, or auth bypasses introduced by the attacker.

3.4 Inspect and Restore Authorizer Configuration

List all authorizers

```
aws apigateway get-authorizers \
```

```
--rest-api-id \
```

```
--query 'items[][id,name,type,authorizerUri,identitySource,authorizerResultTtlInSeconds]
```

```
--output table
```

Check each method to confirm authoriz

```
aws apigateway get-resources \
```

```
--rest-api-id \
```

```
--query 'items[].[id,path,resourceMethods]' \
```

```
--output table
```

For a specific resource and method, g

```
aws apigateway get-method \
```

```
--rest-api-id \
```

```
--resource-id \
```

```
--http-method \
```

```
--query '[authorizationType,authorizerId,apiKeyRequired]'
```

If the authorizer was tampered with or methods were set to NONE auth:

Restore authorizer on a specific method

```
aws apigateway update-method \
```

```
--rest-api-id \
```

```
--resource-id \
```

```
--http-method \
```

```
--patch-operations \
```

```
op=replace,path=/authorizationType,value=CUSTOM \
```

```
op=replace,path=/authorizerId,value=
```

Re-deploy after restoring auth config

```
aws apigateway create-deployment \
```

```
--rest-api-id \
```

```
--stage-name \
```

```
--description "Post-incident auth restoration"
```

■■ **Warning:** Changing authorizer configuration and re-deploying affects all callers of the API. Test in a non-production stage first where possible, and coordinate with the application team before deploying to production.

3.5 Check for Lambda Authorizer Backdoor

If the API uses a Lambda authorizer, inspect the function code for tampering:

Get the Lambda authorizer function name

```
aws apigateway get-authorizer \
```

```
--rest-api-id \
```

```
--authorizer-id \
```

```
--query 'authorizerUri'
```

List Lambda function versions – look

```
aws lambda list-versions-by-function \
```

```
--function-name \
```

```
--query 'Versions[][Version,LastModified,Description]' \
```

```
--output table
```


Download current function code for re

```
aws lambda get-function \
```

```
--function-name \
```

```
--query 'Code.Location'
```

Download the zip from the returned pr

If the Lambda authorizer code was modified to always return `Allow`, redeploy from your source control pipeline immediately.

3.6 Tighten IAM Policies for API Gateway Principals

Review IAM policies for all principals with `apigateway: permissions` and enforce least privilege:

Find all IAM entities with API Gateway

```
aws iam get-account-authorization-details \
```

```
--filter User Role \
```

```
--query 'UserDetailList[?AttachedManagedPolicies[?contains(PolicyName, APIGateway)]].U
```

```
--output text
```

Replace broad `apigateway: grants` with specific actions required per principal. Coordinate IAM policy updates with the application and security teams.

3.7 Iterate Until Clean

After completing Steps 3.1–3.6:

- ☐ Re-run CloudTrail lookup for API Gateway management events — no new unauthorized changes
- ☐ Re-run access log queries — no new requests from attacker IP(s) or patterns
- ☐ Confirm all IAM entities created by attacker have been removed
- ☐ Confirm API definition matches known-good version in source control
- ☐ Confirm all methods have correct authorizer configuration

- [] Confirm Lambda authorizer code matches known-good version in source control

Part 4: Recover from the Incident

Goal: Restore the API to full production operation with verified secure configuration and confirmed legitimate traffic only.

4.1 Remove Emergency WAF IP Block Rules

Once the attack has fully stopped (verified in Part 2), remove temporary IP block rules to avoid permanently blocking IPs that may be reused by legitimate users:

Get current web ACL lock token

```
aws wafv2 get-web-acl \
```

```
--name \
```

```
--scope REGIONAL \
```

```
--id \
```

```
--query '[WebACL.Rules, LockToken]'
```

Update web ACL to remove the emergency

(retain the rate-based rule as a perm

```
aws wafv2 update-web-acl \
```

```
--name \
```

```
--scope REGIONAL \
```

```
--id \
```

```
--lock-token \
```

```
--default-action Allow={} \
```

```
--rules '[]' \
```

```
--visibility-config SampledRequestsEnabled=true,CloudWatchMetricsEnabled=true,MetricName=
```

Delete the IP set if no longer needed

```
aws wafv2 delete-ip-set \
```

```
--name blocked-attacker-ips \
```

```
--scope REGIONAL \
```

```
--id \
```

```
--lock-token
```

■■ **Warning:** Only remove IP block rules after confirming the attack has fully stopped and the attack vector has been closed in Part 3. Removing blocks prematurely may allow the

attacker to resume.

4.2 Re-enable API Stage if Taken Offline

If the stage was taken offline during containment (throttling set to 0), restore to pre-incident throttling values:

Restore stage throttling to pre-incident

(use values documented during containment)


```
aws apigateway update-stage \
```

```
--rest-api-id \
```

```
--stage-name \
```

```
--patch-operations \
```

```
op=replace,path=/defaultRouteSettings/throttlingBurstLimit,value= \
```

```
op=replace,path=/defaultRouteSettings/throttlingRateLimit,value=
```

■■ **Warning:** Restore throttling values to pre-incident levels only after the attack vector is confirmed closed. Coordinate with the application team to confirm the original values before restoring.

4.3 Issue New API Keys to Legitimate Clients

After rotating API keys in Part 3, coordinate with the application team to distribute new keys to all legitimate clients:

Confirm new API key is active and ass

```
aws apigateway get-api-key \
```

```
--api-key \
```

```
--include-value \
```

```
--query '[id,name,enabled,stageKeys]'
```

Confirm usage plan association

```
aws apigateway get-usage-plan-keys \
```

```
--usage-plan-id \
```

```
--query 'items[].[id,name,type]' \
```

```
--output table
```

- [] *Notify all legitimate API clients of new key values via secure channel*
- [] *Confirm each client has updated their configuration and is successfully authenticating*
- [] *Confirm old compromised keys are disabled and deleted (from Part 3.1)*

4.4 Restore Backend Data if Modified

If the attacker modified or deleted data in backend services during the incident:

DynamoDB:

Check Point-in-Time Recovery (PITR) s

```
aws dynamodb describe-continuous-backups \
```

```
--table-name \
```

```
--query 'ContinuousBackupsDescription.PointInTimeRecoveryDescription'
```

Restore table to point in time before

```
aws dynamodb restore-table-to-point-in-time \
```

```
--source-table-name \
```

```
--target-table-name -restored \
```

```
--restore-date-time
```

S3:

Check S3 versioning status

```
aws s3api get-bucket-versioning --bucket
```

List object versions to find pre-att

```
aws s3api list-object-versions \
```

```
--bucket \
```

```
--prefix
```

Restore previous version by copying

```
aws s3api copy-object \
```

```
--bucket \
```

```
--copy-source "/?versionId=" \
```

```
--key
```

■■ **Warning:** Restoring data overwrites current state. Confirm which records were affected using access logs and Lambda logs from the investigation before restoring. Consult the application team to validate data integrity before and after restoration.

4.5 Verify Recovery

Monitor for 30+ minutes after restoring normal operations:

Confirm API invocation count has returned to normal

```
aws cloudwatch get-metric-statistics \
```

```
--namespace AWS/ApiGateway \
```

```
--metric-name Count \
```

```
--dimensions Name=ApiName,Value= Name=Stage,Value= \
```

```
--start-time $(date -u -v-30M +%Y-%m-%dT%H:%M:%SZ) \
```

```
--end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
```

```
--period 60 \
```

```
--statistics Sum
```

Confirm Lambda invocation count and c

```
aws cloudwatch get-metric-statistics \
```

```
--namespace AWS/Lambda \
```

```
--metric-name Invocations \
```

```
--dimensions Name=FunctionName,Value= \
```

```
--start-time $(date -u -v-30M +%Y-%m-%dT%H:%M:%SZ) \
```

```
--end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
```

```
--period 60 \
```

```
--statistics Sum
```


Confirm no new unauthorized CloudTrail

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=ResourceType,AttributeValue=AWS::ApiGateway::RestApi
```

```
--start-time $(date -u -v -30M +%Y-%m-%dT%H:%M:%SZ) \
```

```
--max-results 20
```

MCP batch opportunity: run all CloudWatch metric queries in parallel

Recovery is verified when ALL of the following are true:

- [] API Gateway Count metric is at expected baseline for time of day
- [] Lambda invocation count and duration are at expected baseline
- [] Legitimate clients confirm API is functioning correctly

- ☐ *No WAF blocks on legitimate client IPs*
- ☐ *CloudTrail shows no further unauthorized API Gateway management-plane changes*
- ☐ *Backend data spot-check confirms records are correct post-restoration*
- ☐ *All monitoring and alerting is re-armed and functioning*

Part 5: Post-Incident Activity

5.1 Document Lessons Learned

Create a post-incident report covering:

- **Timeline:** When did each phase of the attack and response occur?
- **Root cause:** How was the API breached? (leaked key, missing authorizer, WAF gap, injection vulnerability)
- **Impact:** What data was accessed or exfiltrated? Which clients/users were affected?
- **Response effectiveness:** What worked well? What slowed the response?
- **Recommendations:** Process, tooling, and configuration improvements

5.2 Retrospective Questions

- *Could this attack have been detected earlier? What monitoring gap allowed it to go unnoticed?*
- *Was WAF enabled and properly configured before the incident? What rules were missing?*
- *Were authorizers enforced on all methods? Were there any unprotected endpoints?*
- *How quickly were logs available for investigation? Was access logging enabled pre-incident?*
- *Could the blast radius have been reduced with tighter IAM policies or usage plan quotas?*

5.3 Update Defenses

Based on findings:

- *[] Enable AWS WAF on all API Gateway stages with Core Rule Set (CRS) and Known Bad Inputs managed rule groups*

- *[] Enable access logging on all API Gateway stages with structured JSON format*
- *[] Enable execution logging on all API Gateway stages (ERROR level minimum)*
- *[] Enforce authorizer (IAM, Cognito, or Lambda) on all methods — remove `NONE` where not explicitly required*
- *[] Enable AWS Config rules: `api-gw-associated-with-waf`, `api-gw-execution-logging-enabled`, `api-gw-ssl-enabled`*
- *[] Enable Security Hub API Gateway controls (`APIGateway.1`, `APIGateway.2`, `APIGateway.9`)*
- *[] Implement API key rotation schedule*
- *[] Review and tighten IAM policies for all principals with `apigateway: permissions`*
- *[] Consider enabling AWS WAF Bot Control for APIs exposed to the public internet*
- *[] Update this playbook with lessons learned*

5.4 Regulatory Notifications

If required by your jurisdiction or compliance framework:

- ☐ Notify relevant authorities within required timeframe (e.g., GDPR 72-hour rule if EU personal data was exposed)
- ☐ Notify affected customers or users if PII was accessed via the API
- ☐ Document notification actions for compliance records (PCI-DSS, HIPAA, SOC 2)

References

- [AWS Security Incident Response Guide](#)
- [Amazon API Gateway Developer Guide](#)
- [AWS WAF Developer Guide](#)
- [API Gateway Access Logging](#)
- [API Gateway Security Best Practices](#)
- [OWASP API Security Top 10](#)

■ `ai-playbooks/skills/skill-irp-builder.md`

description: Interactive builder for creating new AWS incident response skill files from scratch. Use this skill when you want to build an IR playbook from scratch, design a custom incident response process, create a new IR workflow with no existing source playbook, build a new security runbook, design IR procedures for a specific AWS service, or create a

custom response plan for an incident type not covered by existing playbooks.

Skill: IR Playbook Builder (From Scratch)

Purpose

This skill conducts an interactive step-by-step interview to build a complete IR skill file from scratch when no existing source playbook exists. It walks the user through each NIST 800-61 phase, synthesizes a complete skill file matching the repo's established format, and registers it in CLAUDE.md routing.

Differentiation from the factory skill: The factory skill (`skill-irp-playbook-factory`) translates an existing human-centric playbook from `playbooks/` into a skill file. This builder skill creates a skill file from scratch through guided interview when no source playbook exists.

Interview Process Overview

1. **Step 0:** Pre-interview setup — greet, check for existing skills, derive filename

2. **Step 1:** Incident type and scenario definition

3. **Step 2:** Detection and alert sources

4. **Step 3:** Evidence gathering and investigation

5. **Step 4:** Containment strategy

6. **Step 5:** Eradication

7. **Step 6:** Recovery

8. **Step 7:** Post-incident activities

9. **Step 8:** References

10. **Step 9:** Derive Quick Reference table

Quick mode option: At the start, offer the user "quick mode" — if they provide the incident type and primary AWS service, generate reasonable defaults for Steps 2-6 based on AWS documentation research, then present the full draft for review instead of asking every sub-question individually. This is ideal for experienced IR practitioners who prefer to edit a draft rather than answer each question.

AWS Documentation Research Strategy

- **Primary:** Use AWS docs MCP tools (`search_documentation`, `read_documentation`, `read_sections`) to look up relevant GuardDuty finding types, AWS CLI commands, service-specific security guidance, and best practices
- **Fallback:** Use AWS CLI commands (e.g., `aws guardduty list-findings`, `aws configservice describe-compliance-by-config-rule`, `aws securityhub get-findings`) when MCP is unavailable
- **Never block on MCP availability** — always have a CLI-based alternative path ready
- At each interview step, offer research: "Would you like me to search AWS documentation for relevant [detection sources / CLI commands / best practices]?"

Step 0: Pre-Interview Setup

0.1 Greet and Orient the User

Present a brief overview:

- Explain the interview process (10 steps covering all NIST 800-61 phases)
- Explain the expected output (a complete skill file + CLAUDE.md routing update)
- Offer "quick mode" vs "detailed mode" choice
- Set expectation: each step will produce draft content for review before moving on

0.2 Check for Existing Skills

List all existing IR skills:

```
ls .claude/skills/skill-irp-*.md
```

Present the list to the user. If a skill with a similar name or incident type already exists, warn:

- "A skill for already exists at . Do you want to create a new one, or would you prefer to update the existing skill?"

0.3 Priming Question

Ask: "What is the name or short description of the incident type you want to build a skill for?"

From the answer, derive:

- A candidate filename: skill-irp-.md (lowercase, hyphenated)
- Present the filename for confirmation

Step 1: Incident Type and Scenario Definition

1.1 Questions to Ask

- What incident type or scenario does this skill address?
- What AWS services are primarily involved?
- Describe a typical trigger — what does detection look like when this incident occurs?

1.2 Content to Build

From the answers, draft:

- The YAML description field (keyword-rich, covering trigger keywords an operator would use)
- The `## Incident Type` paragraph (one paragraph describing the scenario)
- Confirm the candidate filename

1.3 Research Offer

Offer to search AWS documentation for:

- Relevant GuardDuty finding types for this scenario
- Service-specific security documentation
- Known attack patterns

MCP primary:

```
search_documentation: " security"
```

CLI fallback:

```
aws guardduty list-findings --detector-id \
```

```
--finding-criteria '{"Criterion":{"type":{"Eq":[""]}}}'
```

1.4 User Confirmation Gate

Present the drafted content (description + Incident Type paragraph + filename). Wait for user approval before proceeding.

Step 2: Detection and Alert Sources

2.1 Questions to Ask

- How is this incident type typically detected? (GuardDuty, CloudWatch, Security Hub, billing alerts, user report, external notification)
- Are there specific GuardDuty finding types, Config rules, or Security Hub controls relevant to this scenario?
- Are there secondary signals that could indicate this incident?

2.2 Content to Build

Draft ### 1.1 Identify the Alert Source with a bullet list of common alert sources for this incident type.

2.3 Research Offer

Offer to look up:

- GuardDuty finding types related to the incident
- AWS Config managed rules for the relevant services
- Security Hub controls

MCP primary:

```
search_documentation: "GuardDuty finding types "
```

```
search_documentation: "AWS Config rules "
```

CLI fallback:

```
aws configservice describe-config-rules \
```

```
--query 'ConfigRules[?contains(ConfigRuleName, )].ConfigRuleName'
```

2.4 User Confirmation Gate

Present the drafted alert source section. Wait for approval.

Step 3: Evidence Gathering and Investigation

3.1 Questions to Ask (one at a time)

- What information should be gathered immediately? (scope, affected principal, timeline)
- What AWS APIs or CLI commands are needed for investigation?
- Are there specific CloudTrail event names that serve as indicators?
- What key timestamps or events need to be established?

3.2 Content to Build

From the answers, draft:

- `### 1.2` through `### 1.x` investigation subsections (numbered sequentially)
- CLI commands with params in ``bash`` blocks
- `# MCP batch opportunity:` `comments where parallel execution applies`
- `### 1.x Document and Communicate` `checklist` using `- []` checkboxes

3.3 Research Offer

Offer to search for:

- Relevant AWS CLI commands and API references for the involved services
- CloudTrail event names for key operations
- Investigation best practices

MCP primary:

search_documentation: " CLI reference "

search_documentation: "CloudTrail API events"

3.4 User Confirmation Gate

Present all drafted Part 1 subsections. Wait for approval.

Step 4: Containment Strategy

4.1 Questions to Ask

- What is the containment goal? (one sentence)
- What immediate actions stop the bleeding?
- Are there multiple containment options depending on circumstances?
- Which actions affect legitimate users or production workloads? (these need warning callouts)

4.2 Content to Build

Draft:

- `## Part 2: Contain the Incident with Goal: statement`
- `Containment subsections with CLI commands in `bash` blocks`
- `Warning callouts (using the pattern: Warning: ) for production-impacting actions`
- `### 2.x Verify Containment subsection with monitoring commands`

4.3 Research Offer

Offer to search for:

- Service-specific containment strategies
- Security group, NACL, or IAM policy patterns for isolation
- AWS best practices for incident containment

4.4 User Confirmation Gate

Present all drafted Part 2 content. Wait for approval.

Step 5: Eradication

5.1 Questions to Ask

- What is the eradication goal? (one sentence)
- What attacker-created resources or artifacts need to be removed?
- What persistence mechanisms should be checked?
- What CLI commands find and remove these artifacts?

5.2 Content to Build

Draft:

- ## Part 3: Eradicate the Incident with **Goal:** statement

- Audit subsections (identify malicious artifacts)

- Removal subsections with CLI commands

- # MCP batch opportunity: comments where applicable

5.3 Research Offer

Offer to search for:

- Common persistence mechanisms for the attack type
- Service-specific cleanup procedures

5.4 User Confirmation Gate

Present all drafted Part 3 content. Wait for approval.

Step 6: Recovery

6.1 Questions to Ask

- What is the recovery goal? (one sentence)
- What resources need to be restored? Are backups, snapshots, or versioning available?
- What does "back to normal" look like for this incident type?

6.2 Content to Build

Draft:

- ## Part 4: Recover from the Incident with **Goal:** statement

- Restore steps with CLI commands
- ### 4.x Verify Recovery checklist using - [] checkboxes

6.3 Research Offer

Offer to search for:

- AWS Backup integration for the relevant services
- Service-specific restore procedures
- Recovery verification best practices

6.4 User Confirmation Gate

Present all drafted Part 4 content. Wait for approval.

Step 7: Post-Incident Activities

7.1 Offer Standard Template

Present the standard Part 5 structure shared by all existing skills:

Part 5: Post-Incident Activity

5.1 Document Lessons Learned

- Timeline, root cause, impact, response effectiveness, recommendations

5.2 Retrospective Questions

- Detection improvement, response acceleration, blast radius reduction

5.3 Update Defenses

- Checklist of hardening actions

5.4 Regulatory Notifications

- Jurisdiction-specific notification requirements

Ask: "Do you want to accept this standard template, or customize it for this incident type?"

7.2 Incident-Specific Customizations

If the user wants customization, ask:

- Are there forensic analysis steps specific to this incident type?
- Are there regulatory notification requirements beyond the standard?

- What specific defense updates should be recommended?

Step 8: References

8.1 Always Include

Every skill file must include:

- [AWS Security Incident Response Guide](#)

8.2 Ask for Additional Links

Ask: "Are there specific AWS documentation pages or external references to include?"

8.3 Auto-Include MCP Sources

If AWS documentation was searched during the interview, automatically include relevant documentation URLs discovered during research.

Step 9: Derive Quick Reference Table

9.1 Build from Prior Steps

Synthesize a Quick Reference table from the content built in Steps 2-7:

Phase	Key Action	Verification
-------	------------	--------------

--	--	--

Evidence		
----------	--	--

Contain		
---------	--	--

Eradicate		
-----------	--	--

Recover		
---------	--	--

Post-Incident		
---------------	--	--

9.2 Present for Review

Show the table. User corrects cells if needed.

Quality Checklist

Before saving, verify the generated skill file passes ALL checks:

- ☐ Front-matter description is keyword-rich and covers main trigger scenarios
- ☐ No inclusion field in front-matter
- ☐ No sibling skill cross-references in description
- ☐ ## Incident Type paragraph clearly describes the scenario
- ☐ ## Quick Reference table covers all 5 NIST phases
- ☐ Part 1: alert sources, investigation subsections, Document and Communicate checklist
- ☐ Part 2: **Goal:** statement, containment CLI commands, warning callouts, Verify Containment subsection
- ☐ Part 3: **Goal:** statement, eradication CLI commands

- [] Part 4: **Goal:** statement, recovery CLI commands, Verify Recovery checklist

- [] Part 5: lessons learned, retrospective questions, update defenses, regulatory notifications

- [] All CLI commands use params (not hardcoded values)

- [] **Warning:** callouts present for production-impacting actions

- [] # MCP batch opportunity: comments where parallel execution applies

- [] ## References section includes AWS Security IR Guide

- [] File named skill-irp.md

- [] No markers remain in the final output

If the agent had to assume content (user said "I don't know"): mark those sections with during drafting, then resolve ALL markers before final save. If a marker cannot be resolved, flag it to the user explicitly.

File Creation and Registration

Write Skill Files

After the quality checklist passes:

1. Save to `.claude/skills/skill-irp.md`
2. Mirror to `ai-playbooks/skills/skill-irp.md`

Update CLAUDE.md Routing

Edit ai-playbooks/skills/CLAUDE.md to register the new skill:

1. **Step 1 (Keyword Pattern Matching):** Add primary keywords that should trigger the new skill

2. **Step 2 (Context Analysis):** Add incident-specific indicators if warranted

3. **Default option:** Update only if the new skill should be considered as a fallback

Present the CLAUDE.md diff to the user for approval before saving.

Output Format

The generated skill file MUST exactly match the existing skill format:

description:

Playbook:

Incident Type

--

Quick Reference

Phase	Key Action	Verification	
-------	------------	--------------	--

-----	-----	-----	
-------	-------	-------	--

Evidence	...	...	
----------	-----	-----	--

Contain	...	...	
---------	-----	-----	--

Eradicate	...	...	
-----------	-----	-----	--

Recover	...	...	
---------	-----	-----	--

Post-Incident	...	...	
---------------	-----	-----	--

Part 1: Acquire, Preserve, Document Evidence

1.1 Identify the Alert Source

1.2

1.x Document and Communicate

Part 2: Contain the Incident

Goal:

2.x

2.x Verify Containment

Part 3: Eradicate the Incident

Goal:

3.x

Part 4: Recover from the Incident

Goal:

4.x

4.x Verify Recovery

Part 5: Post-Incident Activity

5.1 Document Lessons Learned

5.2 Retrospective Questions

5.3 Update Defenses

5.4 Regulatory Notifications

References

Conventions: params in CLI commands, **Warning:** callouts, -[] checklists, # MCP batch opportunity: comments, ``bash code blocks.

■ ai-playbooks/skills/skill-irp-credential-compromise.md

description: Incident response playbook for AWS credential leakage/compromise. Use this skill when responding to compromised credentials, leaked or exposed IAM access keys, unauthorized IAM user activity, GuardDuty findings related to credential exfiltration, or any scenario involving stolen AWS credentials or unauthorized API access.

Playbook: Credential Compromise

Incident Type

Credential Leakage/Compromise - When AWS credentials (IAM user access keys or STS temporary credentials) have been obtained by an unauthorized actor.

Quick Reference

| Phase | Key Action | Verification |

|-----|-----|-----|

| Evidence | Identify compromised credentials + timeline | Access Key ID, Principal ID, compromise timestamp documented |

| Contain | Disable credentials | CloudTrail shows no new activity for 30+ min |

| Eradicate | Remove attacker resources + persistence | No unauthorized IAM entities or resources remain |

| Recover | Restore modified/deleted resources | Applications functioning normally |

| Post-Incident | Document lessons learned | Report filed, playbook updated |

Part 1: Acquire, Preserve, Document Evidence

1.1 Identify the Alert Source

Common sources for credential compromise alerts:

- **GuardDuty findings** (e.g., `UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration`)
- **Security Hub alerts**
- **CloudWatch alarms** on IAM changes
- **AWS Config rules** showing IAM non-compliance
- **Billing alerts** (unexpected cost spike)
- **External notification** (security researcher, anonymous tip)

1.2 Identify Compromised Credentials

From GuardDuty finding:

- Navigate to GuardDuty console → Findings
- Locate the finding and expand "Resource" section
- Note the accessKeyDetails:
- `accessKeyId` - The compromised access key
- `principalId` - The IAM principal ID
- `userType` - IAMUser or AssumedRole
- `userName` - The IAM user or role name

CLI to get finding details:

List GuardDuty detectors

```
aws guardduty list-detectors
```

Get findings (replace detector-id)

```
aws guardduty list-findings --detector-id \
```

```
--finding-criteria '{"Criterion":{"type":{"Eq":["UnauthorizedAccess:IAMUser/InstanceC
```

Get specific finding details

```
aws guardduty get-findings --detector-id --finding-ids
```

1.3 Establish Timeline

Document the following timestamps:

- **First seen:** When was the malicious activity first detected? (GuardDuty: `service.eventFirstSeen`)
- **Likely compromise time:** When were credentials likely stolen? (may be earlier than first seen)
- **Current time:** When did you start responding?

All API actions after the compromise time should be considered malicious.

1.4 Document and Communicate

- ☐ Create/update incident ticket with: Access Key ID, Principal, Timeline
- ☐ Identify stakeholders from CMDB
- ☐ Open war room bridge
- ☐ Notify: Security team, Application owners, Legal (if required)

Part 2: Contain the Incident

Goal: Disable compromised credentials immediately.

2.1 For Long-Term IAM User Credentials

Disable the access key (does NOT delete)

```
aws iam update-access-key \
```

```
--user-name \
```

```
--access-key-id \
```

```
--status Inactive
```

Verify the key is disabled

```
aws iam list-access-keys --user-name
```

2.2 For STS Temporary Credentials (Assumed Role)

Option A: Revoke all sessions (immediate but affects all users of the role)

This adds an inline policy that denies

```
aws iam put-role-policy \
```

```
--role-name \
```

```
--policy-name AWSRevokeOlderSessions \
```

```
--policy-document '{
```

```
"Version": "2012-10-17",
```

```
"Statement": [{
```

```
"Effect": "Deny",
```

```
"Action": [""],
```

```
"Resource": [""],
```

```
"Condition": {
```

```
"DateLessThan": {"aws:TokenIssueTime": ""}
```

```
}
```

```
}]
```

```
}'
```

Option B: If attacker can obtain new credentials, modify trust policy:

Get current trust policy

```
aws iam get-role --role-name  --query 'Role.AssumeRolePolicyDocument'
```

Update trust policy to block the att

(specific changes depend on how crede

■■ **Warning:** Both options affect legitimate users of the role. Coordinate with application owners.

2.3 Verify Containment

Monitor CloudTrail for 30+ minutes:

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=AccessKeyId,AttributeValue= \
```

```
--start-time $(date -u -v-30M +%Y-%m-%dT%H:%M:%SZ) \
```

```
--max-results 10
```

If new events appear, containment failed - investigate how attacker is obtaining new credentials.

Part 3: Eradicate the Incident

Goal: Identify and remove everything the attacker created or modified.

3.1 Query CloudTrail for Attacker Actions

Using CloudTrail Lookup (limited to 90 days, recent events):

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=AccessKeyId,AttributeValue= \
```

```
--start-time \
```

```
--end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
```

```
--max-results 50
```

For comprehensive analysis, use Athena with CloudTrail logs in S3.

3.2 Correlate Attacker Activity by Source IP

After querying CloudTrail for a known compromised access key, extract the sourceIPAddress from attacker events. Then search for **other IAM users or roles that made API calls from the same IP** — the attacker may have compromised multiple credentials and be operating them all from the same host.

Extract unique source IPs used by the

(run against downloaded CloudTrail logs)

Look for the same IP appearing under

If you identify a suspicious IP, search

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=SourceIPAddress,AttributeValue= \
```

```
--start-time \
```

```
--max-results 50
```

Pivot to any additional compromised credentials found and repeat Parts 2 and 3 for each one.

3.3 Identify Suspicious Actions (original 3.2)

Look for these API calls in the results:

Identity/Persistence:

- `CreateUser, CreateRole, CreateGroup`
- `CreateAccessKey, CreateLoginProfile`
- `AttachUserPolicy, AttachRolePolicy, PutUserPolicy, PutRolePolicy`
- `UpdateAssumeRolePolicy`
- `CreateInstanceProfile, AddRoleToInstanceProfile`

Resource Creation:

- RunInstances, CreateFunction, CreateBucket
- CreateDBInstance, CreateCluster
- Any Create* calls

Data Access:

- GetObject, GetSecretValue, GetParameter
- Decrypt, GenerateDataKey

Covering Tracks:

- DeleteTrail, StopLogging, DeleteFlowLogs
- DeleteBucket (CloudTrail bucket)

3.3 Check for Persistence Mechanisms

List all IAM users (look for unfamiliar)

```
aws iam list-users
```

List all roles (look for unfamiliar)

```
aws iam list-roles --query 'Roles[?contains(RoleName, admin) || contains(RoleName, test)]'
```

Scan ALL IAM users for active access

```
for user in $(aws iam list-users --query 'Users[].UserName' --output text); do
```

```
keys=$(aws iam list-access-keys --user-name "$user" \
```

```
--query 'AccessKeyMetadata[?Status==Active].[AccessKeyId,Status,CreateDate]' --outp
```

```
if [ -n "$keys" ]; then echo "=== $user ==="; echo "$keys"; fi
```

```
done
```

Check for new access keys on existing

```
aws iam list-access-keys --user-name
```

List EC2 instances with IAM roles (po

```
aws ec2 describe-instances \
```

```
--query 'Reservations[].Instances[?IamInstanceProfile!=null].[InstanceId,IamInstanceP
```

```
--output table
```

3.4 Remove Attacker Resources

For each resource identified as created by the attacker:

Delete IAM user (after removing dependen

```
aws iam delete-access-key --user-name --access-key-id
```

```
aws iam delete-login-profile --user-name
```

```
aws iam detach-user-policy --user-name --policy-arn
```

```
aws iam delete-user --user-name
```

Delete IAM role (after removing dependencies)

```
aws iam detach-role-policy --role-name --policy-arn
```

```
aws iam delete-role-policy --role-name --policy-name
```

```
aws iam delete-role --role-name
```

Terminate EC2 instances

```
aws ec2 terminate-instances --instance-ids
```

Delete Lambda functions

```
aws lambda delete-function --function-name
```

3.5 Iterate

If you discovered new credentials (new IAM users, roles, access keys), repeat Part 3 for each set of credentials until no new persistence mechanisms are found.

Part 4: Recover from the Incident

4.1 Restore Modified Resources

If resource is replaceable (e.g., EC2 in ASG):

Terminate compromised instance, ASG will

```
aws ec2 terminate-instances --instance-ids
```

If resource needs restoration:

1. Check for backups (snapshots, S3 versioning, RDS snapshots)
2. Restore from backup

3. Or rebuild from CMDB configuration

4.2 Restore Deleted Resources

1. Check CMDB for resource configuration

2. Check for backups:

```
# List EBS snapshots
```

```
aws ec2 describe-snapshots --owner-ids self
```

```
# List RDS snapshots
```

```
aws rds describe-db-snapshots
```

```
# Check S3 versioning for deleted objects
```

```
aws s3api list-object-versions --bucket --prefix
```

3. Restore or recreate as needed

4.3 Verify Application Functionality

- ☐ Test affected applications
- ☐ Verify legitimate users can access resources
- ☐ Check monitoring dashboards for anomalies
- ☐ Update CMDB with any resource changes

Part 5: Post-Incident Activity

5.1 Forensic Analysis

If resources were isolated for forensics:

- Analyze to understand attack methods
- Determine initial compromise vector
- Identify any data exfiltration

5.2 Document Lessons Learned

Create post-incident report covering:

- **Timeline:** When did each phase occur?
- **Root cause:** How were credentials compromised?
- **Impact:** What was accessed/modified/deleted?
- **Response effectiveness:** What worked? What didn't?
- **Recommendations:** Process/tooling improvements

5.3 Update Defenses

Based on findings:

- ☐ Rotate any potentially exposed credentials
- ☐ Review IAM policies (least privilege)
- ☐ Enable/enhance monitoring (GuardDuty, CloudTrail, Config)
- ☐ Consider SCPs or permission boundaries

- ☐ Update this playbook with lessons learned

5.4 Regulatory Notifications

If required by your jurisdiction:

- ☐ Notify relevant authorities within required timeframe
- ☐ Document notification for compliance records

References

- [AWS IAM: Rotating Access Keys](#)
- [AWS IAM: Revoking Role Sessions](#)
- [Querying CloudTrail Logs with Athena](#)
- [AWS Security Incident Response Guide](#)

■ ai-playbooks/skills/skill-irp-data-access.md

description: Incident response playbook for unintended access to Amazon S3 buckets. Use this skill when responding to S3 data exposure, public S3 bucket misconfiguration, unauthorized S3 object access, overly permissive bucket policies, modified object ACLs, S3-related GuardDuty or Macie findings, or any scenario involving unintended access to S3 data.

Playbook: Unintended S3 Data Access

Incident Type

Unintended access to an Amazon S3 bucket — when bucket misconfiguration, overly permissive policies, modified object ACLs, or compromised credentials result in unauthorized data access.

Quick Reference

| Phase | Key Action | Verification |

|-----|-----|-----|

| Evidence | Identify affected bucket(s), access vector, timeline | Bucket name, access method (credential vs public), data classification documented |

| Contain | Disable credentials / restrict bucket policy / enable Block Public Access / identify compromised users and credentials | CloudTrail shows no new unauthorized access for 30+ min |

| Eradicate | Harden S3 config, rotate credentials, mitigate instance-level vectors, isolate compromised users | Least-privilege policies in place, no public access, encryption enabled |

| Recover | Restore modified/deleted objects from versioning or backups | Data integrity confirmed, applications functioning normally |

| Post-Incident | Document lessons learned | Report filed, playbook updated, risk documents revised |

Part 1: Acquire, Preserve, Document Evidence

1.1 Identify the Alert Source

Common sources for S3 data access alerts:

- **AWS Config rules** (e.g., `s3-bucket-public-read-prohibited`, `s3-bucket-public-write-prohibited`)
- **GuardDuty findings** (e.g., S3-related findings)
- **Security Hub alerts**
- **CloudWatch/EventBridge alarms** on S3 or IAM changes
- **Amazon Macie** sensitive data discovery alerts
- **Billing anomalies** (unexpected data transfer costs)
- **External notification** (threat actor, security researcher, news article, anonymous tip)

1.2 Determine the Access Vector

The incident may stem from misconfigured buckets, compromised credentials, SSRF via misconfigured EC2 settings, or any combination of these.

Check for overly permissive S3 configuration:

Use IAM Access Analyzer to find public

```
aws accessanalyzer list-findings --analyzer-name \
```

```
--filter '{"resourceType":{"eq":["AWS::S3::Bucket"]}}'
```

Check a specific bucket's public access

```
aws s3api get-public-access-block --bucket
```

Get bucket policy

```
aws s3api get-bucket-policy --bucket
```

Get bucket ACL

```
aws s3api get-bucket-acl --bucket
```

Check for recent configuration changes via CloudTrail:

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventSource,AttributeValue=s3.amazonaws.com \
```

```
--start-time \
```

```
--max-results 50
```

Look for these management events: `PutBucketPolicy`, `PutBucketAcl`, `PutPublicAccessBlock`, `DeletePublicAccessBlock`, `PutObjectAcl`.

Look for bulk `GetObject` as an active exfiltration signal:

A high volume of `GetObject` calls (50+ within seconds) from a single access key against a single bucket is a strong indicator of in-progress or recent data exfiltration. When reviewing CloudTrail logs, filter for `GetObject` events and check the event density — an attacker scripting an `aws s3 sync` or similar command will produce a burst pattern that stands out from normal application access. Also look for `ListObjects/ListBucket` calls immediately preceding the `GetObject` burst, which indicate the attacker was enumerating before downloading.

Check for IAM role trust policy changes:

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventName,AttributeValue=UpdateAssumeRolePolicy \
```

```
--start-time \
```

```
--max-results 20
```

If the credentials belong to a role attached to an EC2 instance:

Describe EC2 instance

```
aws ec2 describe-instances --instance-id
```

1.3 Identify the Affected Bucket(s)

If you know the data but not the bucket:

List buckets and search for the data

```
aws s3 ls
```

```
aws s3 ls s3:/// --recursive
```

Query CloudTrail Data Events for a sp

(requires Athena with CloudTrail logs

If you know the credentials but not the bucket:

Search CloudTrail Data Events filter

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=AccessKeyId,AttributeValue= \
```

```
--start-time \
```

```
--max-results 50
```

To find publicly accessible buckets across the account:

Check all buckets for public access

```
for bucket in $(aws s3api list-buckets --query 'Buckets[].Name' --output text); do
```

```
    echo "=== $bucket ==="
```

```
    aws s3api get-public-access-block --bucket "$bucket" 2>/dev/null || echo "No Block Pub
```

```
done
```

1.4 Determine Data Classification

- Check CMDB or data classification documents for the affected bucket/data
- Classification level may escalate the response path (e.g., PII, financial data, health records)

1.5 Document and Communicate

- ☐ Create/update incident ticket with: Bucket name(s), access vector, timeline, data classification
- ☐ Identify stakeholders from CMDB / application risk register
- ☐ Open war room bridge
- ☐ Notify: Security team, Application owners, Legal (if regulated data involved)

- [] For external communications: Inform legal counsel, PR, and relevant regulatory contacts as needed

Part 2: Contain the Incident

Goal: Stop unauthorized access immediately. Address credentials first, then bucket policies, then Block Public Access.

2.1 If Compromised Credentials Are Involved

For long-term IAM user credentials:

```
aws iam update-access-key \
```

```
--user-name \
```

```
--access-key-id \
```

```
--status Inactive
```

```
aws iam list-access-keys --user-name
```

For STS temporary credentials (assumed role):

Revoke all current sessions

```
aws iam put-role-policy \
```

```
--role-name \
```

```
--policy-name AWSRevokeOlderSessions \
```

```
--policy-document '{
```

```
"Version": "2012-10-17",
```

```
"Statement": [{
```

```
"Effect": "Deny",
```

```
"Action": [""],
```

```
"Resource": [""],
```

```
"Condition": {
```

```
"DateLessThan": {"aws:TokenIssueTime": ""}
```

```
}
```

```
}]
```

```
}'
```

If role is attached to an EC2 instance, also consider:

- SSH key compromise (attacker accessing instance directly)
- SSRF attacks using instance role credentials (mitigate with IMDSv2)

■■ Revoking role sessions affects all legitimate users/applications using that role.

2.2 If Bucket Policy Is Too Permissive

Get current bucket policy

```
aws s3api get-bucket-policy --bucket  --output text > bucket-policy-backup.json
```

Replace with a restrictive policy (c

```
aws s3api put-bucket-policy --bucket  --policy '{
```

```
  "Version": "2012-10-17",
```

```
  "Statement": [{
```

```
    "Effect": "Deny",
```

```
    "Principal": "",
```



```
"Action": "s3:",
```

```
"Resource": [
```

```
"arn:aws:s3:::",
```

```
"arn:aws:s3:::/"
```

```
],
```

```
"Condition": {
```

```
"StringNotEquals": {"aws:PrincipalAccount": ""}
```

```
}
```

```
}]
```

```
}'
```

Priority order:

1. Restrict control plane access first (PutBucketPolicy, PutBucketAcL, PutPublicAccessBlock)

2. Then restrict data plane access (*GetObject*, *PutObject*, etc.)

3. Review and fix individual object ACLs if needed

2.3 If Bucket Allows Public (Unauthenticated) Access

Quickest containment: enable Block A

```
aws s3api put-public-access-block --bucket \
```

```
--public-access-block-configuration \
```

```
BlockPublicAcls=true, IgnorePublicAcls=true, BlockPublicPolicy=true, RestrictPublicBucket
```

■■ This affects all objects in the bucket. Coordinate with application owners.

To fix individual object ACLs:

Remove public access from a specific

```
aws s3api put-object-acl --bucket --key --acl private
```

For bulk operations across many objects

2.4 Verify Containment

Monitor CloudTrail for continued unau

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=ResourceName,AttributeValue= \
```

```
--start-time $(date -u -v-30M +%Y-%m-%dT%H:%M:%SZ) \
```

```
--max-results 20
```

Part 3: Eradicate the Incident

Goal: Remove attack vectors, harden configurations, rotate credentials.

3.1 Harden Instance-Level Attack Vectors

If EC2 instances interact with the affected bucket:

Enforce IMDSv2 on instances (prevents

```
aws ec2 modify-instance-metadata-options \
```

```
--instance-id \
```

```
--http-tokens required \
```

```
--http-endpoint enabled
```

For SSH key rotation:

Identify instances using a specific key

```
aws ec2 describe-instances \
```

```
--query "Reservations[].Instances[].[InstanceId,KeyName,State.Name]" \
```

```
--output table
```

Consider terminating and replacing compromised instances rather than rotating keys in place.

3.2 Harden S3 Configuration

Ensure Block Public Access is enabled

```
aws s3api put-public-access-block --bucket \
    --public-access-block-configuration \
    BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBucket
```

Enable server-side encryption (SSE-KM

```
aws s3api put-bucket-encryption --bucket \
    --server-side-encryption-configuration '{
    "Rules": [{"ApplyServerSideEncryptionByDefault": {"SSEAlgorithm": "aws:kms"}}]
```

```
}'
```

Enable versioning

```
aws s3api put-bucket-versioning --bucket \
```

```
--versioning-configuration Status=Enabled
```

Enable Object Lock (only on new buckets)

```
aws s3api put-object-lock-configuration
```

```
--object-lock-configuration '{"Object'
```


Checklist:

- ☐ Object ACLs restored to secure settings
- ☐ Bucket policy implements least privilege
- ☐ Only authorized principals have data and control plane access
- ☐ IAM policies scoped appropriately; Permission Boundaries / SCPs deployed if needed
- ☐ Server-side encryption enabled (SSE-KMS)
- ☐ Versioning enabled
- ☐ Consider Object Lock for critical data

3.3 If Credential Compromise Was Involved

Issue new credentials after disabling

```
aws iam create-access-key --user-name
```

Remove unauthorized IAM users/roles

```
aws iam delete-user --user-name
```

```
aws iam delete-role --role-name
```

- ☐ Enable MFA Delete on the bucket (requires versioning)
- ☐ For objects that must be publicly readable, use pre-signed URLs instead of public ACLs
- ☐ Close/remove any unauthorized IAM users, roles, or identity providers
- ☐ Flag credential handling processes for post-incident review

Part 4: Recover from the Incident

Goal: Restore lost or modified data, verify mitigations are effective.

4.1 Restore Modified Objects

List object versions to find pre-comp

```
aws s3api list-object-versions --bucket --prefix
```

Restore a previous version by copying

```
aws s3api copy-object \
```

```
--bucket \
```

```
--key \
```

```
--copy-source /?versionId=
```

4.2 Restore Deleted Objects

If deleted without version ID, a dele

```
aws s3api delete-object --bucket --key \
```

```
--version-id
```

If deleted with a specific version ID

Check for earlier versions:

```
aws s3api list-object-versions --bucket --prefix
```

Other recovery sources:

- S3 Glacier / S3 IA (check lifecycle policies)
- S3 Cross-Region Replication (CRR) target bucket
- On-premises backups or S3 buckets in other accounts

4.3 Verify Recovery

Compare post-mitigation logs to incident-period logs:

- ☐ Data plane activity returned to pre-attack levels
- ☐ No further evidence of unauthorized access in CloudTrail
- ☐ Affected applications functioning normally
- ☐ CMDB updated with any resource changes

If suspicious activity reoccurs, return to Part 1 and reassess the attack vector.

Part 5: Post-Incident Activity

5.1 Document Lessons Learned

Create post-incident report covering:

- **Timeline:** When did each phase occur?
- **Root cause:** Misconfiguration, credential compromise, or both?
- **Impact:** What data was accessed/modified/deleted? Data classification?
- **Response effectiveness:** What worked? What didn't?
- **Recommendations:** Process/tooling improvements

5.2 Retrospective Questions

- What information would have helped respond more swiftly?
- What detection would have alerted to the issue sooner?
- What configuration would have prevented this exposure?
- What automation or tooling would have made root cause investigation easier?

5.3 Update Defenses

Based on findings:

- ☐ Update risk documents with newly discovered threat/vulnerability combinations
- ☐ Implement required infrastructure or application configuration changes
- ☐ Update CMDB entries for affected applications and buckets
- ☐ Review and update this playbook with lessons learned

- ☐ Assign follow-up actions from Parts 3, 4, and 5 and track to completion

5.4 Regulatory Notifications

If required by your jurisdiction:

- ☐ Notify relevant authorities within required timeframe
- ☐ Document notification for compliance records

References

- [S3 Security Best Practices](#)
- [IAM Access Analyzer](#)
- [S3 Block Public Access](#)
- [S3 Server-Side Encryption](#)
- [S3 Versioning](#)
- [S3 Object Lock](#)
- [IMDSv2 for SSRF Mitigation](#)
- [Querying CloudTrail Logs with Athena](#)
- [AWS Security Incident Response Guide](#)

description: Guidance for creating new AWS incident response skills for Claude Code by translating human-centric IR playbooks into agent-actionable skill files. Use this skill when you want to create a new IR playbook skill, translate an existing playbook from the playbooks/ folder into a Claude Code skill, or add a new incident type to the IR skill library.

Guidance: Translating IR Playbooks into Agent-Centric Claude Code Skills

Purpose

This skill guides you (the agent) through creating a new incident response skill by translating a human-centric IR playbook from the `playbooks/` folder into an agent-actionable skill file stored in `.claude/skills/`.

Procedure Overview

1. User indicates which playbook(s) in `playbooks/` to translate
2. You analyze the source playbook, reference existing skills for format/structure, and produce a new skill file
3. You register the new skill in the routing system so it can be invoked during incidents

Step 1: Analyze the Source Playbook

Read the source playbook from `playbooks/` and extract:

- The incident type (e.g., "Credential Compromise", "Unintended S3 Data Access", "DoS/DDoS")
- The NIST 800-61 phases covered (Detection/Evidence, Containment, Eradication, Recovery, Post-Incident)
- Specific investigation steps, containment actions, and recovery procedures
- Any AWS services, APIs, or CLI commands referenced
- Alert sources and detection mechanisms mentioned

Step 2: Reference Existing Skills

Read the existing skills in `.claude/skills/` to understand the target format:

- `.claude/skills/skill-irp-credential-compromise.md`
- `.claude/skills/skill-irp-data-access.md`

Use these as structural templates. The new skill file **MUST** follow the same patterns.

Step 3: Create the New Skill File

3.1 File Naming Convention

The file MUST be named: `skill-irp-.md`

Where is a concise, lowercase, hyphenated descriptor of the incident type.

Examples:

- `skill-irp-credential-compromise.md`
- `skill-irp-data-access.md`
- `skill-irp-dos-ddos.md`

- `skill-irp-ransomware.md`

Store the file in: `.claude/skills/`

3.2 Front-Matter (REQUIRED)

Every skill file MUST begin with a YAML front-matter description block:

```
---
```

```
description: Incident response playbook for . Use this skill when responding to .
```

```
---
```

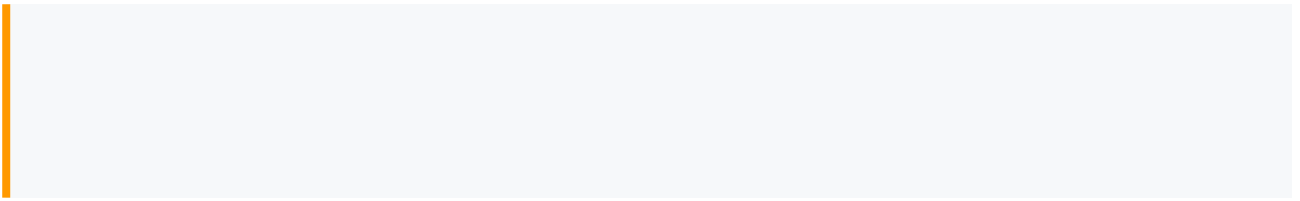
Rules:

- The `description` field is used by Claude Code for skill trigger matching — make it keyword-rich with terms that operators are likely to use when describing this incident type
- Do NOT include an `inclusion` field — skills are loaded on demand, not based on inclusion metadata
- Do NOT include cross-references to sibling skills in the description — `CLAUDE.md` handles all routing

3.3 Required Document Structure

The new skill file MUST follow this exact 5-part structure aligned with NIST 800-61 R2:

Playbook:



Incident Type

Quick Reference

Part 1: Acquire, Preserve, Document Evidence

1.1 Identify the Alert Source

1.2

1.x Document and Communicate

Part 2: Contain the Incident

2.x

2.x Verify Containment

Part 3: Eradicate the Incident

3.x

Part 4: Recover from the Incident

4.x

4.x Verify Recovery

Part 5: Post-Incident Activity

5.1 Document Lessons Learned

5.2 Retrospective Questions

5.3 Update Defenses

5.4 Regulatory Notifications

References

3.4 Content Transformation Rules

When translating from the human-centric playbook to the agent-centric skill file:

STRIP the following from source playbooks:

- Legal disclaimers and shared responsibility boilerplate
- "Introduction" and "Summary" sections that describe the playbook itself
- References to "customizing this template" or "Game Days"
- Cost Explorer disclaimers about incurred costs
- Generic NIST framework explanations (the core playbook already covers this)

TRANSFORM the following:

- Numbered prose steps → concise subsections with descriptive headings
- Vague instructions like "check CloudTrail" → specific AWS CLI commands with placeholders
- Human decision points → checklists with - [] checkboxes
- References to "consult your CMDB" → keep but make them actionable checklist items
- Long paragraphs → bullet points or short paragraphs

ADD the following if missing from the source playbook:

- AWS CLI commands for every investigation and remediation step where applicable
- A Quick Reference table summarizing all phases
- Verification steps after containment and recovery
- Specific API call names to look for in CloudTrail (relevant to the incident type)

- Warning callouts (■) for actions that affect production workloads
- Goal statements at the start of Parts 2, 3, and 4

PRESERVE the following from source playbooks:

- The logical flow and ordering of incident response phases
- All AWS service-specific guidance and technical details
- Alert source identification steps
- Communication and stakeholder notification steps
- Post-incident and lessons-learned processes

3.5 CLI Command and Tool Selection Standards

All IR skills use AWS CLI commands as the canonical representation of each operation. The

When authoring CLI commands in the skill file:

- Use aws CLI syntax
- Include placeholder parameters in angle brackets:
- Wrap in ``bash code blocks
- Include a brief comment above the command explaining its purpose when not obvious
- Group related commands under a single code block where logical

MCP-aware authoring: Where a set of CLI commands would benefit from parallel execution,

MCP batch opportunity: the following

```
aws s3api get-public-access-block --bucket
```

```
aws s3api get-public-access-block --bucket
```

```
aws s3api get-public-access-block --bucket
```

The agent will use MCP batch calls when available, or fall back to sequential CLI execut

3.6 Quality Checklist Before Saving

Before writing the file, verify:

- ☐ Front-matter has a keyword-rich description covering the main trigger scenarios f
- ☐ No inclusion field in front-matter
- ☐ No sibling skill cross-references in description (CLAUDE.md handles routing)

- [] All 5 NIST phases are covered (Evidence, Contain, Eradicate, Recover, Post-Incidence)
- [] Quick Reference table is present with all 5 phases
- [] CLI commands use placeholder parameters, not hardcoded values
- [] Warning callouts exist for destructive or production-impacting actions
- [] Verification steps exist after containment and recovery
- [] References section includes relevant AWS documentation links
- [] No legal boilerplate or template meta-commentary remains from the source playbook
- [] File is named skill-irp-.md
- [] File is saved in .claude/skills/

Step 4: Register the New Skill in the Routing Sys

After creating the skill file, you MUST update CLAUDE.md to wire it into the incident r

4.1 Update CLAUDE.md

Edit CLAUDE.md at the repo root:

1. In the "Keyword Pattern Matching" section (Step 1), add relevant primary and secondar

2. If the new incident type warrants its own selection logic, add a new subsection under

3. Update the "Default option" section if the new incident type should be the default for

Note: Unlike the Kiro steering system, you do NOT need to update sibling skill files – O

Example Workflow

User prompt: "Create a new IR skill based on the DoS playbook in playbooks/IRP-DoS.md"

Agent actions:

1. Read playbooks/IRP-DoS.md

2. Read existing skills for format reference (.claude/skills/skill-irp-credential-comprom

3. Create .claude/skills/skill-irp-dos-ddos.md following all rules in Step 3

4. Update CLAUDE.md` keyword routing (Step 4.1)

5. Present the created file to the user for review

■ ai-playbooks/skills/skill-irp-ransomware.md

description: Incident response playbook for ransomware incidents affecting AWS resources

Playbook: Ransomware

Incident Type

Ransomware – when malicious code encrypts data or locks access to AWS resources (EC2 in

Quick Reference

| Phase | Key Action | Verification |

|-----|-----|-----|

| Evidence | Identify affected resources, ransomware strain, attack vector, timeline | A

| Contain | Isolate infected resources, revoke compromised credentials, block spread vec

| Eradicate | Remove malware, eliminate persistence mechanisms, patch attack vector | N

| Recover | Restore from clean backups/snapshots, rebuild if necessary | Data integrity

| Post-Incident | Document lessons learned, harden defenses | Report filed, playbook up

Part 1: Acquire, Preserve, Document Evidence

1.1 Identify the Alert Source

Common sources for ransomware alerts:

- **EC2 instance unreachable** (SSH/RDP fails despite correct network config, no AWS service) *(Note: This alert is not specific to ransomware)*
- **Monitoring alarms** (CloudWatch metrics anomalies, application health checks failing)
- **GuardDuty findings** (e.g., malware-related or unauthorized access findings)
- **Security Hub alerts**
- **Ransom demand** received via email, on-screen message, or alternate channel
- **Suspicious S3 bucket with threatening name** (e.g., *we-stole-ur-data-*, *your-files-encrypt-*)
- **Ransom note object found in S3** (e.g., README_DECRYPT.txt, warning.txt, all_your_data_is_gone.txt)
- **Billing anomalies** (unexpected cost spikes from attacker resource usage)

- **Amazon Inspector findings** (known CVE exploitation)
- **External notification** (security researcher, law enforcement, anonymous tip)

1.2 Determine Ransomware Type

Identify the variant to guide response strategy:

- **Crypto ransomware** – files/objects are encrypted (check S3 object properties for unexp
- **Locker ransomware** – device access is blocked (instance unreachable but running)
- **Extortion ransomware (delete + extort)** – data is exfiltrated then deleted (not encryp
- **Unknown/hybrid** – treat as crypto ransomware until confirmed otherwise

Check S3 object encryption (if S3 objects are inaccessible):

Check encryption properties on affected

```
aws s3api head-object --bucket --key
```

List recent PutObject or CopyObject e

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventSource,AttributeValue=s3.amazonaws.com \
```

```
--start-time \
```

```
--max-results 50
```

Check EBS volume encryption changes:

Describe volumes attached to the affe

```
aws ec2 describe-volumes \
```

```
--filters Name=attachment.instance-id,Values= \
```

```
--query 'Volumes[][VolumeId,Encrypted,KmsKeyId,State]' \
```

```
--output table
```

1.3 Identify Affected Resources and Scope

Describe the affected EC2 instance

```
aws ec2 describe-instances --instance-ids
```

Check instance status and reachability

```
aws ec2 describe-instance-status --instance-ids
```

Use Amazon Detective to investigate a

Navigate to Detective console → Search

Determine scope:

- [] How many instances/resources are affected?
- [] Is the infection spreading to other resources?
- [] What is the data classification level of affected resources?
- [] Are there related abuse notifications from AWS?

1.4 Establish Timeline

Search CloudTrail for suspicious activity

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=ResourceName,AttributeValue= \
```

```
--start-time \
```

```
--max-results 50
```

Check CloudWatch metrics for anomalies

```
aws cloudwatch get-metric-statistics \
```

```
--namespace AWS/EC2 \
```

```
--metric-name NetworkIn \
```

```
--dimensions Name=InstanceId,Value= \
```

```
--start-time \
```

```
--end-time \
```

```
--period 300 \
```

```
--statistics Average
```

Document:

- **First anomaly:** When did metrics/logs first show unusual behavior?
- **Likely infection time:** When was the ransomware likely deployed? (may precede detection)
- **Detection time:** When was the incident identified?
- **Response start:** When did you begin responding?

1.5 Check for Credential Compromise as Attack Vector

Search CloudTrail for unauthorized IAM

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventName,AttributeValue=CreateUser \
```

```
--start-time \
```

```
--max-results 20
```

Check for new access keys created

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventName,AttributeValue=CreateAccessKey \
```

```
--start-time \
```

```
--max-results 20
```

If credential compromise is confirmed, also invoke the credential compromise skill (ski

1.6 Document and Communicate

- [] Create/update incident ticket with: affected resource IDs, ransomware type, timeL
- [] Identify stakeholders from CMDB or application risk register
- [] Open incident response bridge
- [] Notify: Security team, application owners, legal counsel
- [] If regulated data is involved, notify compliance team
- [] Inform law enforcement contacts if required by jurisdiction

- [] Preserve evidence chain of custody documentation

Part 2: Contain the Incident

Goal: Isolate infected resources to prevent spread and block attacker access.

2.1 Network Isolation of Affected Instances

Create a quarantine security group with

```
aws ec2 create-security-group \
```

```
--group-name ransomware-quarantine \
```

```
--description "Quarantine SG for ransomware incident" \
```

```
--vpc-id
```

Replace all security groups on the at

```
aws ec2 modify-instance-attribute \
```

```
--instance-id \
```

```
--groups
```

■■ **Warning:** This immediately cuts all network access to the instance, including legitim

Note on connection tracking: Existing established connections may persist after security

Add deny-all rules to the subnet NACL

```
aws ec2 create-network-acl-entry \
```

```
--network-acl-id \
```

```
--rule-number 1 \
```

```
--protocol -1 \
```

```
--rule-action deny \
```

```
--cidr-block 0.0.0.0/0 \
```

```
--ingress
```

```
aws ec2 create-network-acl-entry \
```

```
--network-acl-id \
```

```
--rule-number 1 \
```

```
--protocol -1 \
```

```
--rule-action deny \
```

```
--cidr-block 0.0.0.0/0 \
```

```
--egress
```

■■ **Warning:** NACL changes affect all instances in the subnet. Only use if the entire sub

2.2 Contain S3-Based Ransomware

This section covers two S3 ransomware patterns: **crypto** (objects re-encrypted) and **extort**

Step 1 – Identify and investigate any suspicious new buckets:

List all buckets with creation timestamp

```
aws s3api list-buckets --query 'Buckets[].[Name,CreationDate]' --output table
```

List contents of any suspicious buckets

```
aws s3 ls s3:// --recursive
```

Read any ransom note objects

```
aws s3 cp s3:/// -
```

Step 2 – Check all victim buckets for planted ransom notes:

Look for unexpected objects in bucket

Common ransom note filenames: warning

```
aws s3 ls s3:// | grep -iE 'warning|readme|decrypt|ransom|locked|restore'
```

Step 3 – Assess data recoverability via versioning:

Check versioning status – if Suspended

```
aws s3api get-bucket-versioning --bucket
```

If versioning is Enabled, check for o

```
aws s3api list-object-versions --bucket \
```

```
--query 'DeleteMarkers[][Key,LastModified,VersionId]' --output table
```

If versioning is Enabled, restore a o

```
aws s3api delete-object --bucket --key \
```

```
--version-id
```

■■ If versioning was NOT enabled on the bucket when the deletion occurred, the objects

Step 4 – Restrict the attacker-created ransom bucket:

Prevent further writes to the attacked

```
aws s3api put-bucket-policy --bucket --policy '{
```

```
  "Version": "2012-10-17",
```

```
  "Statement": [{
```

```
    "Effect": "Deny",
```

```
    "Principal": "",
```

```
    "Action": "s3:",
```

```
    "Resource": [
```

```
      "arn:aws:s3:::",
```

```
      "arn:aws:s3:::/"
```

```
    ],
```

```
    "Condition": {
```

```
"StringNotEquals": {"aws:PrincipalAccount": ""}
```

```
}
```

```
}]
```

```
}'
```

Step 5 – Lock down victim buckets to stop further access:

Enable Block Public Access on all affected buckets

```
aws s3api put-public-access-block --bucket \
```

```
--public-access-block-configuration \
```

```
BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBucket
```

Apply restrictive bucket policy to de

```
aws s3api put-bucket-policy --bucket --policy '{
```

```
  "Version": "2012-10-17",
```

```
  "Statement": [{
```

```
    "Effect": "Deny",
```

```
    "Principal": "",
```

```
    "Action": "s3:",
```

```
    "Resource": [
```

```
      "arn:aws:s3:::",
```

```
      "arn:aws:s3::/"
```

```
    ],
```

```
    "Condition": {
```

```
      "StringNotEquals": {"aws:PrincipalAccount": ""}
```

```
}
```

```
}]
```

```
}'
```

2.3 Revoke Compromised Credentials

If credentials were used as the attack vector:

Disable IAM user access keys

```
aws iam update-access-key \
```

```
--user-name \
```

```
--access-key-id \
```

```
--status Inactive
```

Revoke temporary role sessions

```
aws iam put-role-policy \
```

```
--role-name \
```

```
--policy-name AWSRevokeOlderSessions \
```

```
--policy-document '{
```

```
  "Version": "2012-10-17",
```

```
  "Statement": [{
```

```
    "Effect": "Deny",
```

```
    "Action": [""],
```

```
    "Resource": [""],
```

```
    "Condition": {
```

```
"DateLessThan": {"aws:TokenIssueTime": ""}
```

```
}
```

```
}]
```

```
}'
```

2.4 Check for Unauthorized IAM Activity and Revoke

Check for unauthorized IAM users, po

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventName,AttributeValue=CreateUser \
```

```
--start-time \
```

```
--max-results 20
```

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventName,AttributeValue=CreateRole \
```

```
--start-time \
```

```
--max-results 20
```

MCP batch opportunity: the above look

Delete any unauthorized IAM users, roles, policies, and revoke temporary credentials for

2.5 Consider Account-Level Containment (Drastic)

If the infection is widespread and the account is part of AWS Organizations:

Apply a deny-all SCP to the affected

```
aws organizations create-policy \
```

```
--name "RansomwareQuarantine" \
```

```
--description "Emergency quarantine for ransomware incident" \
```

```
--type SERVICE_CONTROL_POLICY \
```

```
--content '{
```

```
  "Version": "2012-10-17",
```

```
  "Statement": [{
```

```
    "Effect": "Deny",
```

```
    "Action": "",
```



```
"Resource": ""
```

```
}]
```

```
}'
```

```
aws organizations attach-policy \
```

```
--policy-id \
```

```
--target-id
```

■■ **Warning:** This blocks ALL API calls in the account, affecting all workloads. Use only

2.6 Patch the Attack Vector

If the ransomware exploited unpatched software:

- For mutable architectures: apply patches in place using AWS Systems Manager
- For immutable architectures: update the AMI and redeploy

Check for missing patches via Systems

```
aws ssm describe-instance-patch-states --instance-ids
```

Run patch baseline

```
aws ssm send-command \
```

```
--instance-ids \
```

```
--document-name "AWS-RunPatchBaseline" \
```

```
--parameters '{"Operation":["Install"]}'
```

2.7 Verify Containment

Monitor for 30+ minutes to confirm no new infections or attacker activity:

Monitor CloudTrail for continued acti

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=AccessKeyId,AttributeValue= \
```

```
--start-time $(date -u -v -30M +%Y-%m-%dT%H:%M:%SZ) \
```

```
--max-results 10
```

Check for new GuardDuty findings

```
aws guardduty list-findings --detector-id \
```

```
--finding-criteria '{"Criterion":{"updatedAt":{"GreaterThanOrEqualTo":"'$(date -u -v-30M
```

If new infections or activity appear, containment has failed – reassess the attack vector

Part 3: Eradicate the Incident

Goal: Remove all malicious artifacts, eliminate persistence mechanisms, and close the attack.

3.1 Identify the Ransomware Strain

- Review ransom notes, encrypted file extensions, or on-screen messages
- Cross-reference with known ransomware databases (e.g., ID Ransomware, No More Ransom)
- Check if third-party decryption tools are available for the identified strain

3.2 Forensic Analysis in Isolated Environment

■■ **Warning:** Do not run anti-malware on the live infected instance – this may alert the

Create a snapshot of the infected instance

```
aws ec2 create-snapshot \
```

```
--volume-id \
```

```
--description "Forensic snapshot - ransomware incident "
```

Create an AMI of the infected instance

```
aws ec2 create-image \
```

```
--instance-id \
```

```
--name "forensic-image-" \
```

```
--description "Forensic image - ransomware incident" \
```

```
--no-reboot
```

Analyze the forensic copy in an isolated environment:

- Run anti-malware/anti-virus scans
- Identify indicators of compromise (IOCs)
- Determine the initial infection vector
- Identify any lateral movement or persistence mechanisms

3.3 Review GuardDuty Findings

List recent high/medium severity findings

```
aws guardduty list-findings --detector-id \
```

```
--finding-criteria '{"Criterion":{"severity":{"GreaterThanOrEqualTo":4}}}'
```

Get finding details

```
aws guardduty get-findings --detector-id --finding-ids
```

GuardDuty findings provide remediation recommendations – follow them for each relevant finding.

3.4 Remove Malicious Artifacts

Based on forensic analysis:

- [] Remove any malware identified on infected instances
- [] Delete unauthorized IAM users, roles, policies, and access keys
- [] Terminate any attacker-created resources (EC2 instances, Lambda functions, etc.)
- [] Remove any persistence mechanisms (scheduled tasks, cron jobs, startup scripts)
- [] Revoke any modified trust policies or permission escalations

List all IAM users (look for unfamiliar)

```
aws iam list-users
```

List all roles (look for unfamiliar ones)

```
aws iam list-roles
```

Check for recently launched instances

```
aws ec2 describe-instances \
```

```
--filters Name=launch-time,Values=* \
```

```
--query 'Reservations[].Instances[].[InstanceId,LaunchTime,InstanceType,Tags]' \
```

```
--output table
```

MCP batch opportunity: the above checks

3.5 Close the Attack Vector

Based on root cause analysis:

- [] Patch exploited vulnerabilities on all potentially affected instances
- [] Update AMIs used in Auto Scaling launch configurations/templates
- [] Verify patches against the Mitre CVE database
- [] Block identified malicious IPs/domains in security groups or WAF rules
- [] Enforce IMDSv2 if SSRF was part of the attack chain

Enforce IMDSv2 on instances

```
aws ec2 modify-instance-metadata-options \
```

```
--instance-id \
```

```
--http-tokens required \
```

```
--http-endpoint enabled
```

```
---
```

Part 4: Recover from the Incident

Goal: Restore data and services from clean backups, or rebuild from known-good configurations

4.1 Identify Restore Points

List available EBS snapshots

```
aws ec2 describe-snapshots --owner-ids self \
```

```
--filters Name=volume-id,Values= \
```

```
--query 'Snapshots[][SnapshotId,StartTime,Description]' \
```

```
--output table
```

List available AMIs

```
aws ec2 describe-images --owners self \
```

```
--query 'Images[].[ImageId,CreationDate,Name]' \
```

```
--output table
```

List RDS snapshots (if applicable)

```
aws rds describe-db-snapshots \
```

```
--query 'DBSnapshots[].[DBSnapshotIdentifier,SnapshotCreateTime,Status]' \
```

```
--output table
```

Check S3 versioning for object recovery

```
aws s3api list-object-versions --bucket --prefix
```

MCP batch opportunity: the above check

Choose a restore point that predates the infection time established in Part 1.

4.2 Restore from Backups

For EC2 instances (from snapshot):

Create a new volume from a clean snap

```
aws ec2 create-volume \
```

```
--snapshot-id \
```

```
--availability-zone \
```

```
--volume-type gp3
```

Terminate the infected instance (or I

Launch a new instance with the restor

For S3 objects (from versioning):

Restore a previous version by copying

```
aws s3api copy-object \
```

```
--bucket \
```

```
--key \
```

```
--copy-source /?versionId=
```

Remove delete markers if objects were

```
aws s3api delete-object --bucket --key \
```

```
--version-id
```

For Auto Scaling groups:

Update launch template with clean AMI

```
aws ec2 create-launch-template-version \
```

```
--launch-template-id \
```

```
--source-version \
```

```
--launch-template-data '{"ImageId":""}'
```

Trigger instance refresh

```
aws autoscaling start-instance-refresh \
```

```
--auto-scaling-group-name
```

If no backup exists and decryption tools are available:

- Use identified third-party decryption tools in an isolated environment

- Validate decrypted data integrity before restoring to production
- Remove decrypted data from the infected instance, rebuild the instance, then restore

4.3 Rebuild if Necessary

If backups are unavailable and decryption is not possible:

- Rebuild from CMDB configuration documentation
- Redeploy from infrastructure-as-code (CloudFormation, CDK, Terraform)
- Restore application data from any available secondary sources (cross-region replicas,

4.4 Verify Recovery

- ☐ Restored data integrity confirmed (checksums, application-level validation)
- ☐ Affected applications functioning normally
- ☐ Monitoring dashboards show normal metrics
- ☐ No evidence of reinfection in CloudTrail or GuardDuty
- ☐ CMDB updated with any resource changes

If suspicious activity reoccurs, return to Part 1 and reassess.

Part 5: Post-Incident Activity

5.1 Document Lessons Learned

Create post-incident report covering:

- **Timeline:** When did each phase occur?
- **Root cause:** How did the ransomware gain entry? (unpatched vulnerability, phishing, cr
- **Ransomware strain:** What variant was used? Was decryption possible?
- **Impact:** What resources were affected? What data was lost or encrypted? Data classific
- **Response effectiveness:** What worked? What didn't? How long did each phase take?
- **Recommendations:** Process, tooling, and architecture improvements

5.2 Retrospective Questions

- What detection would have alerted to the ransomware sooner?
- Were backups adequate for recovery? Were they tested?
- What patching or configuration gap allowed the initial infection?
- Could network segmentation have limited the blast radius?
- What automation would have accelerated containment?

5.3 Update Defenses

Based on findings:

- [] Ensure all instances are patched and using current AMIs

- ☐ Enable/verify AWS Backup with appropriate retention policies
- ☐ Enable S3 Versioning and consider Object Lock for critical data
- ☐ Enforce IMDSv2 across all EC2 instances
- ☐ Review and tighten IAM policies (least privilege)
- ☐ Enable/enhance GuardDuty, Security Hub, and Amazon Inspector
- ☐ Implement network segmentation to limit lateral movement
- ☐ Deploy endpoint protection on EC2 instances
- ☐ Consider SCPs or permission boundaries for additional guardrails
- ☐ Update this playbook with lessons learned

5.4 Regulatory Notifications

If required by your jurisdiction:

- [] Notify relevant authorities within required timeframe

- [] Report to law enforcement agencies if applicable

- [] Document notification for compliance records

- [] Consider voluntary reporting to ransomware tracking databases or government agencies

References

- [AWS Security Incident Response Guide](#)
- [NIST SP 800-61 R2 – Computer Security Incident Handling Guide](#)
- [AWS Well-Architected Security Pillar](#)
- [Amazon S3 Security Best Practices](#)
- [AWS Backup Documentation](#)
- [Amazon GuardDuty Documentation](#)
- [Amazon Inspector Documentation](#)
- [IMDSv2 for SSRF Mitigation](#)
- [AWS Systems Manager Patch Manager](#)
- [Querying CloudTrail Logs with Athena](#)
- [No More Ransom Project](#)

■ ai-playbooks/steering/README.md

AI-Powered Incident Response Automation

This README relates specifically to this directory subsection of the aws-incident-response

These markdown documents are created to be used as templates only. They should be custom

These markdown documents are written to facilitate editing and consumption into a variety

<<<<<<< HEAD:ai-playbooks/README.md

Note that you need to copy the files in this repo into the correct directory in your IDE

The markdown documents included cover several common scenarios faced by AWS customers. 7

=====

The markdown documents included cover several common scenarios faced by AWS customers. 7

>>>>>> upstream/master:ai-playbooks/steering/README.md

Gather evidence

Contain and then eradicate the incident

Recover from the incident

Conduct post-incident activities, including post-mortem and feedback processes

Usage

Kiro IDE

File types

There are three categories of markdown files we have created for this AI-powered incident

Steering Factory: Use the `.kiro-steering/steering-factory/steering-factory-creation-gu`

Steering Core: The `.kiro-steering/steering-irp-core.md` file is `_always_` used when you a

Steering Incident Response: These files are invoked as needed based on the initial prob

How the files direct Kiro IDE

The core file (`steering-irp-core.md`) is set to inclusion: `always`, so it's loaded into ev

The incident-specific files (such as "credential compromise", "unintended data access",

So the flow is:

User describes a security incident

Core file (always loaded) kicks in, analyzes the prompt against its keyword patterns

Core file directs the AI to invoke the appropriate manual steering file(s)

The specific playbook guides the actual IR steps

It's essentially a router pattern – one always-on dispatcher that selectively activates

config:

flowchart:

padding: 15

flowchart TD

*A[User describes security incident] --> B[steering-irp-core.md
always loaded]*

*B --> C{Keyword Pattern
Matching}*

*C -->|credential, access key,
IAM user, GuardDuty| D[steering-irp-credential-compromise.md]*

*C -->|S3 bucket, data access,
public bucket, object ACL| E[steering-irp-data-access.md]*

*C -->|Ransomware, ransom,
encrypted files, locked out| F[steering-irp-ransomware.md]*

C -->|no clear match| D

*D --> G[Execute NIST 800-61 Lifecycle
Evidence → Contain → Eradicate → Recover → Post-Incident]*

E --> G

F --> G

*G --> H{Multi-vector
detected?}*

*H -->|Yes| I[Invoke additional
playbook]*

I --> G

*H -->|No| J[Root cause analysis
(actions taken)
+ further actions needed]*

style B fill:#2d6a4f,color:#fff

style D fill:#1b4965,color:#fff

style E fill:#1b4965,color:#fff

style F fill:#1b4965,color:#fff

style C fill:#6c757d,color:#fff

```
style H fill:#6c757d,color:#fff
```

The "Tool Selection Strategy" section in the core file is also a nice touch – it tells

Claude Code

Claude Code uses a parallel structure to the Kiro steering system, adapted to how Claude

```
#### File types
```

There are three categories of files for Claude Code:

Skill Factory: Use `.claude/skills/skill-irp-playbook-factory.md` to create new IR skills

CLAUDE.md (Core Router): The `CLAUDE.md` file at the repo root is **always loaded** by Claude

IR Skills: The incident-specific skill files (`.claude/skills/skill-irp-.md`) are *invoked*

Architecture mapping

Kiro Concept	Claude Code Equivalent	
--------------	------------------------	--

---	---	
-----	-----	--

<code>steering-irp-core.md</code> (inclusion: always)	<code>CLAUDE.md</code> at repo root (always loaded)	
---	---	--

<code>steering-irp-.md</code> (inclusion: manual)	<code>.claude/skills/skill-irp-*.md</code> (loaded on demand)	
---	---	--

Steering factory guide	<code>.claude/skills/skill-irp-playbook-factory.md</code>	
------------------------	---	--

How the files direct Claude Code

CLAUDE.md is loaded into every conversation automatically. It acts as the orchestrator -

The incident-specific skills (credential compromise, unintended data access, ransomware,

The flow is identical to Kiro:

config:

flowchart:

padding: 15

flowchart TD

A[User describes security incident] --> B[CLAUDE.md
always loaded]

B --> C{Keyword Pattern
Matching}

C -->|credential, access key,
IAM user, GuardDuty| D[skill-irp-credential-compromise]

C -->|S3 bucket, data access,
public bucket, object ACL| E[skill-irp-data-access]

C -->|Ransomware, ransom,
encrypted files, locked out| F[skill-irp-ransomware]

C -->|no clear match| D

D --> G[Execute NIST 800-61 Lifecycle
Evidence → Contain → Eradicate → Recover → Post-Incident]

E --> G

F --> G

G --> H{Multi-vector
detected?}

H -->|Yes| I[Invoke additional skill]

I --> G

H -->|No| J[Root cause analysis (actions taken) + further actions needed]

style B fill:#2d6a4f,color:#fff

style D fill:#1b4965,color:#fff

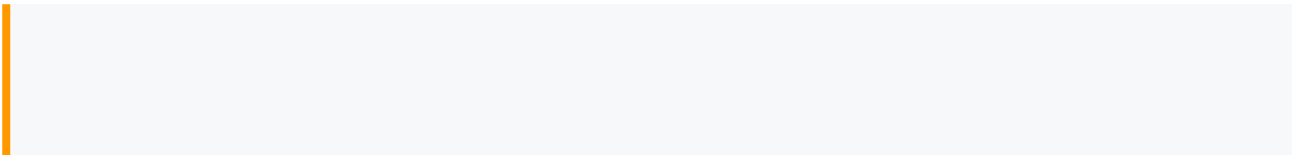
style E fill:#1b4965,color:#fff

style F fill:#1b4965,color:#fff

style C fill:#6c757d,color:#fff

style H fill:#6c757d,color:#fff

The Tool Selection Strategy section in CLAUDE.md tells Claude Code to prefer MCP tools (



Security

See [CONTRIBUTING](#) for more information.

Artificial intelligence can make mistakes. Ensure that operators carefully check and verify

License Summary

The documentation is made available under the Creative Commons Attribution-ShareAlike 4

The sample code within this documentation is made available under the MIT-0 license. See

■ `ai-playbooks/steering/steering-irp-core.md`

inclusion: always

description: |

This is the core playbook that will ALWAYS be invoked when taking actions on security

- Invoke with "steering-irp-credential-compromise.md" when responding to compromised cr
- Invoke with "steering-irp-data-access.md" when responding to unintended access to Ama
- Invoke with "steering-irp-ransomware.md" when responding to ransomware incidents.

- Invoke with "steering-irp-api-security-breach.md" when responding to API security inc
- Invoke with "steering-factory-creation-guide.md" when the user wants to translate an
- Invoke with "steering-irp-builder.md" when the user wants to build a new IR steering

PRIORITY: The incident response workf

When user requests security incident

Core Philosophy & Principles

Human-directed instruction

You are authorized to gather account data and present it back to the incident response c

Incident response life cycle

When you start incident response (IR) process, ALWAYS follow the IR life cycle aligned w

Step1: Detection - Identify potential security events

Step2: Analysis - Determine if an event is an incident and assess scope

Step3: Containment - Minimize and limit the scope of the security event

Step4: Eradication - Remove unauthorized resources/artifacts and implement mitigations

Step5: Recovery - Restore systems to known safe state and monitor for threat recurrence

Cyclical lifecycle: This is NOT a linear process. Findings at any phase may require loop

Use specific incident response playbook based on

- When you determine to trigger the IR process, ALWAYS start from this core playbook
- Then based on the following playbook selection section, you will choose specific IR p
- All specific IR playbooks stored in either ".kiro/steering/" folder for Kiro and Kiro

Detailed approach to select specific

ALWAYS starts from analyzing user's prompt

Step 1: Keyword Pattern Matching

Check if the prompt contains direct indicators:

Primary Keywords:

- "credential" + ("compromise", "leak", "exposed", "stolen", "unauthorized")
- "access key" + ("compromised", "leaked", "exposed")
- "IAM user" + ("compromised", "unauthorized")
- "GuardDuty" + ("finding", "alert")
- "unauthorized access"
- "credential exfiltration"
- "ransomware" / "ransom" / "encrypted files" / "locked out"

- "ransom demand" / "ransom note"

- "crypto ransomware" / "locker ransomware"

- "API" + ("attack", "breach", "exploit", "abuse", "vulnerability", "injection", "unaut"

- "OWASP" / "BOLA" / "BFLA" / "broken authentication" / "broken authorization"

- "API Gateway" + ("attack", "suspicious", "abuse", "exploit")

- "WAF" + ("alert", "block", "trigger", "rule")

- "SQL injection" / "command injection" / "XSS" + ("API", "endpoint")

- "rate limiting" + ("bypass", "exhaustion", "abuse")

- "SSRF" / "server-side request forgery"

- "API key" + ("compromised", "leaked", "stolen", "abuse")

Secondary Keywords:

- "suspicious activity"

- "unknown API calls"

- "billing spike" / "unexpected costs"

- "security alert"

- "CloudTrail" + ("suspicious", "unauthorized")

- "instance unreachable" + ("encrypted", "locked", "ransom")

- "files encrypted" / "data encrypted" / "objects inaccessible"

- "strange bucket" / "unknown bucket" / "unexpected S3 bucket" / "suspicious bucket name"

- "unrecognized resource" / "unknown resource" / "unexpected resource"

- "4xx spike" / "5xx spike" / "error rate" + ("API", "endpoint")

- "brute force" + ("login", "API", "authentication")

- "credential stuffing"

- "data leakage" / "excessive data" + ("API", "response")

- "bot traffic" / "automated abuse" + ("API", "endpoint")

- "unexpected API traffic" / "anomalous requests"

Step 2: Context Analysis

Check for incident characteristics mentioned:

1. Alert Sources:

- GuardDuty findings mentioned
- Security Hub alerts
- CloudWatch alarms on IAM
- AWS Config non-compliance
- Billing anomalies
- External notification (researcher, tip)

2. Suspicious Activities:

- Unfamiliar IAM users/roles created
- New access keys on existing users
- API calls from unusual locations/IPs
- Unauthorized resource creation (EC2, Lambda, S3)
- IAM policy modifications
- CloudTrail logging disabled

3. Timeline Indicators:

- "First seen" timestamps
- "Started happening" timeframes
- Recent IAM changes

4. API Security Indicators:

- WAF rule triggers (SQLi, XSS, rate-based)
- API Gateway 4xx/5xx error rate spikes
- Unusual API traffic patterns or volumes
- Brute-force authentication attempts
- Sequential ID enumeration (BOLA pattern)
- Unauthorized endpoint access attempts
- Data leakage in API responses
- Lambda/ECS GuardDuty findings related to API backends
- API key abuse or compromise
- SSRF attempts through API parameters

5. Ransomware Indicators:

- Ransom demand or ransom note received
- Files or S3 objects encrypted with unknown keys

- EC2 instances unreachable despite correct network configuration
- EBS volume encryption changes
- Unusual data transfer patterns (potential exfiltration before encryption)
- Anti-malware or endpoint protection alerts

Default option

- If no keywords from Step 1 or Step 2 match, fall back to steering-irp-credential-comp
- The default does NOT override or suppress other steering files – if keyword or contex

Expected behaviour

- You start the incident response based on this core playbook
- Then decide which specific IR steering file(s) to use from the `.kiro/steering/` folder
- **Re-evaluate steering file selection as new evidence emerges** – if investigation findings
- Follow specific IR steering file(s) to walk through the incident response life cycle
- Presents critical findings to user, and ask for approval **WHENEVER** you need to change
- By end of the process, **ALWAYS** present a root cause analysis to user, actions taken, a

Investigation Resilience Principle

During Phase 2 (Analysis), some investigation paths may be blocked – for example, an IA

Examples of fallback thinking:

- `cloudtrail:LookupEvents` denied → find the CloudTrail S3 trail via `describe-trails`, then
- GuardDuty API denied → look for equivalent signals in CloudTrail management events, such as
- S3 server access logs unavailable → check CloudTrail S3 data events if enabled
- Direct API access denied → check if data is available via Resource Explorer, AWS Con

Apply this principle broadly across all IR phases: if a specific tool or API is unavailable

Tool Selection Strategy

When executing IR actions, use the most efficient tool available in your environment. For

1. **MCP tools (preferred):** If an MCP server is available (e.g., AWS API MCP Server with c

2. **AWS CLI (fallback):** If MCP tools are not available, execute the equivalent aws CLI c

The CLI commands in each IR steering file represent the logical operations to perform. T

When MCP batch execution is especially valuable during IR:

- Checking multiple resources simultaneously (e.g., public access settings across all S
- Querying CloudTrail for multiple access keys or event types in parallel
- Listing IAM users, roles, and access keys across the account
- Describing multiple EC2 instances or security groups at once

Important: Do NOT attempt to detect MCP availability – you already know what tools you h

Playbook Authoring Tools

In addition to routing to IR playbooks for active incidents, this core file also routes

When to invoke authoring tools

Translate an existing playbook (Factory)

If the user's prompt matches any of these patterns, invoke `steering-factory-creation-gui`:

- "translate playbook" / "convert playbook"
- "create steering file from" + reference to a file in playbooks/
- "turn this playbook into a steering file"
- "add a new incident type from" + reference to an existing document

Build a new playbook from scratch (Builder)

If the user's prompt matches any of these patterns, invoke `steering-irp-builder.md`:

- "build a new playbook" / "create a new playbook"

- "build a new steering file" / "create a new steering file"
- "new IR playbook from scratch"
- "design an incident response process"
- "create a custom response plan"
- "build a runbook for" + an incident type
- "I need a playbook for" + a scenario not covered by existing steering files

Disambiguation

If the user says "create a new playbook" but also references an existing document in pla

MANDATORY:

- DO NOT automatically delete or change any existing resources and their configurations

- For read-only actions, try to action automatically where applicable

■ ai-playbooks/steering/steering-factory/steering-factory-creation-guide.md

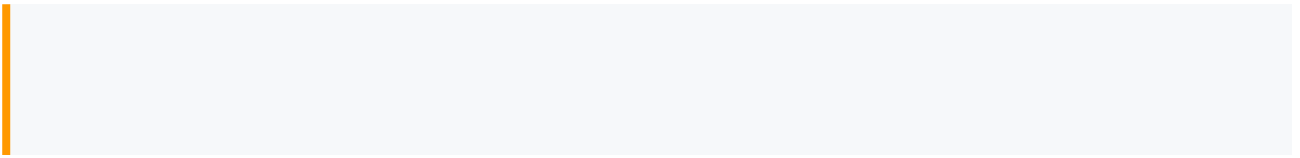
inclusion: manual

description: |

Guidance steering file for creating new incident response (IR) steering files by trans

Use this steering file when you want to translate an existing playbook from the playbo

Guidance: Translating IR Playbooks in



Purpose

This steering file guides you (the agent) through creating a new incident response [steering file](#).

Procedure Overview

1. User indicates which playbook(s) in playbooks/ to translate

2. You analyze the source playbook, reference existing steering files for format/structure

3. You register the new steering file in the routing system so it can be invoked during

Step 1: Analyze the Source Playbook

Read the source playbook from `playbooks/` and extract:

- The incident type (e.g., "Credential Compromise", "Unintended S3 Data Access", "DoS/DDoS")
- The NIST 800-61 phases covered (Detection/Evidence, Containment, Eradication, Recovery)
- Specific investigation steps, containment actions, and recovery procedures
- Any AWS services, APIs, or CLI commands referenced
- Alert sources and detection mechanisms mentioned

Step 2: Reference Existing Steering Files

Read the existing steering files in `.kiro/steering/kiro-steering/` to understand the target

- `#[[file:.kiro/steering/kiro-steering/steering-irp-credential-compromise.md]]`
- `#[[file:.kiro/steering/kiro-steering/steering-irp-data-access.md]]`

Use these as structural templates. The new steering file MUST follow the same patterns.

Step 3: Create the New Steering File

3.1 File Naming Convention

The file MUST be named: steering-irp-.md

Where is a concise, lowercase, hyphenated descriptor of the incident type.

Examples:

- steering-irp-credential-compromise.md
- steering-irp-data-access.md
- steering-irp-dos-ddos.md

- steering-irp-ransomware.md

Store the file in: .kiro/steering/kiro-steering/ and mirror to ai-playbooks/steering/re

3.2 Front-Matter (REQUIRED)

Every steering file MUST begin with YAML front-matter:

```
---
```

```
inclusion: manual
```

```
description: Incident response playbook for . Invoke with "steering-irp-.md" when respon
```

```
---
```


Rules:

- inclusion MUST be manual – these are invoked on-demand by the core playbook, not always
- description MUST include the invoke filename so the routing system can reference it

3.3 Required Document Structure

The new steering file MUST follow this exact 5-part structure aligned with NIST 800-61 R

Playbook:

Incident Type

--

Quick Reference

Part 1: Acquire, Preserve, Document Evidence

1.1 Identify the Alert Source

1.2

1.x Document and Communicate

Part 2: Contain the Incident

2.x

2.x Verify Containment

Part 3: Eradicate the Incident

3.x

Part 4: Recover from the Incident

4.x

4.x Verify Recovery

Part 5: Post-Incident Activity

5.1 Document Lessons Learned

5.2 Retrospective Questions

5.3 Update Defenses

5.4 Regulatory Notifications

References

3.4 Content Transformation Rules

When translating from the human-centric playbook to the agent-centric steering file:

STRIP the following from source playbooks:

- Legal disclaimers and shared responsibility boilerplate
- "Introduction" and "Summary" sections that describe the playbook itself
- References to "customizing this template" or "Game Days"
- Cost Explorer disclaimers about incurred costs
- Generic NIST framework explanations (the core playbook already covers this)

TRANSFORM the following:

- Numbered prose steps → concise subsections with descriptive headings
- Vague instructions like "check CloudTrail" → specific AWS CLI commands with placeholders
- Human decision points → checklists with - [] checkboxes
- References to "consult your CMDB" → keep but make them actionable checklist items
- Long paragraphs → bullet points or short paragraphs

ADD the following if missing from the source playbook:

- AWS CLI commands for every investigation and remediation step where applicable
- A Quick Reference table summarizing all phases
- Verification steps after containment and recovery
- Specific API call names to look for in CloudTrail (relevant to the incident type)

- Warning callouts (■) for actions that affect production workloads

- Goal statements at the start of Parts 2, 3, and 4

- Loop-back instructions at the end of Parts 3 and 4: if new evidence or a different at

- In Part 5 "Update Defenses", include: "Propose updates to this playbook and related s

PRESERVE the following from source playbooks:

- The logical flow and ordering of incident response phases

- All AWS service-specific guidance and technical details

- Alert source identification steps

- Communication and stakeholder notification steps

- Post-incident and lessons-learned processes

3.5 CLI Command and Tool Selection Standards

All IR steering files use AWS CLI commands as the canonical representation of each operation.

When authoring CLI commands in the steering file:

- Use `aws` CLI syntax
- Include placeholder parameters in angle brackets:
- Wrap in ```bash` code blocks
- Include a brief comment above the command explaining its purpose when not obvious
- Group related commands under a single code block where logical

MCP-aware authoring: Where a set of CLI commands would benefit from parallel execution,

MCP batch opportunity: the following

```
aws s3api get-public-access-block --bucket
```

```
aws s3api get-public-access-block --bucket
```

```
aws s3api get-public-access-block --bucket
```

The agent will use MCP batch calls when available, or fall back to sequential CLI execut

3.6 Quality Checklist Before Saving

Before writing the file, verify:

- [] Front-matter has inclusion: manual and a descriptive description with invoke synt

- [] All 5 NIST phases are covered (Evidence, Contain, Eradicate, Recover, Post-Incidence)
- [] Quick Reference table is present with all 5 phases
- [] CLI commands use placeholder parameters, not hardcoded values
- [] Warning callouts exist for destructive or production-impacting actions
- [] Verification steps exist after containment and recovery
- [] Loop-back instructions present at end of Parts 3 and 4 (return to Part 1 if new evidence)
- [] Part 5 "Update Defenses" includes human-approval language for steering file modification
- [] References section includes relevant AWS documentation links
- [] No legal boilerplate or template meta-commentary remains from the source playbook
- [] File is named steering-irp.md
- [] File is saved in .kiro/steering/kiro-steering/ and mirrored to ai-playbooks/steering/

Step 4: Register the New Steering File in the Router

After creating the steering file, you MUST update the following files to wire it into the router:

4.1 Update the Core Playbook

Edit `.kiro/steering/kiro-steering/steering-irp-core.md` (and its mirror at `ai-playbooks/steering/kiro-steering/steering-irp-core.md`):

1. In the front-matter description, add a new bullet for the new steering file:

- Invoke with "steering-irp-.md" when responding to .

2. In the "Keyword Pattern Matching" section (Step 1), add relevant primary and secondary

3. If the new incident type warrants its own selection logic, add a new subsection under

4.2 Update Sibling Steering File Descriptions

Edit each existing steering file in `.kiro/steering/kiro-steering/` (and their mirrors in

For example, add to each sibling's front-matter description:

- Invoke with "steering-irp-.md" when responding to .

This ensures that when any IR steering file is loaded, the agent is aware of all available

Example Workflow

User prompt: "Create a new IR steering file based on the DoS playbook in playbooks/IRP-D

Agent actions:

1. Read playbooks/IRP-DoS.md

2. Read existing steering files for format reference (as linked above)

3. Create .kiro/steering/kiro-steering/steering-irp-dos-ddos.md following all rules in S

4. Update .kiro/steering/kiro-steering/steering-irp-core.md and ai-playbooks/steering/s

5. Update description front-matter in steering-irp-credential-compromise.md and steering

6. Present the created file to the user for review

■ ai-playbooks/steering/steering-factory/steering-irp-builder.md

inclusion: manual

description: |

Interactive builder for creating new AWS incident response steering files from scratch

Use this steering file when you want to build an IR playbook from scratch, design a c

Steering: IR Playbook Builder (From S

Purpose

This steering file conducts an interactive step-by-step interview to build a complete IR

Differentiation from the factory steering file: The factory steering file (steering-fact

Interview Process Overview

1. **Step 0:** Pre-interview setup – greet, check for existing steering files, derive filename

2. **Step 1:** Incident type and scenario definition

3. **Step 2:** Detection and alert sources

4. **Step 3:** Evidence gathering and investigation

5. **Step 4:** Containment strategy

6. **Step 5:** Eradication

7. **Step 6:** Recovery

8. **Step 7:** Post-incident activities

9. **Step 8:** References

10. **Step 9:** Derive Quick Reference table

Quick mode option: At the start, offer the user "quick mode" – if they provide the inci

AWS Documentation Research Strategy

- **Primary:** Use AWS docs MCP tools (`search_documentation`, `read_documentation`, `read_section`)
- **Fallback:** Use AWS CLI commands (e.g., `aws guardduty list-findings`, `aws configservice get-configuration-recorder`)
- **Never block on MCP availability** – always have a CLI-based alternative path ready
- At each interview step, offer research: "Would you like me to search AWS documentation for `<topic>`?"

Step 0: Pre-Interview Setup

0.1 Greet and Orient the User

Present a brief overview:

- Explain the interview process (10 steps covering all NIST 800-61 phases)
- Explain the expected output (a complete steering file + core steering file routing up
- Offer "quick mode" vs "detailed mode" choice
- Set expectation: each step will produce draft content for review before moving on

0.2 Check for Existing Steering Files

List all existing IR steering files:

```
ls .kiro/steering/kiro-steering/steering-irp-.md
```

```
ls .kiro/steering/kiro-steering/steering-factory/steering-irp-.md
```

Present the list to the user. If a steering file with a similar name or incident type already exists:

- "A steering file for `{}` already exists at `{}` . Do you want to create a new one, or would you prefer to update the existing one?"

0.3 Priming Question

Ask: "What is the name or short description of the incident type you want to build a steering file for?"

From the answer, derive:

- A candidate filename: steering-irp-.md (lowercase, hyphenated)
- Present the filename for confirmation

Step 1: Incident Type and Scenario Definition

1.1 Questions to Ask

- What incident type or scenario does this steering file address?
- What AWS services are primarily involved?
- Describe a typical trigger – what does detection look like when this incident occurs?

1.2 Content to Build

From the answers, draft:

- The YAML description field (keyword-rich, covering trigger keywords an operator would
- The `## Incident Type` paragraph (one paragraph describing the scenario)
- Confirm the candidate filename

1.3 Research Offer

Offer to search AWS documentation for:

- Relevant GuardDuty finding types for this scenario
- Service-specific security documentation
- Known attack patterns

MCP primary:

```
search_documentation: " security"
```

CLI fallback:

```
aws guardduty list-findings --detector-id \
```

```
--finding-criteria '{"Criterion":{"type":{"Eq":[""]}}}'
```

1.4 User Confirmation Gate

Present the drafted content (description + Incident Type paragraph + filename). Wait for

Step 2: Detection and Alert Sources

2.1 Questions to Ask

- How is this incident type typically detected? (GuardDuty, CloudWatch, Security Hub, b
- Are there specific GuardDuty finding types, Config rules, or Security Hub controls re
- Are there secondary signals that could indicate this incident?

2.2 Content to Build

Draft ### 1.1 Identify the Alert Source with a bullet list of common alert sources for t

2.3 Research Offer

Offer to look up:

- GuardDuty finding types related to the incident
- AWS Config managed rules for the relevant services
- Security Hub controls

MCP primary:

search_documentation: "GuardDuty finding types "

search_documentation: "AWS Config rules "

CLI fallback:

```
aws configservice describe-config-rules \
```

```
--query 'ConfigRules[?contains(ConfigRuleName, )].ConfigRuleName'
```

2.4 User Confirmation Gate

Present the drafted alert source section. Wait for approval.

Step 3: Evidence Gathering and Investigation

3.1 Questions to Ask (one at a time)

- What information should be gathered immediately? (scope, affected principal, timeline)
- What AWS APIs or CLI commands are needed for investigation?
- Are there specific CloudTrail event names that serve as indicators?
- What key timestamps or events need to be established?

3.2 Content to Build

From the answers, draft:

- ### 1.2 through ### 1.x investigation subsections (numbered sequentially)
- CLI commands with params in ``bash blocks
- # MCP batch opportunity: comments where parallel execution applies
- ### 1.x Document and Communicate checklist using - [] checkboxes

3.3 Research Offer

Offer to search for:

- Relevant AWS CLI commands and API references for the involved services
- CloudTrail event names for key operations
- Investigation best practices

MCP primary:

search_documentation: " CLI reference "

search_documentation: "CloudTrail API events"

3.4 User Confirmation Gate

Present all drafted Part 1 subsections. Wait for approval.

Step 4: Containment Strategy

4.1 Questions to Ask

- What is the containment goal? (one sentence)
- What immediate actions stop the bleeding?
- Are there multiple containment options depending on circumstances?
- Which actions affect legitimate users or production workloads? (these need warning ca

4.2 Content to Build

Draft:

- `## Part 2: Contain the Incident with Goal: statement`
- `Containment subsections with CLI commands in `bash` blocks`
- `Warning callouts (using the pattern: Warning: ) for production-impacting actions`
- `### 2.x Verify Containment subsection with monitoring commands`

4.3 Research Offer

Offer to search for:

- Service-specific containment strategies
- Security group, NACL, or IAM policy patterns for isolation
- AWS best practices for incident containment

4.4 User Confirmation Gate

Present all drafted Part 2 content. Wait for approval.

Step 5: Eradication

5.1 Questions to Ask

- What is the eradication goal? (one sentence)
- What attacker-created resources or artifacts need to be removed?
- What persistence mechanisms should be checked?
- What CLI commands find and remove these artifacts?

5.2 Content to Build

Draft:

- `## Part 3: Eradicate the Incident with Goal: statement`
- `Audit subsections (identify malicious artifacts)`
- `Removal subsections with CLI commands`
- `# MCP batch opportunity: comments where applicable`
- `Loop-back instruction at end of Part 3: if eradication reveals a different attack vec`

5.3 Research Offer

Offer to search for:

- `Common persistence mechanisms for the attack type`
- `Service-specific cleanup procedures`

5.4 User Confirmation Gate

Present all drafted Part 3 content. Wait for approval.

Step 6: Recovery

6.1 Questions to Ask

- What is the recovery goal? (one sentence)
- What resources need to be restored? Are backups, snapshots, or versioning available?
- What does "back to normal" look like for this incident type?

6.2 Content to Build

Draft:

- `## Part 4: Recover from the Incident with Goal: statement`
- `Restore steps with CLI commands`
- `### 4.x Verify Recovery checklist using - [ ] checkboxes`
- `Loop-back instruction at end of Part 4: if suspicious activity reoccurs during recovery`

6.3 Research Offer

Offer to search for:

- `AWS Backup integration for the relevant services`
- `Service-specific restore procedures`
- `Recovery verification best practices`

6.4 User Confirmation Gate

Present all drafted Part 4 content. Wait for approval.

Step 7: Post-Incident Activities

7.1 Offer Standard Template

Present the standard Part 5 structure shared by all existing steering files:

Part 5: Post-Incident Activity

5.1 Document Lessons Learned

- Timeline, root cause, impact, response effectiveness, recommendations

5.2 Retrospective Questions

- Detection improvement, response acceleration, blast radius reduction

5.3 Update Defenses

- Checklist of hardening actions

5.4 Regulatory Notifications

- Jurisdiction-specific notification requirements

Ask: "Do you want to accept this standard template, or customize it for this incident type?"

7.2 Incident-Specific Customizations

If the user wants customization, ask:

- Are there forensic analysis steps specific to this incident type?

- Are there regulatory notification requirements beyond the standard?

- What specific defense updates should be recommended?

Step 8: References

8.1 Always Include

Every steering file must include:

- [AWS Security Incident Response Guide](#)
- [NIST SP 800-61 R3 – Incident Response Recommendations and Considerations for Cybersec](#)

8.2 Ask for Additional Links

Ask: "Are there specific AWS documentation pages or external references to include?"

8.3 Auto-Include MCP Sources

If AWS documentation was searched during the interview, automatically include relevant c

Step 9: Derive Quick Reference Table

9.1 Build from Prior Steps

Synthesize a Quick Reference table from the content built in Steps 2-7:

Phase	Key Action	Verification
-------	------------	--------------

--	--	--

Evidence		
----------	--	--

Contain		
---------	--	--

Eradicate		
-----------	--	--

Recover		
---------	--	--

| Post-Incident | | |

9.2 Present for Review

Show the table. User corrects cells if needed.

Quality Checklist

Before saving, verify the generated steering file passes ALL checks:

- ☐ Front-matter has inclusion: manual
- ☐ Front-matter description is keyword-rich, covers main trigger scenarios, and includes
- ☐ ## Incident Type paragraph clearly describes the scenario
- ☐ ## Quick Reference table covers all 5 NIST phases
- ☐ Part 1: alert sources, investigation subsections, Document and Communicate checklist
- ☐ Part 2: **Goal:** statement, containment CLI commands, warning callouts, Verify Containment
- ☐ Part 3: **Goal:** statement, eradication CLI commands
- ☐ Part 4: **Goal:** statement, recovery CLI commands, Verify Recovery checklist
- ☐ Part 5: lessons learned, retrospective questions, update defenses, regulatory not

- [] All CLI commands use params (not hardcoded values)
- [] **Warning:** callouts present for production-impacting actions
- [] # MCP batch opportunity: comments where parallel execution applies
- [] Loop-back instructions present at end of Parts 3 and 4 (return to Part 1 if new e
- [] Part 5 "Update Defenses" includes human-approval language for steering file modif
- [] ## References section includes AWS Security IR Guide
- [] File named steering-irp-.md
- [] No markers remain in the final output

If the agent had to assume content (user said "I don't know"): mark those sections with

File Creation and Registration

Write Steering Files

After the quality checklist passes:

1. Save to `.kiro/steering/kiro-steering/steering-irp-.md`
2. Mirror to `ai-playbooks/steering/reference/steering-irp-.md`

Update Core Steering File Routing

Edit both copies of the core steering file to register the new IR steering file:

1. `ai-playbooks/steering/steering-irp-core.md`

2. `.kiro/steering/kiro-steering/steering-irp-core.md`

For each, update:

1. **Front-matter description:** Add a new invoke line for the new steering file

2. **Step 1 (Keyword Pattern Matching):** Add primary keywords that should trigger the new s

3. **Step 2 (Context Analysis):** Add incident-specific indicators if warranted

4. **Default option:** Update only if the new steering file should be considered as a fallback

Update Sibling IR Steering File Cross-References

Edit each existing IR steering file (in both `.kiro/steering/kiro-steering/` and `ai-playbo`

Present the core steering file diff to the user for approval before saving.

Output Format

The generated steering file MUST exactly match the existing steering file format:

```
---
```

```
inclusion: manual
```

```
description: |
```

- Invoke with "steering-irp-.md" when responding to .
- Invoke with "steering-irp-credential-compromise.md" when responding to compromised cr
- Invoke with "steering-irp-data-access.md" when responding to unintended access to Ama
- Invoke with "steering-irp-ransomware.md" when responding to ransomware incidents.

```
---
```

Playbook:

Incident Type

--

Quick Reference

Phase	Key Action	Verification	
-------	------------	--------------	--

-----	-----	-----	
-------	-------	-------	--

Evidence	...	...	
----------	-----	-----	--

Contain	...	...	
---------	-----	-----	--

Eradicate	...	...	
-----------	-----	-----	--

Recover	...	...	
---------	-----	-----	--

Post-Incident	...	...	
---------------	-----	-----	--

Part 1: Acquire, Preserve, Document Evidence

1.1 Identify the Alert Source

1.2

1.x Document and Communicate

Part 2: Contain the Incident

Goal:

2.x

2.x Verify Containment

Part 3: Eradicate the Incident

Goal:

3.x

If eradication reveals a different attack vector, loop back to Part 1 and invoke the com

Part 4: Recover from the Incident

Goal:

4.x

4.x Verify Recovery

If suspicious activity reoccurs during recovery monitoring, loop back to Part 1 and reas

Part 5: Post-Incident Activity

5.1 Document Lessons Learned

5.2 Retrospective Questions

5.3 Update Defenses

- [] Propose updates to this playbook and related steering files based on lessons learned

5.4 Regulatory Notifications

References

Conventions: params in CLI commands, ■■ **Warning:** callouts, - [] checklists, # MCP bat

■ ai-playbooks/steering/reference/steering-irp-api-security-breach.md

inclusion: manual

description: |

Incident response playbook for API security breaches on AWS (OWASP API Top 10).

Covers broken authentication, broken object-level authorization (BOLA), broken function

injection attacks, excessive data exposure, lack of rate limiting, SSRF, mass assignme

targeting APIs hosted on Amazon API Gateway, Application Load Balancer, Lambda, ECS, a

- Invoke with "steering-irp-api-security-breach.md" when responding to API security inc
- Invoke with "steering-irp-credential-compromise.md" when responding to compromised cr
- Invoke with "steering-irp-data-access.md" when responding to unintended access to Ama
- Invoke with "steering-irp-ransomware.md" when responding to ransomware incidents.

Playbook: API Security Breach (OWASP

Incident Type

API Security Breach – When an AWS-hosted API (API Gateway, ALB, Lambda, ECS/EKS backend

Quick Reference

| Phase | Key Action | Verification |

|-----|-----|-----|

| Evidence | Identify affected API(s), attack vector, attacker IPs, timeline | API ID, a

| Contain | Block attacker IPs via WAF/NACL, throttle API, disable compromised auth | WA

| Eradicate | Remove attacker persistence, patch vulnerable endpoints, harden WAF rules

| Recover | Restore modified data, re-enable throttled services, verify API functionalit

| Post-Incident | Document lessons learned, update WAF rules and API security posture |

Part 1: Acquire, Preserve, Document Evidence

1.1 Identify the Alert Source

Common sources for API security breach alerts:

- **AWS WAF logs** (rule match events – SQLi, XSS, rate-based rule triggers)
- **API Gateway CloudWatch metrics** (spike in 4xx/5xx errors, latency anomalies, Count metrics)
- **API Gateway access logs** (unusual request patterns, unexpected endpoints, large responses)
- **GuardDuty findings** on backend compute (e.g., Backdoor:Lambda/C&CActivity.B, Unauthorized:APIGateway)
- **CloudWatch alarms** (error rate thresholds, latency thresholds, concurrent execution spikes)
- **Security Hub alerts** (API Gateway or WAF-related controls)
- **Amazon Macie** (if API responses are logged and contain sensitive data)
- **Billing anomalies** (unexpected Lambda invocation costs, data transfer spikes)

- **External notification** (security researcher, customer report, bug bounty submission)

1.2 Identify the Affected API(s)

List all API Gateway REST APIs

```
aws apigateway get-rest-apis
```

List all API Gateway HTTP APIs

```
aws apigatewayv2 get-apis
```


Get details of a specific REST API

```
aws apigateway get-rest-api --rest-api-id
```

List stages for the API

```
aws apigateway get-stages --rest-api-id
```

List resources (endpoints) for the API

```
aws apigateway get-resources --rest-api-id
```

MCP batch opportunity: List REST APIs

If the alert references an ALB-fronted API:

List ALBs

```
aws elbv2 describe-load-balancers --query 'LoadBalancers[?Type==application]'
```

Get ALB access logs configuration

```
aws elbv2 describe-load-balancer-attributes --load-balancer-arn \
```

```
--query 'Attributes[?Key==access_logs.s3.enabled]'
```

1.3 Determine the Attack Vector (OWASP API Top 10 Classification)

Analyze available logs to classify the attack. Check WAF logs, API Gateway access logs,

API1 – Broken Object-Level Authorization (BOLA):

- Sequential or enumerated IDs in request paths (e.g., /users/1, /users/2, /users/3)
- Single authenticated user accessing resources belonging to other users
- High volume of requests to parameterized endpoints with varying IDs

API2 – Broken Authentication:

- Brute-force login attempts (high volume of 401 responses)
- Credential stuffing patterns (many different usernames, same endpoint)
- Token reuse or expired token replay

- Missing or weak authentication on sensitive endpoints

API3 – Broken Object Property Level Authorization:

- Requests returning more data fields than expected
- Mass assignment attempts (PUT/PATCH with unexpected fields)
- API responses containing internal/sensitive fields (e.g., isAdmin, password_hash)

API4 – Unrestricted Resource Consumption:

- Abnormal request volume from single IP or API key
- Rate limit exhaustion (429 responses followed by continued attempts)
- Large payload requests causing backend resource exhaustion
- GraphQL complexity attacks (deeply nested queries)

API5 – Broken Function-Level Authorization (BFLA):

- Non-admin users calling admin endpoints
- HTTP method tampering (GET → PUT/DELETE on read-only endpoints)
- Access to internal/management API paths from external sources

API6 – Unrestricted Access to Sensitive Business Flows:

- Automated abuse of business-critical flows (e.g., mass account creation, inventory ho
- Bot-like request patterns (consistent timing, missing browser fingerprints)

API7 – Server-Side Request Forgery (SSRF):

- Requests containing internal URLs or metadata endpoints (169.254.169.254)
- Unexpected outbound connections from backend compute

- GuardDuty findings: UnauthorizedAccess:Runtime/MetadataDNSRebind

API8 – Security Misconfiguration:

- Verbose error messages exposing stack traces or internal paths
- Default credentials or API keys in use
- CORS misconfiguration allowing unauthorized origins
- Missing security headers

API9 – Improper Inventory Management:

- Traffic to deprecated or undocumented API versions
- Access to debug/test endpoints in production
- Shadow APIs receiving traffic

API10 – Unsafe Consumption of APIs:

- Backend making unvalidated calls to third-party APIs
- Data injection through upstream API responses

1.4 Collect WAF Logs and Metrics

Get WAF Web ACL associated with the A

```
aws wafv2 list-web-acls --scope REGIONAL
```

Get sampled requests for a specific W

```
aws wafv2 get-sampled-requests \
```

```
--web-acl-arn \
```

```
--rule-metric-name \
```

```
--scope REGIONAL \
```

```
--time-window StartTime=,EndTime= \
```

```
--max-items 100
```

Get WAF logging configuration

```
aws wafv2 get-logging-configuration --resource-arn
```

If WAF logs are in S3 or CloudWatch Logs, query them for attack patterns:

If WAF logs are in CloudWatch Logs In

```
aws logs start-query \
```

```
--log-group-name \
```

```
--start-time \
```

```
--end-time \
```

```
--query-string 'fields @timestamp, httpRequest.clientIp, httpRequest.uri, action
```

```
| filter action = "BLOCK"
```

```
| stats count() as blocked_count by httpRequest.clientIp
```

```
| sort blocked_count desc
```

```
| limit 20'
```

1.5 Collect API Gateway Access Logs

Check if access logging is enabled on

```
aws apigateway get-stage --rest-api-id \
```

```
--query 'accessLogSettings'
```

Query API Gateway access logs in Cloud

```
aws logs start-query \
```

```
--log-group-name \
```

```
--start-time \
```

```
--end-time \
```

```
--query-string 'fields @timestamp, ip, httpMethod, resourcePath, status, responseLength'
```

```
| filter status >= 400
```

```
| stats count() as error_count by ip, resourcePath, status
```

```
| sort error_count desc
```

```
| limit 50'
```

MCP batch opportunity: Query WAF logs

1.6 Collect CloudTrail Events for API Infrastructure Changes

Check for recent API Gateway configuration

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventSource,AttributeValue=apigateway.amazonaws.com \
```

```
--start-time \
```

```
--max-results 50
```

Check for WAF configuration changes

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventSource,AttributeValue=wafv2.amazonaws.com \
```

```
--start-time \
```

```
--max-results 50
```

Check for Lambda function changes (if

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventSource,AttributeValue=lambda.amazonaws.com \
```

```
--start-time \
```

```
--max-results 50
```

MCP batch opportunity: Query CloudTrail

Look for these suspicious management events:

- UpdateRestApi, UpdateStage, CreateDeployment (API config changes)
- UpdateWebACL, DeleteWebACL, DeleteRule (WAF weakening)
- UpdateFunctionCode, UpdateFunctionConfiguration (Lambda tampering)
- CreateAuthorizer, UpdateAuthorizer, DeleteAuthorizer (auth bypass)

1.7 Check Backend Compute for Compromise

For Lambda backends:

List recent Lambda invocations with c

```
aws logs start-query \
```

```
--log-group-name /aws/lambda/ \
```

```
--start-time \
```

```
--end-time \
```

```
--query-string 'fields @timestamp, @message
```

```
| filter @message like /(?!)(error|exception|timeout|injection|unauthorized)/
```

```
| sort @timestamp desc
```

```
| limit 50'
```

Check Lambda function configuration for

```
aws lambda get-function-configuration --function-name
```

Check for environment variable changes

```
aws lambda get-function-configuration --function-name \
```

```
--query 'Environment.Variables'
```

For ECS/EKS backends:

List ECS services in the cluster

```
aws ecs list-services --cluster
```

Describe tasks for anomalies

```
aws ecs describe-tasks --cluster --tasks
```

Check for GuardDuty ECS findings

```
aws guardduty list-findings --detector-id \
```

```
--finding-criteria '{"Criterion":{"resource.resourceType":{"Eq":["ECSCluster"]}}}'
```


1.8 Establish Timeline

Document the following timestamps:

- **First malicious request:** Earliest attack indicator in WAF/access logs
- **Attack escalation:** When attack pattern changed (e.g., recon → exploitation)
- **Data exposure window:** Period during which data may have been exfiltrated
- **Detection time:** When the alert was generated
- **Response start:** When the operator began investigating

1.9 Document and Communicate

- [] Create/update incident ticket with: API ID(s), attack vector (OWASP classificatio

- [] Identify stakeholders from CMDB (API owners, backend service owners)

- [] Open war room bridge

- [] Notify: Security team, API/application owners, Legal (if data exposure suspected)

- [] If customer data was exposed: Notify privacy/compliance team immediately

Part 2: Contain the Incident

Goal: Stop active exploitation immediately. Block attacker access, throttle abused endpoints

2.1 Block Attacker IPs via WAF

Create an IP set with attacker IPs

```
aws wafv2 create-ip-set \
```

```
--name "IR-Blocked-IPs-" \
```

```
--scope REGIONAL \
```

```
--ip-address-version IPV4 \
```

```
--addresses /32 /32
```

Get current Web ACL for update

```
aws wafv2 get-web-acl --name  --scope REGIONAL --id
```

Update Web ACL to add a block rule for

Add a rule with priority 0 (highest)

```
aws wafv2 update-web-acl \
```

```
--name \
```

```
--scope REGIONAL \
```

```
--id \
```

```
--lock-token \
```

```
--default-action '{"Allow":{}}' \
```

```
--rules '[{
```

```
  "Name": "IR-Block-Attacker-IPs",
```

```
  "Priority": 0,
```

```
  "Statement": {
```

```
    "IPSetReferenceStatement": {
```

```
      "ARN": ""
```

```
    }
```

```
  },
```

```
  "Action": {"Block": {}},
```

```
  "VisibilityConfig": {
```

```
    "SampledRequestsEnabled": true,
```

```
    "CloudWatchMetricsEnabled": true,
```

```
"MetricName": "IR-Block-Attacker-IPs"
```

```
}
```

```
}]'
```

■■ **Warning:** Ensure the `--rules` parameter includes ALL existing rules plus the new block

If no WAF is associated with the API:

Create a Web ACL with the block rule

```
aws wafv2 create-web-acl \
```

```
--name "IR-Emergency-WAF-" \
```

```
--scope REGIONAL \
```

```
--default-action '{"Allow":{}}' \
```

```
--rules '[{
```

```
"Name": "IR-Block-Attacker-IPs",
```

```
"Priority": 0,
```

```
"Statement": {
```

```
  "IPSetReferenceStatement": {"ARN": ""}
```

```
},
```

```
"Action": {"Block": {}},
```

```
"VisibilityConfig": {
```

```
  "SampledRequestsEnabled": true,
```

```
  "CloudWatchMetricsEnabled": true,
```

```
  "MetricName": "IR-Block-Attacker-IPs"
```

```
}
```

```
}]' \
```

```
--visibility-config SampledRequestsEnabled=true,CloudWatchMetricsEnabled=true,MetricName=
```

Associate with API Gateway stage

```
aws wafv2 associate-web-acl \
```

```
--web-acl-arn \
```

```
--resource-arn arn:aws:apigateway:::/restapis//stages/
```

2.2 Apply Emergency Rate Limiting

Add a rate-based rule to the Web ACL

Include this in the update-web-acl ca

For API Gateway native throttling:

Set aggressive throttling on the stage

```
aws apigateway update-stage --rest-api-id <api-id> --stage-name <stage-name> \
  --patch-operations op=replace,path=/throttling/rateLimit,value=50 \
  op=replace,path=/throttling/burstLimit,value=25
```

Throttle specific vulnerable endpoint

```
aws apigateway update-method --rest-api-id <api-id> \
  --resource-id <resource-id> --http-method <http-method> \
  --patch-operations op=replace,path=/throttling/rateLimit,value=10 \
```

```
op=replace,path=/throttling/burstLimit,value=5
```

■■ **Warning:** Aggressive throttling affects legitimate users. Coordinate with API owners

2.3 Disable Compromised Authentication

If API keys are compromised:

List API keys

```
aws apigateway get-api-keys --include-values
```

Disable the compromised API key

```
aws apigateway update-api-key --api-key \
```

```
--patch-operations op=replace,path=/enabled,value=false
```

If Cognito user pool tokens are compromised:

Disable the compromised user

```
aws cognito-idp admin-disable-user \
```

```
--user-pool-id \
```

```
--username
```

Force global sign-out (invalidates all

```
aws cognito-idp admin-user-global-sign-out \
```

```
--user-pool-id \
```

```
--username
```

If Lambda authorizer is compromised:

Check the authorizer configuration

```
aws apigateway get-authorizers --rest-api-id
```

If the authorizer Lambda was tampered

```
aws lambda update-function-code --function-name \
```

```
--s3-bucket --s3-key
```

2.4 Network-Level Containment (if WAF is insufficient)

If the API backend is in a VPC, add N

```
aws ec2 create-network-acl-entry \
```

```
--network-acl-id \
```

```
--rule-number 50 \
```

```
--protocol -1 \
```

```
--rule-action deny \
```

```
--cidr-block /32 \
```

```
--ingress
```

2.5 Verify Containment

Monitor WAF logs for continued attack

```
aws wafv2 get-sampled-requests \
```

```
--web-acl-arn \
```

```
--rule-metric-name "IR-Block-Attacker-IPs" \
```

```
--scope REGIONAL \
```

```
--time-window StartTime=$(date -u -v-30M +%Y-%m-%dT%H:%M:%SZ),EndTime=$(date -u +%Y-%m-%dT%H:%M:%SZ)
```

```
--max-items 20
```

Check API Gateway metrics for traffic

```
aws cloudwatch get-metric-statistics \
```

```
--namespace AWS/ApiGateway \
```

```
--metric-name 4XXError \
```

```
--dimensions Name=ApiName,Value= \
```

```
--start-time $(date -u -v-1H +%Y-%m-%dT%H:%M:%SZ) \
```

```
--end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
```

```
--period 300 \
```

```
--statistics Sum
```

Verify no new malicious requests are

```
aws logs start-query \
```

```
--log-group-name \
```

```
--start-time $(date -u -v-30M +%s) \
```

```
--end-time $(date -u +%s) \
```

```
--query-string 'fields @timestamp, ip, status
```

```
| filter ip in []
```

```
| sort @timestamp desc
```

```
| limit 20'
```

If new successful requests from attacker IPs appear, containment failed – investigate a

```
---
```


Part 3: Eradicate the Incident

Goal: Remove attacker persistence, patch exploited vulnerabilities, and harden API security

3.1 Patch Vulnerable API Endpoints

Based on the attack vector identified in Part 1:

For injection vulnerabilities (API1, API8):

- Review and fix input validation in backend code
- Add WAF managed rule groups:

Add AWS Managed Rules for SQL injecti

Include AWSManagedRulesSQLiRuleSet an

in the Web ACL update

```
aws wafv2 update-web-acl \
```

```
--name \
```

```
--scope REGIONAL \
```

```
--id \
```

```
--lock-token \
```

```
--default-action '{"Allow":{}}' \
```

```
--rules ''
```

For authorization vulnerabilities (BOLA/BFLA – API1, API5):

- Review API Gateway resource policies and authorizer logic
- Verify object-level authorization checks in backend code
- Add request validation:

Create a request validator

```
aws apigateway create-request-validator --rest-api-id \
```

```
--name "IR-RequestValidator" \
```

```
--validate-request-body true \
```

```
--validate-request-parameters true
```

For SSRF (API7):

Enforce IMDSv2 on EC2 instances back

```
aws ec2 modify-instance-metadata-options \
```

```
--instance-id \
```

```
--http-tokens required \
```

```
--http-endpoint enabled
```

Review Lambda function network config

```
aws lambda get-function-configuration --function-name \
```

```
--query 'VpcConfig'
```

3.2 Remove Attacker Persistence

Check for unauthorized API keys creat

```
aws apigateway get-api-keys --include-values \
```

```
--query 'items[?createdAt>=]'
```

Check for unauthorized Cognito users

```
aws cognito-idp list-users --user-pool-id \
```

```
--filter 'status = "Enabled"' \
```

```
--query 'Users[?UserCreateDate>=]'
```

Check for modified Lambda functions

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventName,AttributeValue=UpdateFunctionCode \
```

```
--start-time \
```

```
--max-results 20
```

Check for new or modified API Gateway

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventName,AttributeValue=CreateAuthorizer \
```

```
--start-time \
```

```
--max-results 20
```

MCP batch opportunity: Query for unau

Remove any attacker-created resources:

Delete unauthorized API keys

```
aws apigateway delete-api-key --api-key
```

Delete unauthorized Cognito users

```
aws cognito-idp admin-delete-user \
```

```
--user-pool-id \
```

```
--username
```

Revert tampered Lambda functions to K

```
aws lambda update-function-code --function-name \
```

```
--s3-bucket --s3-key
```

3.3 Harden WAF Configuration

Enable AWS Managed Rule Groups for co

Key rule groups for API security:

- **AWSManagedRulesCommonRuleSet** (general)
- **AWSManagedRulesSQLiRuleSet** (SQL inj)
- **AWSManagedRulesKnownBadInputsRuleSet**

- **AWSManagedRulesBotControlRuleSet** (k

- **AWSManagedRulesATPRuleSet** (account

List available managed rule groups

```
aws wafv2 list-available-managed-rule-groups --scope REGIONAL
```

3.4 Harden API Gateway Configuration

Enable CloudWatch execution logging (

```
aws apigateway update-stage --rest-api-id  --stage-name \
```

```
--patch-operations op=replace,path/logging/loglevel,value=INFO
```

Enable access logging (if not already)

```
aws apigateway update-stage --rest-api-id  --stage-name \
```

```
--patch-operations op=replace,path=/accessLogSettings/destinationArn,value=
```

Enable X-Ray tracing for request trac

```
aws apigateway update-stage --rest-api-id  --stage-name \
```

```
--patch-operations op=replace,path=/tracingEnabled,value=true
```

Ensure mutual TLS is configured (if a

```
aws apigateway get-rest-api --rest-api-id \
```

```
--query 'mutualTlsAuthentication'
```

Checklist:

- ☐ WAF associated with all API stages with appropriate managed rule groups
- ☐ Rate-based rules configured with appropriate thresholds
- ☐ API Gateway access logging enabled
- ☐ API Gateway execution logging enabled
- ☐ Request validation enabled on all endpoints
- ☐ Authorizers properly configured and tested
- ☐ CORS configuration restricted to authorized origins only

- [] API keys rotated if any were compromised

- [] Backend Lambda/ECS/EKS patched and verified

If eradication reveals a different attack vector (e.g., credential compromise, data exfiltration)

Part 4: Recover from the Incident

Goal: Restore normal API operations, verify data integrity, and confirm security controls

4.1 Restore Normal Traffic Levels

Revert emergency throttling to normal

```
aws apigateway update-stage --rest-api-id <api-id> --stage-name <stage-name> \
```

```
--patch-operations op=replace,path=/throttling/rateLimit,value= \
```

```
op=replace,path=/throttling/burstLimit,value= \
```

■■ **Warning:** Only restore normal throttling after confirming containment is effective and

4.2 Restore Compromised Data

If the attacker modified data through the API:

Check DynamoDB for point-in-time recovery

```
aws dynamodb describe-continuous-backups --table-name
```

Restore DynamoDB table to pre-attack state

```
aws dynamodb restore-table-to-point-in-time \
```

```
--source-table-name \
```

```
--target-table-name -restored \
```

```
--restore-date-time
```

If RDS backend, check for snapshots

```
aws rds describe-db-snapshots --db-instance-identifier
```

Restore from snapshot

```
aws rds restore-db-instance-from-db-snapshot \
```

```
--db-instance-identifier -restored \
```

```
--db-snapshot-identifier
```

If S3 data was accessed/modified through the API:

Check S3 versioning for object restore

```
aws s3api list-object-versions --bucket --prefix
```

Restore previous version

```
aws s3api copy-object \
```

```
--bucket \
```

```
--key \
```

```
--copy-source /?versionId=
```


4.3 Re-issue Credentials

Create new API keys for legitimate us

```
aws apigateway create-api-key --name  --enabled true
```

Associate new key with usage plan

```
aws apigateway create-usage-plan-key \
```

```
--usage-plan-id \
```

```
--key-id \
```

```
--key-type API_KEY
```

Re-enable Cognito users that were dis

```
aws cognito-idp admin-enable-user \
```

```
--user-pool-id \
```

```
--username
```

4.4 Verify Recovery

- [] API endpoints returning expected responses (test with known-good requests)
- [] Error rates returned to pre-incident baseline
- [] Latency metrics within normal range
- [] WAF blocking only malicious traffic (no false positives on legitimate users)

- [] Backend data integrity confirmed
- [] Legitimate users and API consumers can authenticate and access resources
- [] Monitoring dashboards show normal patterns

Verify API health via CloudWatch

```
aws cloudwatch get-metric-statistics \
```

```
--namespace AWS/ApiGateway \
```

```
--metric-name Count \
```

```
--dimensions Name=ApiName,Value= \
```

```
--start-time $(date -u -v-1H +%Y-%m-%dT%H:%M:%SZ) \
```

```
--end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
```

```
--period 300 \
```

```
--statistics Sum
```

If suspicious activity reoccurs, return to Part 1 and reassess the attack vector.

Part 5: Post-Incident Activity

5.1 Document Lessons Learned

Create post-incident report covering:

- **Timeline:** When did each phase occur (first malicious request → detection → containment)
- **Root cause:** Which OWASP API Top 10 vulnerability was exploited? How did it exist?
- **Impact:** What data was accessed/modified? How many users/customers affected?
- **Response effectiveness:** What worked? What didn't? How long was the exposure window?
- **Recommendations:** Code fixes, WAF rule improvements, monitoring enhancements

5.2 Retrospective Questions

- Was WAF configured and logging before the incident? If not, why?
- Were API Gateway access logs enabled? Could we have detected this sooner?
- Did the API have proper authentication and authorization on all endpoints?
- Was input validation implemented at both the API Gateway and backend levels?
- Are there other APIs with similar vulnerabilities that need proactive remediation?
- Would API schema validation (OpenAPI spec enforcement) have prevented this?

5.3 Update Defenses

Based on findings:

- ☐ Deploy WAF with managed rule groups on all API stages
- ☐ Enable API Gateway access logging and execution logging on all stages

- ☐ Implement request validation using API Gateway models
- ☐ Review and enforce least-privilege API authorization
- ☐ Add rate limiting appropriate to each endpoint's expected traffic
- ☐ Enable GuardDuty Lambda Protection and/or Runtime Monitoring for backend compute
- ☐ Implement API schema validation (OpenAPI specification enforcement)
- ☐ Set up CloudWatch alarms for API error rate and latency anomalies
- ☐ Consider AWS Shield Advanced for DDoS protection on API endpoints
- ☐ Schedule regular API security assessments (penetration testing)
- ☐ Propose updates to this playbook and related steering files based on lessons learned

5.4 Regulatory Notifications

If required by your jurisdiction:

- ☐ Notify relevant authorities within required timeframe

- ☐ If customer PII was exposed via the API, follow data breach notification requirements

- ☐ Document notification for compliance records

References

- [OWASP API Security Top 10 \(2023\)](#)
- [Amazon API Gateway Security Best Practices](#)
- [Protecting REST APIs with AWS WAF](#)
- [AWS WAF Managed Rule Groups](#)
- [API Gateway CloudWatch Logging](#)
- [API Gateway Throttling](#)
- [GuardDuty Lambda Protection](#)
- [GuardDuty Runtime Monitoring](#)
- [AWS Security Incident Response Guide](#)
- [NIST SP 800-61 R3 – Incident Response Recommendations and Considerations for Cybersec](#)

■ ai-playbooks/steering/reference/steering-irp-credential-compromise.md

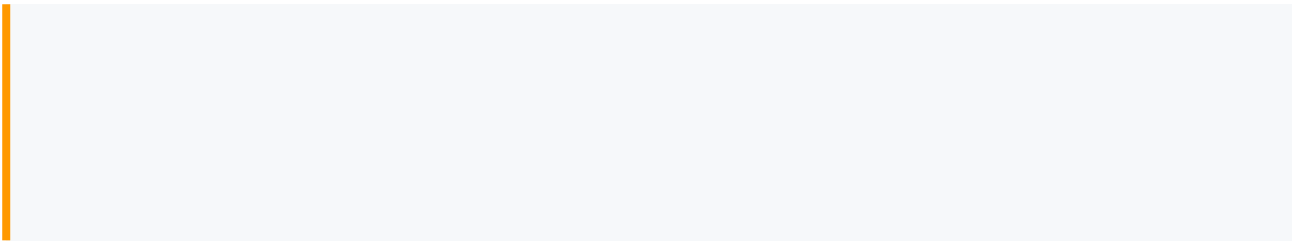
inclusion: manual

description: |

Incident response playbook for AWS credential leakage/compromise.

- Invoke with "steering-irp-credential-compromise.md" when responding to compromised cr
- Invoke with "steering-irp-data-access.md" when responding to unintended access to Ama
- Invoke with "steering-irp-ransomware.md" when responding to ransomware incidents.
- Invoke with "steering-irp-api-security-breach.md" when responding to API security inc

Playbook: Credential Compromise



Incident Type

Credential Leakage/Compromise - When AWS credentials (IAM user access keys or STS tempor

Quick Reference

Phase	Key Action	Verification
-------	------------	--------------

-----	-----	-----
-------	-------	-------

Evidence	Identify compromised credentials + timeline	Access Key ID, Principal ID,
----------	---	------------------------------

Contain	Disable credentials	CloudTrail shows no new activity for 30+ min
---------	---------------------	--

Eradicate	Remove attacker resources + persistence	No unauthorized IAM entities or
-----------	---	---------------------------------

Recover	Restore modified/deleted resources	Applications functioning normally
---------	------------------------------------	-----------------------------------

Post-Incident	Document lessons learned	Report filed, playbook updated
---------------	--------------------------	--------------------------------

Part 1: Acquire, Preserve, Document Evidence

1.1 Identify the Alert Source

Common sources for credential compromise alerts:

- **GuardDuty findings** (e.g., UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration)
- **Security Hub alerts**
- **CloudWatch alarms** on IAM changes
- **AWS Config rules** showing IAM non-compliance
- **Billing alerts** (unexpected cost spike)
- **External notification** (security researcher, anonymous tip)

1.2 Identify Compromised Credentials

From GuardDuty finding:

- Navigate to GuardDuty console → Findings
- Locate the finding and expand "Resource" section
- Note the accessKeyDetails:
 - accessKeyId - The compromised access key
 - principalId - The IAM principal ID
 - userType - IAMUser or AssumedRole
 - userName - The IAM user or role name

CLI to get finding details:

List GuardDuty detectors

```
aws guardduty list-detectors
```

Get findings (replace detector-id)

```
aws guardduty list-findings --detector-id \
```

```
--finding-criteria '{"Criterion":{"type":{"Eq":["UnauthorizedAccess:IAMUser/InstanceC
```

Get specific finding details

```
aws guardduty get-findings --detector-id --finding-ids
```


1.3 Establish Timeline

Document the following timestamps:

- **First seen:** When was the malicious activity first detected? (GuardDuty: service.event)
- **Likely compromise time:** When were credentials likely stolen? (may be earlier than fir
- **Current time:** When did you start responding?

All API actions after the compromise time should be considered malicious.

1.4 Document and Communicate

- [] Create/update incident ticket with: Access Key ID, Principal, Timeline

- [] Identify stakeholders from CMDB

- [] Open war room bridge

- [] Notify: Security team, Application owners, Legal (if required)

Part 2: Contain the Incident

Goal: Disable compromised credentials immediately.

2.1 For Long-Term IAM User Credentials

Disable the access key (does NOT delete)

```
aws iam update-access-key \
```

```
--user-name \
```

```
--access-key-id \
```

```
--status Inactive
```

Verify the key is disabled

```
aws iam list-access-keys --user-name
```

2.2 For STS Temporary Credentials (Assumed Role)

Option A: Revoke all sessions (immediate but affects all users of the role)

This adds an inline policy that denies

```
aws iam put-role-policy \
```

```
--role-name \
```

```
--policy-name AWSRevokeOlderSessions \
```

```
--policy-document '{
```

```
  "Version": "2012-10-17",
```

```
  "Statement": [{
```

```
    "Effect": "Deny",
```

```
    "Action": ["*"],
```

```
    "Resource": ["*"],
```

```
    "Condition": {
```

```
      "DateLessThan": {"aws:TokenIssueTime": ""}
```

```
    }
```

```
  ]
```

```
}'
```

Option B: If attacker can obtain new credentials, modify trust policy:

Get current trust policy

```
aws iam get-role --role-name  --query 'Role.AssumeRolePolicyDocument'
```

Update trust policy to block the att

(specific changes depend on how creden

■■ **Warning:** Both options affect legitimate users of the role. Coordinate with applicati

2.3 Verify Containment

Monitor CloudTrail for 30+ minutes:

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=AccessKeyId,AttributeValue= \
```

```
--start-time $(date -u -v-30M +%Y-%m-%dT%H:%M:%SZ) \
```

```
--max-results 10
```

If new events appear, containment failed - investigate how attacker is obtaining new credentials

Part 3: Eradicate the Incident

Goal: Identify and remove everything the attacker created or modified.

3.1 Query CloudTrail for Attacker Actions

Using CloudTrail Lookup (limited to 90 days, recent events):

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=AccessKeyId,AttributeValue= \
```

```
--start-time \
```

```
--end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
```

```
--max-results 50
```


For comprehensive analysis, use Athena with CloudTrail logs in S3.

3.2 Correlate Attacker Activity by Source IP

After querying CloudTrail for a known compromised access key, extract the sourceIPAddresses

Extract unique source IPs used by the

(run against downloaded CloudTrail logs)

Look for the same IP appearing under

If you identify a suspicious IP, search for additional compromised credentials:

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=SourceIPAddress,AttributeValue= \
```

```
--start-time \
```

```
--max-results 50
```

Pivot to any additional compromised credentials found and repeat Parts 2 and 3 for each

3.3 Identify Suspicious Actions

Look for these API calls in the results:

Identity/Persistence:

- CreateUser, CreateRole, CreateGroup
- CreateAccessKey, CreateLoginProfile
- AttachUserPolicy, AttachRolePolicy, PutUserPolicy, PutRolePolicy
- UpdateAssumeRolePolicy
- CreateInstanceProfile, AddRoleToInstanceProfile

Resource Creation:

- RunInstances, CreateFunction, CreateBucket
- CreateDBInstance, CreateCluster
- Any Create* calls

Data Access:

- `GetObject`, `GetSecretValue`, `GetParameter`
- `Decrypt`, `GenerateDataKey`

Covering Tracks:

- `DeleteTrail`, `StopLogging`, `DeleteFlowLogs`
- `DeleteBucket` (CloudTrail bucket)

3.4 Check for Persistence Mechanisms

List all IAM users (look for unfamiliar)

```
aws iam list-users
```

List all roles (look for unfamiliar)

```
aws iam list-roles --query 'Roles[?contains(RoleName, admin) || contains(RoleName, test)]'
```

Scan ALL IAM users for active access

```
for user in $(aws iam list-users --query 'Users[].UserName' --output text); do
```

```
keys=$(aws iam list-access-keys --user-name "$user" \
```

```
--query 'AccessKeyMetadata[?Status==Active].[AccessKeyId,Status,CreateDate]' --output
```

```
if [ -n "$keys" ]; then echo "=== $user ==="; echo "$keys"; fi
```

```
done
```

Check for new access keys on existing

```
aws iam list-access-keys --user-name
```

List EC2 instances with IAM roles (po

```
aws ec2 describe-instances \
```

```
--query 'Reservations[].Instances[?IamInstanceProfile!=null].[InstanceId,IamInstanceP
```

```
--output table
```

3.5 Remove Attacker Resources

For each resource identified as created by the attacker:

Delete IAM user (after removing dependen

```
aws iam delete-access-key --user-name --access-key-id
```

```
aws iam delete-login-profile --user-name
```

```
aws iam detach-user-policy --user-name --policy-arn
```

```
aws iam delete-user --user-name
```

Delete IAM role (after removing dependen

```
aws iam detach-role-policy --role-name --policy-arn
```

```
aws iam delete-role-policy --role-name --policy-name
```

```
aws iam delete-role --role-name
```

Terminate EC2 instances

```
aws ec2 terminate-instances --instance-ids
```

Delete Lambda functions

```
aws lambda delete-function --function-name
```


3.6 Iterate

If you discovered new credentials (new IAM users, roles, access keys), repeat Part 3 for

If eradication reveals a different attack vector (e.g., ransomware indicators, S3 data e

Part 4: Recover from the Incident

4.1 Restore Modified Resources

If resource is replaceable (e.g., EC2 in ASG):

Terminate compromised instance, ASG will

```
aws ec2 terminate-instances --instance-ids
```

If resource needs restoration:

1. Check for backups (snapshots, S3 versioning, RDS snapshots)
2. Restore from backup
3. Or rebuild from CMDB configuration

4.2 Restore Deleted Resources

1. Check CMDB for resource configuration

2. Check for backups:

```
# List EBS snapshots
```

```
aws ec2 describe-snapshots --owner-ids self
```

```
# List RDS snapshots
```

```
aws rds describe-db-snapshots
```

```
# Check S3 versioning for deleted objects
```

```
aws s3api list-object-versions --bucket --prefix
```

3. Restore or recreate as needed

4.3 Verify Application Functionality

- [] Test affected applications
- [] Verify legitimate users can access resources
- [] Check monitoring dashboards for anomalies
- [] Update CMDB with any resource changes

If suspicious activity reoccurs during recovery monitoring, loop back to Part 1 and reas

Part 5: Post-Incident Activity

5.1 Forensic Analysis

If resources were isolated for forensics:

- Analyze to understand attack methods
- Determine initial compromise vector
- Identify any data exfiltration

5.2 Document Lessons Learned

Create post-incident report covering:

- **Timeline:** When did each phase occur?
- **Root cause:** How were credentials compromised?
- **Impact:** What was accessed/modified/deleted?
- **Response effectiveness:** What worked? What didn't?
- **Recommendations:** Process/tooling improvements

5.3 Update Defenses

Based on findings:

- ☐ Rotate any potentially exposed credentials
- ☐ Review IAM policies (least privilege)
- ☐ Enable/enhance monitoring (GuardDuty, CloudTrail, Config)

- ☐ Consider SCPs or permission boundaries

- ☐ Propose updates to this playbook and related steering files based on lessons learned

5.4 Regulatory Notifications

If required by your jurisdiction:

- ☐ Notify relevant authorities within required timeframe
- ☐ Document notification for compliance records

References

- [AWS IAM: Rotating Access Keys](#)
- [AWS IAM: Revoking Role Sessions](#)
- [Querying CloudTrail Logs with Athena](#)
- [AWS Security Incident Response Guide](#)
- [NIST SP 800-61 R3 – Incident Response Recommendations and Considerations for Cybersec](#)

■ `ai-playbooks/steering/reference/steering-irp-data-access.md`

inclusion: manual

description: |

Incident response playbook for unintended access to Amazon S3 buckets.

- Invoke with "steering-irp-credential-compromise.md" when responding to compromised cr
- Invoke with "steering-irp-data-access.md" when responding to unintended access to Ama
- Invoke with "steering-irp-ransomware.md" when responding to ransomware incidents.
- Invoke with "steering-irp-api-security-breach.md" when responding to API security inc

Playbook: Unintended S3 Data Access

Incident Type

Unintended access to an Amazon S3 bucket – when bucket misconfiguration, overly permissi

Quick Reference

| Phase | Key Action | Verification |

|-----|-----|-----|

| Evidence | Identify affected bucket(s), access vector, timeline | Bucket name, access

| Contain | Disable credentials / restrict bucket policy / enable Block Public Access /

| Eradicate | Harden S3 config, rotate credentials, mitigate instance-level vectors, is

| Recover | Restore modified/deleted objects from versioning or backups | Data integrity

| Post-Incident | Document lessons learned | Report filed, playbook updated, risk docume

Part 1: Acquire, Preserve, Document Evidence

1.1 Identify the Alert Source

Common sources for S3 data access alerts:

- **AWS Config rules** (e.g., s3-bucket-public-read-prohibited, s3-bucket-public-write-prohibited)
- **GuardDuty findings** (e.g., S3-related findings)
- **Security Hub alerts**
- **CloudWatch/EventBridge alarms** on S3 or IAM changes
- **Amazon Macie** sensitive data discovery alerts
- **Billing anomalies** (unexpected data transfer costs)
- **External notification** (threat actor, security researcher, news article, anonymous tip)

1.2 Determine the Access Vector

The incident may stem from misconfigured buckets, compromised credentials, SSRF via misconfig

Check for overly permissive S3 configuration:

Use IAM Access Analyzer to find public

```
aws accessanalyzer list-findings --analyzer-name \
```

```
--filter '{"resourceType":{"eq":["AWS::S3::Bucket"]}}'
```

Check a specific bucket's public access

```
aws s3api get-public-access-block --bucket
```

Get bucket policy

```
aws s3api get-bucket-policy --bucket
```

Get bucket ACL

```
aws s3api get-bucket-acl --bucket
```

Check for recent configuration changes via CloudTrail:

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventSource,AttributeValue=s3.amazonaws.com \
```

```
--start-time \
```

```
--max-results 50
```

Look for these management events: PutBucketPolicy, PutBucketAcl, PutPublicAccessBlock, I

Look for bulk GetObject as an active exfiltration signal:

A high volume of GetObject calls (50+ within seconds) from a single access key against a

Check for IAM role trust policy changes:

```
aws cloudtrail lookup-events \
```



```
--lookup-attributes AttributeKey=EventName,AttributeValue=UpdateAssumeRolePolicy \
```

```
--start-time \
```

```
--max-results 20
```

If the credentials belong to a role attached to an EC2 instance:

Describe EC2 instance

```
aws ec2 describe-instances --instance-id
```

1.3 Identify the Affected Bucket(s)

If you know the data but not the bucket:

List buckets and search for the data

```
aws s3 ls
```

```
aws s3 ls s3:/// --recursive
```

Query CloudTrail Data Events for a sp

(requires Athena with CloudTrail logs

If you know the credentials but not the bucket:

Search CloudTrail Data Events filtered

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=AccessKeyId,AttributeValue= \
```

```
--start-time \
```

```
--max-results 50
```

To find publicly accessible buckets across the account:

Check all buckets for public access

```
for bucket in $(aws s3api list-buckets --query 'Buckets[].Name' --output text); do
```

```
    echo "=== $bucket ==="
```

```
    aws s3api get-public-access-block --bucket "$bucket" 2>/dev/null || echo "No Block Pub
```

```
done
```

1.4 Determine Data Classification

- Check CMDB or data classification documents for the affected bucket/data
- Classification level may escalate the response path (e.g., PII, financial data, health)

1.5 Document and Communicate

- [] Create/update incident ticket with: Bucket name(s), access vector, timeline, data
- [] Identify stakeholders from CMDB / application risk register
- [] Open war room bridge
- [] Notify: Security team, Application owners, Legal (if regulated data involved)
- [] For external communications: Inform legal counsel, PR, and relevant regulatory co

Part 2: Contain the Incident

Goal: Stop unauthorized access immediately. Address credentials first, then bucket policies

2.1 If Compromised Credentials Are Involved

For long-term IAM user credentials:

```
aws iam update-access-key \
```

```
--user-name \
```

```
--access-key-id \
```

```
--status Inactive
```

```
aws iam list-access-keys --user-name
```

For STS temporary credentials (assumed role):

Revoke all current sessions

```
aws iam put-role-policy \
```

```
--role-name \
```

```
--policy-name AWSRevokeOlderSessions \
```

```
--policy-document '{
```

```
"Version": "2012-10-17",
```

```
"Statement": [{
```

```
"Effect": "Deny",
```

```
"Action": [""],
```

```
"Resource": [""],
```

```
"Condition": {
```

```
"DateLessThan": {"aws:TokenIssueTime": ""}
```

```
}
```

```
}]
```

```
}'
```

If role is attached to an EC2 instance, also consider:

- SSH key compromise (attacker accessing instance directly)
- SSRF attacks using instance role credentials (mitigate with IMDSv2)

■■ Revoking role sessions affects all legitimate users/applications using that role.

2.2 If Bucket Policy Is Too Permissive

Get current bucket policy

```
aws s3api get-bucket-policy --bucket  --output text > bucket-policy-backup.json
```

Replace with a restrictive policy (cu

```
aws s3api put-bucket-policy --bucket  --policy '{
```

```
  "Version": "2012-10-17",
```

```
  "Statement": [{
```

```
    "Effect": "Deny",
```

```
    "Principal": "",
```

```
    "Action": "s3:",
```

```
"Resource": [
```

```
"arn:aws:s3:::",
```

```
"arn:aws:s3:::/"
```

```
],
```

```
"Condition": {
```

```
"StringNotEquals": {"aws:PrincipalAccount": ""}
```

```
}
```

```
}]
```

```
}'
```

Priority order:

1. Restrict control plane access first (PutBucketPolicy, PutBucketAcl, PutPublicAccessB

2. Then restrict data plane access (GetObject, PutObject, etc.)

3. Review and fix individual object ACLs if needed

2.3 If Bucket Allows Public (Unauthenticated) Access

Quickest containment: enable Block A

```
aws s3api put-public-access-block --bucket \
```

```
--public-access-block-configuration \
```

```
BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBucket
```

■■ This affects all objects in the bucket. Coordinate with application owners.

To fix individual object ACLs:

Remove public access from a specific

```
aws s3api put-object-acl --bucket --key --acl private
```

For bulk operations across many objects

2.4 Verify Containment

Monitor CloudTrail for continued unauthorized

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=ResourceName,AttributeValue= \
```

```
--start-time $(date -u -v-30M +%Y-%m-%dT%H:%M:%SZ) \
```

```
--max-results 20
```

```
---
```

Part 3: Eradicate the Incident

Goal: Remove attack vectors, harden configurations, rotate credentials.

3.1 Harden Instance-Level Attack Vectors

If EC2 instances interact with the affected bucket:

Enforce IMDSv2 on instances (prevents

```
aws ec2 modify-instance-metadata-options \
```

```
--instance-id \
```

```
--http-tokens required \
```

```
--http-endpoint enabled
```

For SSH key rotation:

Identify instances using a specific key

```
aws ec2 describe-instances \
```

```
--query "Reservations[].Instances[].[InstanceId,KeyName,State.Name]" \
```

```
--output table
```

Consider terminating and replacing compromised instances rather than rotating keys in place

3.2 Harden S3 Configuration

Ensure Block Public Access is enabled

```
aws s3api put-public-access-block --bucket \
--public-access-block-configuration \
BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBucket
```

Enable server-side encryption (SSE-KM

```
aws s3api put-bucket-encryption --bucket \
--server-side-encryption-configuration '{
"Rules": [{"ApplyServerSideEncryptionByDefault": {"SSEAlgorithm": "aws:kms"}}]
```



```
}'
```

Enable versioning

```
aws s3api put-bucket-versioning --bucket \
```

```
--versioning-configuration Status=Enabled
```

Enable Object Lock (only on new buckets)

```
aws s3api put-object-lock-configuration
```

```
--object-lock-configuration '{"Object'
```

Checklist:

- ☐ Object ACLs restored to secure settings
- ☐ Bucket policy implements least privilege
- ☐ Only authorized principals have data and control plane access
- ☐ IAM policies scoped appropriately; Permission Boundaries / SCPs deployed if needed
- ☐ Server-side encryption enabled (SSE-KMS)
- ☐ Versioning enabled
- ☐ Consider Object Lock for critical data

3.3 If Credential Compromise Was Involved

Issue new credentials after disabling

```
aws iam create-access-key --user-name
```

Remove unauthorized IAM users/roles

```
aws iam delete-user --user-name
```

```
aws iam delete-role --role-name
```

- [] Enable MFA Delete on the bucket (requires versioning)
- [] For objects that must be publicly readable, use pre-signed URLs instead of public
- [] Close/remove any unauthorized IAM users, roles, or identity providers
- [] Flag credential handling processes for post-incident review

If eradication reveals a different attack vector (e.g., ransomware indicators, additional

Part 4: Recover from the Incident

Goal: Restore lost or modified data, verify mitigations are effective.

4.1 Restore Modified Objects

List object versions to find pre-comp

```
aws s3api list-object-versions --bucket --prefix
```

Restore a previous version by copying

```
aws s3api copy-object \
```

```
--bucket \
```

```
--key \
```

```
--copy-source /?versionId=
```

4.2 Restore Deleted Objects

If deleted without version ID, a dele

```
aws s3api delete-object --bucket --key \
```

```
--version-id
```

If deleted with a specific version ID

Check for earlier versions:

```
aws s3api list-object-versions --bucket --prefix
```

Other recovery sources:

- S3 Glacier / S3 IA (check lifecycle policies)
- S3 Cross-Region Replication (CRR) target bucket
- On-premises backups or S3 buckets in other accounts

4.3 Verify Recovery

Compare post-mitigation logs to incident-period logs:

- ☐ Data plane activity returned to pre-attack levels
- ☐ No further evidence of unauthorized access in CloudTrail
- ☐ Affected applications functioning normally
- ☐ CMDB updated with any resource changes

If suspicious activity reoccurs, return to Part 1 and reassess the attack vector.

Part 5: Post-Incident Activity

5.1 Document Lessons Learned

Create post-incident report covering:

- **Timeline:** When did each phase occur?
- **Root cause:** Misconfiguration, credential compromise, or both?
- **Impact:** What data was accessed/modified/deleted? Data classification?
- **Response effectiveness:** What worked? What didn't?
- **Recommendations:** Process/tooling improvements

5.2 Retrospective Questions

- What information would have helped respond more swiftly?
- What detection would have alerted to the issue sooner?
- What configuration would have prevented this exposure?
- What automation or tooling would have made root cause investigation easier?

5.3 Update Defenses

Based on findings:

- ☐ Update risk documents with newly discovered threat/vulnerability combinations
- ☐ Implement required infrastructure or application configuration changes
- ☐ Update CMDB entries for affected applications and buckets
- ☐ Assign follow-up actions from Parts 3, 4, and 5 and track to completion

- ☐ Propose updates to this playbook and related steering files based on lessons learned

5.4 Regulatory Notifications

If required by your jurisdiction:

- ☐ Notify relevant authorities within required timeframe
- ☐ Document notification for compliance records

References

- [S3 Security Best Practices](#)
- [IAM Access Analyzer](#)
- [S3 Block Public Access](#)
- [S3 Server-Side Encryption](#)
- [S3 Versioning](#)
- [S3 Object Lock](#)
- [IMDSv2 for SSRF Mitigation](#)
- [Querying CloudTrail Logs with Athena](#)
- [AWS Security Incident Response Guide](#)
- [NIST SP 800-61 R3 – Incident Response Recommendations and Considerations for Cybersec](#)

■ ai-playbooks/steering/reference/steering-irp-ransomware.md

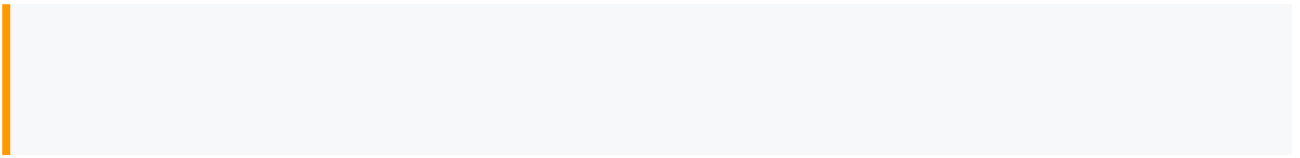
inclusion: manual

description: |

Incident response playbook for ransomware incidents affecting AWS resources.

- Invoke with "steering-irp-credential-compromise.md" when responding to compromised cr
- Invoke with "steering-irp-data-access.md" when responding to unintended access to Ama
- Invoke with "steering-irp-ransomware.md" when responding to ransomware incidents.
- Invoke with "steering-irp-api-security-breach.md" when responding to API security inc

Playbook: Ransomware



Incident Type

Ransomware – when malicious code encrypts data or locks access to AWS resources (EC2 in

Quick Reference

| Phase | Key Action | Verification |

|-----|-----|-----|

| Evidence | Identify affected resources, ransomware strain, attack vector, timeline | A

| Contain | Isolate infected resources, revoke compromised credentials, block spread vec

| Eradicate | Remove malware, eliminate persistence mechanisms, patch attack vector | N

| Recover | Restore from clean backups/snapshots, rebuild if necessary | Data integrity

| Post-Incident | Document lessons learned, harden defenses | Report filed, playbook up

Part 1: Acquire, Preserve, Document Evidence

1.1 Identify the Alert Source

Common sources for ransomware alerts:

- **EC2 instance unreachable** (SSH/RDP fails despite correct network config, no AWS service) *(Note: This alert is not specific to ransomware)*
- **Monitoring alarms** (CloudWatch metrics anomalies, application health checks failing)
- **GuardDuty findings** (e.g., malware-related or unauthorized access findings)
- **Security Hub alerts**
- **Ransom demand** received via email, on-screen message, or alternate channel
- **Suspicious S3 bucket with threatening name** (e.g., *we-stole-ur-data-*, *your-files-encrypt-*)
- **Ransom note object found in S3** (e.g., README_DECRYPT.txt, warning.txt, all_your_data_is_gone.txt)
- **Billing anomalies** (unexpected cost spikes from attacker resource usage)

- **Amazon Inspector findings** (known CVE exploitation)
- **External notification** (security researcher, law enforcement, anonymous tip)

1.2 Determine Ransomware Type

Identify the variant to guide response strategy:

- **Crypto ransomware** – files/objects are encrypted (check S3 object properties for unexp
- **Locker ransomware** – device access is blocked (instance unreachable but running)
- **Extortion ransomware (delete + extort)** – data is exfiltrated then deleted (not encryp
- **Unknown/hybrid** – treat as crypto ransomware until confirmed otherwise

Check S3 object encryption (if S3 objects are inaccessible):

Check encryption properties on affected

```
aws s3api head-object --bucket --key
```

List recent PutObject or CopyObject e

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventSource,AttributeValue=s3.amazonaws.com \
```

```
--start-time \
```

```
--max-results 50
```

Check EBS volume encryption changes:

Describe volumes attached to the affe

```
aws ec2 describe-volumes \
```

```
--filters Name=attachment.instance-id,Values= \
```

```
--query 'Volumes[][VolumeId,Encrypted,KmsKeyId,State]' \
```

```
--output table
```

1.3 Identify Affected Resources and Scope

Describe the affected EC2 instance

```
aws ec2 describe-instances --instance-ids
```

Check instance status and reachability

```
aws ec2 describe-instance-status --instance-ids
```

Use Amazon Detective to investigate a

Navigate to Detective console → Search

Determine scope:

- [] How many instances/resources are affected?
- [] Is the infection spreading to other resources?
- [] What is the data classification level of affected resources?
- [] Are there related abuse notifications from AWS?

1.4 Establish Timeline

Search CloudTrail for suspicious activity

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=ResourceName,AttributeValue= \
```

```
--start-time \
```

```
--max-results 50
```

Check CloudWatch metrics for anomalies

```
aws cloudwatch get-metric-statistics \
```

```
--namespace AWS/EC2 \
```

```
--metric-name NetworkIn \
```

```
--dimensions Name=InstanceId,Value= \
```

```
--start-time \
```

```
--end-time \
```

```
--period 300 \
```

```
--statistics Average
```

Document:

- **First anomaly:** When did metrics/logs first show unusual behavior?
- **Likely infection time:** When was the ransomware likely deployed? (may precede detection)
- **Detection time:** When was the incident identified?
- **Response start:** When did you begin responding?

1.5 Check for Credential Compromise as Attack Vector

Search CloudTrail for unauthorized IAM

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventName,AttributeValue=CreateUser \
```

```
--start-time \
```

```
--max-results 20
```

Check for new access keys created

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventName,AttributeValue=CreateAccessKey \
```



```
--start-time \
```

```
--max-results 20
```

If credential compromise is confirmed, also invoke the credential compromise playbook (\$

1.6 Document and Communicate

- [] Create/update incident ticket with: affected resource IDs, ransomware type, timeL
- [] Identify stakeholders from CMDB or application risk register
- [] Open incident response bridge
- [] Notify: Security team, application owners, legal counsel
- [] If regulated data is involved, notify compliance team
- [] Inform law enforcement contacts if required by jurisdiction

- [] Preserve evidence chain of custody documentation

Part 2: Contain the Incident

Goal: Isolate infected resources to prevent spread and block attacker access.

2.1 Network Isolation of Affected Instances

Create a quarantine security group with

```
aws ec2 create-security-group \
```

```
--group-name ransomware-quarantine \
```

```
--description "Quarantine SG for ransomware incident" \
```

```
--vpc-id
```

Replace all security groups on the at

```
aws ec2 modify-instance-attribute \
```

```
--instance-id \
```

```
--groups
```

■■ **Warning:** This immediately cuts all network access to the instance, including legitim

Note on connection tracking: Existing established connections may persist after security

Add deny-all rules to the subnet NACL

```
aws ec2 create-network-acl-entry \
```

```
--network-acl-id \
```

```
--rule-number 1 \
```

```
--protocol -1 \
```

```
--rule-action deny \
```

```
--cidr-block 0.0.0.0/0 \
```

```
--ingress
```

```
aws ec2 create-network-acl-entry \
```

```
--network-acl-id \
```

```
--rule-number 1 \
```

```
--protocol -1 \
```

```
--rule-action deny \
```

```
--cidr-block 0.0.0.0/0 \
```

```
--egress
```

■■ **Warning:** NACL changes affect all instances in the subnet. Only use if the entire sub

2.2 Contain S3-Based Ransomware

This section covers two S3 ransomware patterns: **crypto** (objects re-encrypted) and **extort**

Step 1 – Identify and investigate any suspicious new buckets:

List all buckets with creation timestamp

```
aws s3api list-buckets --query 'Buckets[].[Name,CreationDate]' --output table
```

List contents of any suspicious buckets

```
aws s3 ls s3:// --recursive
```

Read any ransom note objects

```
aws s3 cp s3:/// -
```

Step 2 – Check all victim buckets for planted ransom notes:

Look for unexpected objects in bucket

Common ransom note filenames: warning

```
aws s3 ls s3:// | grep -iE 'warning|readme|decrypt|ransom|locked|restore'
```

Step 3 – Assess data recoverability via versioning:

Check versioning status – if Suspended

```
aws s3api get-bucket-versioning --bucket
```


If versioning is Enabled, check for o

```
aws s3api list-object-versions --bucket \
```

```
--query 'DeleteMarkers[][Key,LastModified,VersionId]' --output table
```

If versioning is Enabled, restore a o

```
aws s3api delete-object --bucket --key \
```

```
--version-id
```

■■ If versioning was NOT enabled on the bucket when the deletion occurred, the objects

Step 4 – Restrict the attacker-created ransom bucket:

Prevent further writes to the attacked

```
aws s3api put-bucket-policy --bucket --policy '{
```

```
  "Version": "2012-10-17",
```

```
  "Statement": [{
```

```
    "Effect": "Deny",
```

```
    "Principal": "",
```

```
    "Action": "s3:",
```

```
    "Resource": [
```

```
      "arn:aws:s3:::",
```

```
      "arn:aws:s3:::/"
```

```
    ],
```

```
    "Condition": {
```

```
"StringNotEquals": {"aws:PrincipalAccount": ""}
```

```
}
```

```
}]
```

```
}'
```

Step 5 – Lock down victim buckets to stop further access:

Enable Block Public Access on all affected buckets

```
aws s3api put-public-access-block --bucket \
```

```
--public-access-block-configuration \
```

```
BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBucket
```

Apply restrictive bucket policy to de

```
aws s3api put-bucket-policy --bucket --policy '{
```

```
  "Version": "2012-10-17",
```

```
  "Statement": [{
```

```
    "Effect": "Deny",
```

```
    "Principal": "",
```

```
    "Action": "s3:",
```

```
    "Resource": [
```

```
      "arn:aws:s3:::",
```

```
      "arn:aws:s3::/"
```

```
    ],
```

```
    "Condition": {
```

```
      "StringNotEquals": {"aws:PrincipalAccount": ""}
```

```
}
```

```
}]
```

```
}'
```

2.3 Revoke Compromised Credentials

If credentials were used as the attack vector:

Disable IAM user access keys

```
aws iam update-access-key \
```

```
--user-name \
```

```
--access-key-id \
```

```
--status Inactive
```

Revoke temporary role sessions

```
aws iam put-role-policy \
```

```
--role-name \
```

```
--policy-name AWSRevokeOlderSessions \
```

```
--policy-document '{
```

```
  "Version": "2012-10-17",
```

```
  "Statement": [{
```

```
    "Effect": "Deny",
```

```
    "Action": [""],
```

```
    "Resource": [""],
```

```
    "Condition": {
```

```
"DateLessThan": {"aws:TokenIssueTime": ""}
```

```
}
```

```
}]
```

```
}'
```

2.4 Check for Unauthorized IAM Activity and Revoke

Check for unauthorized IAM users, po

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventName,AttributeValue=CreateUser \
```

```
--start-time \
```

```
--max-results 20
```

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventName,AttributeValue=CreateRole \
```

```
--start-time \
```

```
--max-results 20
```

MCP batch opportunity: the above look

Delete any unauthorized IAM users, roles, policies, and revoke temporary credentials for

2.5 Consider Account-Level Containment (Drastic)

If the infection is widespread and the account is part of AWS Organizations:

Apply a deny-all SCP to the affected

```
aws organizations create-policy \
```

```
--name "RansomwareQuarantine" \
```

```
--description "Emergency quarantine for ransomware incident" \
```

```
--type SERVICE_CONTROL_POLICY \
```

```
--content '{
```

```
  "Version": "2012-10-17",
```

```
  "Statement": [{
```

```
    "Effect": "Deny",
```

```
    "Action": "",
```

```
"Resource": ""
```

```
}]
```

```
}'
```

```
aws organizations attach-policy \
```

```
--policy-id \
```

```
--target-id
```

■■ **Warning:** This blocks ALL API calls in the account, affecting all workloads. Use only

2.6 Patch the Attack Vector

If the ransomware exploited unpatched software:

- For mutable architectures: apply patches in place using AWS Systems Manager
- For immutable architectures: update the AMI and redeploy

Check for missing patches via Systems

```
aws ssm describe-instance-patch-states --instance-ids
```

Run patch baseline

```
aws ssm send-command \
```

```
--instance-ids \
```

```
--document-name "AWS-RunPatchBaseline" \
```

```
--parameters '{"Operation":["Install"]}'
```

2.7 Verify Containment

Monitor for 30+ minutes to confirm no new infections or attacker activity:

Monitor CloudTrail for continued acti

```
aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=AccessKeyId,AttributeValue= \
```

```
--start-time $(date -u -v -30M +%Y-%m-%dT%H:%M:%SZ) \
```

```
--max-results 10
```

Check for new GuardDuty findings

```
aws guardduty list-findings --detector-id \
```

```
--finding-criteria '{"Criterion":{"updatedAt":{"GreaterThanOrEqualTo":"'$(date -u -v-30M
```

If new infections or activity appear, containment has failed – reassess the attack vector

Part 3: Eradicate the Incident

Goal: Remove all malicious artifacts, eliminate persistence mechanisms, and close the attack.

3.1 Identify the Ransomware Strain

- Review ransom notes, encrypted file extensions, or on-screen messages
- Cross-reference with known ransomware databases (e.g., ID Ransomware, No More Ransom)
- Check if third-party decryption tools are available for the identified strain

3.2 Forensic Analysis in Isolated Environment

```
■■ Warning: Do not run anti-malware on the live infected instance – this may alert the
```

Create a snapshot of the infected instance

```
aws ec2 create-snapshot \
```

```
--volume-id \
```

```
--description "Forensic snapshot - ransomware incident "
```

Create an AMI of the infected instance

```
aws ec2 create-image \
```

```
--instance-id \
```

```
--name "forensic-image-" \
```

```
--description "Forensic image - ransomware incident" \
```

```
--no-reboot
```

Analyze the forensic copy in an isolated environment:

- Run anti-malware/anti-virus scans
- Identify indicators of compromise (IOCs)
- Determine the initial infection vector
- Identify any lateral movement or persistence mechanisms

3.3 Review GuardDuty Findings

List recent high/medium severity findings

```
aws guardduty list-findings --detector-id \
```

```
--finding-criteria '{"Criterion":{"severity":{"GreaterThanOrEqualTo":4}}}'
```

Get finding details

```
aws guardduty get-findings --detector-id --finding-ids
```

GuardDuty findings provide remediation recommendations – follow them for each relevant finding.

3.4 Remove Malicious Artifacts

Based on forensic analysis:

- [] Remove any malware identified on infected instances
- [] Delete unauthorized IAM users, roles, policies, and access keys
- [] Terminate any attacker-created resources (EC2 instances, Lambda functions, etc.)
- [] Remove any persistence mechanisms (scheduled tasks, cron jobs, startup scripts)
- [] Revoke any modified trust policies or permission escalations

List all IAM users (look for unfamiliar)

```
aws iam list-users
```

List all roles (look for unfamiliar ones)

```
aws iam list-roles
```

Check for recently launched instances

```
aws ec2 describe-instances \
```

```
--filters Name=launch-time,Values=* \
```

```
--query 'Reservations[].Instances[].[InstanceId,LaunchTime,InstanceType,Tags]' \
```

```
--output table
```

MCP batch opportunity: the above checks

3.5 Close the Attack Vector

Based on root cause analysis:

- [] Patch exploited vulnerabilities on all potentially affected instances
- [] Update AMIs used in Auto Scaling launch configurations/templates
- [] Verify patches against the Mitre CVE database
- [] Block identified malicious IPs/domains in security groups or WAF rules
- [] Enforce IMDSv2 if SSRF was part of the attack chain

If eradication reveals a different attack vector (e.g., credential compromise, S3 data e

Enforce IMDSv2 on instances

```
aws ec2 modify-instance-metadata-options \
```

```
--instance-id \
```

```
--http-tokens required \
```

```
--http-endpoint enabled
```

```
---
```

Part 4: Recover from the Incident

Goal: Restore data and services from clean backups, or rebuild from known-good configurations

4.1 Identify Restore Points

List available EBS snapshots

```
aws ec2 describe-snapshots --owner-ids self \
```

```
--filters Name=volume-id,Values= \
```

```
--query 'Snapshots[][SnapshotId,StartTime,Description]' \
```

```
--output table
```

List available AMIs

```
aws ec2 describe-images --owners self \
```

```
--query 'Images[].[ImageId,CreationDate,Name]' \
```

```
--output table
```

List RDS snapshots (if applicable)

```
aws rds describe-db-snapshots \
```

```
--query 'DBSnapshots[].[DBSnapshotIdentifier,SnapshotCreateTime,Status]' \
```

```
--output table
```

Check S3 versioning for object recovery

```
aws s3api list-object-versions --bucket --prefix
```

MCP batch opportunity: the above check

Choose a restore point that predates the infection time established in Part 1.

4.2 Restore from Backups

For EC2 instances (from snapshot):

Create a new volume from a clean snap

```
aws ec2 create-volume \
```

```
--snapshot-id \
```

```
--availability-zone \
```

```
--volume-type gp3
```

Terminate the infected instance (or I

Launch a new instance with the restor

For S3 objects (from versioning):

Restore a previous version by copying

```
aws s3api copy-object \
```

```
--bucket \
```

```
--key \
```

```
--copy-source /?versionId=
```

Remove delete markers if objects were

```
aws s3api delete-object --bucket --key \
```

```
--version-id
```

For Auto Scaling groups:

Update launch template with clean AMI

```
aws ec2 create-launch-template-version \
```

```
--launch-template-id \
```

```
--source-version \
```

```
--launch-template-data '{"ImageId":""}'
```

Trigger instance refresh

```
aws autoscaling start-instance-refresh \
```

```
--auto-scaling-group-name
```

If no backup exists and decryption tools are available:

- Use identified third-party decryption tools in an isolated environment

- Validate decrypted data integrity before restoring to production
- Remove decrypted data from the infected instance, rebuild the instance, then restore

4.3 Rebuild if Necessary

If backups are unavailable and decryption is not possible:

- Rebuild from CMDB configuration documentation
- Redeploy from infrastructure-as-code (CloudFormation, CDK, Terraform)
- Restore application data from any available secondary sources (cross-region replicas,

4.4 Verify Recovery

- ☐ Restored data integrity confirmed (checksums, application-level validation)
- ☐ Affected applications functioning normally
- ☐ Monitoring dashboards show normal metrics
- ☐ No evidence of reinfection in CloudTrail or GuardDuty
- ☐ CMDB updated with any resource changes

If suspicious activity reoccurs, return to Part 1 and reassess.

Part 5: Post-Incident Activity

5.1 Document Lessons Learned

Create post-incident report covering:

- **Timeline:** When did each phase occur?
- **Root cause:** How did the ransomware gain entry? (unpatched vulnerability, phishing, cr
- **Ransomware strain:** What variant was used? Was decryption possible?
- **Impact:** What resources were affected? What data was lost or encrypted? Data classific
- **Response effectiveness:** What worked? What didn't? How long did each phase take?
- **Recommendations:** Process, tooling, and architecture improvements

5.2 Retrospective Questions

- What detection would have alerted to the ransomware sooner?
- Were backups adequate for recovery? Were they tested?
- What patching or configuration gap allowed the initial infection?
- Could network segmentation have limited the blast radius?
- What automation would have accelerated containment?

5.3 Update Defenses

Based on findings:

- [] Ensure all instances are patched and using current AMIs

- ☐ Enable/verify AWS Backup with appropriate retention policies
- ☐ Enable S3 Versioning and consider Object Lock for critical data
- ☐ Enforce IMDSv2 across all EC2 instances
- ☐ Review and tighten IAM policies (least privilege)
- ☐ Enable/enhance GuardDuty, Security Hub, and Amazon Inspector
- ☐ Implement network segmentation to limit lateral movement
- ☐ Deploy endpoint protection on EC2 instances
- ☐ Consider SCPs or permission boundaries for additional guardrails
- ☐ Propose updates to this playbook and related steering files based on lessons learned

5.4 Regulatory Notifications

If required by your jurisdiction:

- [] Notify relevant authorities within required timeframe

- [] Report to law enforcement agencies if applicable

- [] Document notification for compliance records

- [] Consider voluntary reporting to ransomware tracking databases or government agencies

References

- [AWS Security Incident Response Guide](#)
- [NIST SP 800-61 R3 – Incident Response Recommendations and Considerations for Cybersec](#)
- [AWS Well-Architected Security Pillar](#)
- [Amazon S3 Security Best Practices](#)
- [AWS Backup Documentation](#)
- [Amazon GuardDuty Documentation](#)
- [Amazon Inspector Documentation](#)
- [IMDSv2 for SSRF Mitigation](#)
- [AWS Systems Manager Patch Manager](#)
- [Querying CloudTrail Logs with Athena](#)
- [No More Ransom Project](#)

